



Elisabeth Egger, BSc

Investigation of routing possibilities for "Hike and Fly" within paragliding

Master's Thesis

to achieve the university degree of

Diplom-Ingenieurin

Master's degree programme: Geodäsie

submitted to

Graz University of Technology

Supervisor

Ao.Univ.-Prof. Dipl.-Ing. Dr.techn. Manfred Wieser

Institute of Geodesy

Graz, September 2022

Affidavit

I declare that I have authored this thesis independently, that I have not used other than the declared sources/resources, and that I have explicitly indicated all material which has been quoted either literally or by content from the sources used. The text document uploaded to TUGRAZonline is identical to the present master's thesis.



Date



Signature

Acknowledgements

First of all, I would like to express my thankfulness to my supervisor, Ao.Univ.-Prof. Dipl.-Ing. Dr.techn. Manfred Wieser, who allowed me to research in topic of personal interest. I am grateful for the trust he had in my vision and that he encouraged me to work independently.

Many thanks also to Primož Susa, who always lent me an ear when I had programming difficulties, no matter the time. He proved not only to be an amazing paragliding-pilot with whom I could discuss the achieved results but also a friend in need, when it came to set up a client-server architecture for the first time in my life.

I am also grateful to have such a loving family that supported me in every possible way, by allocating the kitchen table to myself while I was working on the thesis, listening to my paragliding and programming problems despite none of them being a pilot or just by giving me a hug, when something did not work out the way I wished for.

A big "Thank You" to all of my colleagues working at Flugschule Aufwind as well. They encouraged me to take days off to finish my thesis, even though we had an incredible workload at the time present.

But the biggest thanks goes to my little sister Chrissi who proofread this thesis and made me smile so many times with her comments about my writing style.

Abstract

Paragliding is a recreational and competitive art of flying a lightweight, foot-launched glider aircraft with no rigid primary structure. The small packing volume of a paraglider makes it possible to carry it in a backpack while hiking. The term "Hike and Fly" refers to the outdoor activity of hiking up a mountain with the paraglider in a backpack and then flying down with it. In the last years "Hike and Fly"-competitions have become increasingly popular and the athletes' levels accordingly. Therefore the need for route optimisation arose.

Within this thesis, algorithms were developed, which automatically detect take-off areas from where it is possible to launch with the glider aircraft. Furthermore, calculations were made to determine the topmost distance that can be covered from each take-off point. The provided routes were then added to the Open Street Map (OSM) street network. This graph network was manipulated so it could be used for navigational purposes. A cost-function was engineered, which shows the time needed to travel along a path or through the air. Three different routing tasks were implemented: Routing from A to B, Out-and-Return and a search for paragliding take-offs in the given area.

As final product a webpage was set up, which allows a pilot to calculate and plan optimized routes. The developed cost-function and its influence on routing results was tested against real data and shows great potential for future use throughout competitions, but also for amateur-pilots.

Kurzfassung

Paragleiten bezeichnet eine Luftsportart, bei der der Pilot mit einer fußstartfähigen Kappe aus Segeltuch fliegt. Die Kappe sowie das Gurtzeug können bei kleinem Packmaß einfach in einem Rucksack getragen werden und ermöglichen so "Hike and Fly" - eine Freizeitaktivität, bei welcher man auf einen Berg wandert, um dann von dort zu starten und hinunter zu fliegen. In den letzten Jahren wurden "Hike and Fly" Wettkämpfe immer populärer und somit eine optimale Routenwahl immer wichtiger.

Im Zuge dieser Masterarbeit wurde ein Algorithmus entwickelt, welcher automatisiert Start- und Landeplätze erkennt, und berechnet, welche Landeplätze von einem Startplatz aus erreichbar sind. Diese Flugstrecken wurden an das OSM Straßennetz angebunden, wobei die Daten für Navigationszwecke optimiert wurden. Des Weiteren entstand bei der Erarbeitung eine Kostenfunktion, die die Reisezeit entlang der unterschiedlichen Kanten innerhalb des Graphen-Netzwerkes berechnet. Anhand dieser Kosten wurde die Routenoptimierung bei drei unterschiedlichen Aufgabenstellungen implementiert: Routenoptimierung von A nach B, Berechnung der optimalen Out-and-Return Route und Suche nach umliegenden Startplätzen.

Als Endprodukt wurde eine Homepage erstellt, mit der Routenberechnungen durchgeführt werden können. Zudem wurden die berechneten Routen Echtzeiten gegenübergestellt. Die erlangten Ergebnisse verifizieren, dass eine Routenoptimierung für "Paragleit Hike and Fly" Wettkämpfe funktioniert und im Wettkampf einen Vorteil bringen kann.

Acronyms

AGL	Above Ground Level
ATC	Air Traffic Control
AVS	Alpen Verein Südtirol
CRS	Coordinate Reference System
CSV	Comma Seperated Values
CTR	Control Zone
DAV	Deutscher Alpen Verein
DEM	Digital Elevation Model
DHV	Deutscher Hänggleiterverband e.V.
DSM	Digital Surface Model
EPSG	European Petroleum Survey Group Geodesy
FBO	Flugbetriebsordnung für Hänge- und Paragleiter
GIS	Geographic Information System
GPL	General Public License
HTML	Hypertext Markup Language
JS	JavaScript
MGI	Militär-Geographisches Institut
ÖAV	Österreichischer Alpenverein
ODbL	Open Data Commons Open Database License

OSM	Open Street Map
PSQL	PostgreSQL
SQL	Structured Query Language
TRA	Temporary Reserved Airspace
WWW	World Wide Web
W₃C	World Wide Web Consortium
ERD	Entity Relationship Diagram

Contents

Abstract	iv
1. Introduction	1
1.1. Motivation	1
1.2. Paragliding And "Hike And Fly"-Competitions	2
1.2.1. Red Bull Xalps	3
1.2.2. Bordairrace	5
1.3. Research Aims	6
1.4. Status Quo	6
1.5. Outline	7
 I. Theoretical Foundations	 8
2. Paragliding	9
2.1. Aerodynamic Parameters	9
2.2. Parameters Needed For A Take-off	10
2.3. Parameters Needed For A Landing	12
3. Graph Theory	13
4. Routing Or Route Planning	17
4.1. Dijkstra	17
4.2. A*	20
4.3. Routing Options	22

II. Data and Tools	25
5. Data	26
5.1. Digital Elevation Model (DEM)	26
5.2. Land use Raster	27
5.3. Open Street Map	29
5.4. DHV-Geländedatenbank	29
5.5. Airspaces	30
5.6. National Parks	32
5.7. Test Area	33
6. Programs	34
6.1. ArcGIS	34
6.1.1. ArcGIS Toolboxes	34
6.2. QGIS	34
6.3. pgAdmin	35
6.4. Visual Studio Code	35
7. Programming Languages And Tools For Data Interfaces	36
7.1. Python	36
7.2. PostgreSQL	36
7.2.1. PostGIS	37
7.2.2. PgRouting	37
7.3. HTML5	37
7.3.1. JavaScript	38
7.4. Tools For Data Interfaces	38
7.4.1. Command Line Tools	38
7.4.2. express.js	39
III. Algorithms and Realisation	40
8. Algorithms	41
8.1. Preparatory Work	41
8.2. Take-Off Calculation	44
8.3. Landing Calculation	52
8.4. Calculation Of Glidepaths	55

Contents

8.5. Combination Of OSM Data And Glidepaths	59
8.6. Calculation Of Ascent And Descent	59
8.7. Data Migration Into The PostgreSQL Database	64
8.8. Determination Of Cost	67
8.8.1. Hiking	69
8.8.2. Flying	70
8.9. Setting Of Restrictions	70
8.10. Calculating The Closest Vertex	74
8.11. Xalps - Routing (A To B)	74
8.12. Bordairrace - Routing (Out And Return)	77
8.13. Routing To Reachable Take-Offs	81
9. Realisation	85
9.1. Webpage: Hike And Fly Routing	85
 IV. Results	 95
 10. Routing Graph	 96
 11. Test Cases	 98
11.1. A To B	98
11.2. Out And Return	109
11.3. Take-Offs Around	110
 12. Conclusio And Outlook	 114
 Bibliography	 116

List of Figures

1.1. Paragliding Macedonia	2
1.2. Red Bull Xalps 2021 Route	3
2.1. Glide ratio	10
2.2. Perfect take-off	11
2.3. Wind flow	12
3.1. Basic graphs	14
4.1. Search tree breadth-first	21
4.2. Search tree depth-first	22
5.1. DEM Austria	26
5.2. Landuse Austria	28
5.3. OSM Website	29
5.4. Testarea Salzburg (graphic by the author)	33
8.1. Development levels	42
8.2. Raster interpolation	43
8.3. Data import	43
8.4. Flow Chart: Take-off calculation	45
8.5. Evaluation of take-off pixel	46
8.6. Flow Chart: Inverse Flow Analysis	47
8.7. Flow Chart: Vector based check of obstacles	49
8.8. Take-off evaluation, vector based	50
8.9. Flow Chart: Raster based check of obstacles	51
8.10. Flow Chart: Raster based check of obstacles	53
8.11. Landings	54
8.12. Flow Chart: Glidepath calculation	56
8.13. Above Ground Level (AGL)	57

List of Figures

8.14. Shortest path with or without obstacles	57
8.15. AGL calculation	58
8.16. Flow Chart: Assembly of the different network parts	60
8.17. Schematic overview about the different vectors in the routing graph	61
8.18. Flow Chart: Ascent and descent calculation	62
8.19. Schematic overview stack profile	63
8.20. Ascent and descent calculation	63
8.21. Data migration	65
8.22. Entity Relationship Diagram (ERD)	66
8.23. Flow Chart: Determination of cost	68
8.24. Cost function hiking	69
8.25. Flow Chart: Set restrictions	71
8.26. Flow Chart: Get closest vertex	75
8.27. Flow Chart: Xalps - Routing (A to B)	76
8.28. Flow Chart: Bordairrace - Routing (Out and Return), part 1	78
8.29. Flow Chart: Bordairrace - Routing (Out and Return), part 2	80
8.30. Out and return	81
8.31. Flow Chart: Take-off extraction	83
8.32. Flow Chart: Routing to reachable take-offs	84
9.1. Webpage: Main window	86
9.2. Webpage: Main window with route	87
9.3. Webpage: Route close up	88
9.4. Cost parameters	89
9.5. Webpage: Data flow for cost calculation	90
9.6. Webpage: Set restrictions	91
9.7. Webpage: Parameters Bordairrace-Routing	92
9.8. Webpage: Parameters routing to take-offs	92
9.9. Webpage: Flow Chart route calculation	93
9.10. Webpage: Main window with routes to take-offs	94
11.1. Map of turnpoints	99
11.2. Results: Route TP0 to TP1	103
11.3. Comparison of real tracklog vs. calculated path	104
11.4. Results: Route TP1 to TP2	106
11.5. Results: Route TP1 to TP2 when unflyable	108

List of Figures

11.6. Results: Out and Return	110
11.7. Results: Take-offs around Dorfgastein	112
11.8. Results: Selected Take-offs around Dorfgastein	113
11.9. Results: Hike and fly options around Dorfgastein	113

List of Algorithms

4.1. Shortest-path algorithm based on Dijkstra's approach	19
4.2. A* algorithm based on Dijkstra's algorithm	23
8.1. Calculation of the ascent and descent along a given path	61
8.2. Calculation of the wind factor	72
8.3. Re-Calculation of the cost for glidepaths in respect to a given wind situation	73

1. Introduction

This chapter gives a short overview about the master's thesis *Investigation of routing possibilities for "Hike and Fly" within paragliding*. First, the fundamental motivation will be discussed. Second, the basic principle of paragliding "Hike and Fly" competitions will be explained. Furthermore the research aim will be explained. And finally, the structure of this thesis is stated.

1.1. Motivation

Since 2010 I have been a passionate paragliding pilot. The thrill of losing the ground beneath my feet and launching into the air without any type of engine has fascinated me ever since my first flight. In 2019 Aaron Durogati, a two-times paragliding worldcup champion, asked me to support him at the Red Bull Xalps. During the race it was my job to find the next take-off and the fastest route up and to calculate how far he could glide down from that specific peak, land and immediately continue hiking on the best route. Collecting all the necessary data on the possible paths, finding grassy hillsides with the right aspect (direction of the hillside) and slope (angle of descent) for the current weather and wind situation is time-consuming. During the race, this decision-making process can be incredibly stressful and options must be evaluated within minutes. The immense pressure can easily lead to wrong decisions, unnecessary detours, and a worse result at the end of the day. All those issues and the experiences that I have made in 2019 have motivated me to investigate the options of automatization of the evaluation of possible take-offs, the reachable distance around the take-off and also the optimization of the route on the ground and in the air. In some cases a glide of 100 meters across a river can save five kilometers of hiking to the next bridge. As such an event would be inefficient and a

loss of time, an automatic evaluation of strategically advantageous take-offs, hiking distances and optimal routes would minimize these issues and allow athletes to further advance their performances.

1.2. Paragliding And "Hike And Fly"-Competitions

The recreational and competitive art of flying a paraglider, a lightweight, foot-launched glider aircraft with no rigid primary structure, is called paragliding. The picture 1.1 shows me flying at the World Championship 2019 in Macedonia. Paragliding is characterized by the use of an inflatable



Figure 1.1.: Myself flying a paraglider during the World Championship 2019 in Macedonia
(Photographer: Ewa Korneluk-Guzy)

canopy made out of nylon foil, which can be easily folded into a small backpack on the ground. Having an aircraft folded within a backpack of 60l or less volume introduces a lot of possibilities. The average equipment for so-called "Hike and Fly" trips, where the glider and harness are carried up the mountain before flying down, weights around seven to ten kilograms. Thus, more and more pilots prefer hiking over the use of a cable car, as it combines physical activity and the freedom of getting airborne. Within "Hike and Fly" races, the competitors need to cover a certain distance just by

1. Introduction

foot or the use of a paraglider. The usage of any other type of transportation like cars, bikes or cable cars is forbidden. The two most popular races in Austria are the Red Bull Xalps (see also 1.2.1) and the Bordairrace (to be explained in section 1.2.2).

1.2.1. Red Bull Xalps

The Red Bull Xalps is named the world's toughest adventure race. Only by foot or paraglider, 33 athletes from around the globe take up the challenge to cover a straight-line distance of more than 1000 km across the Alps. To win the race, they must find the fastest way to the goal via several fixed turnpoints, often covering more than 100km per day. The route for the 2021 edition is shown in Figure 1.2.



Figure 1.2.: Route of the 2021 edition of the Red Bull Xalps (Source: [1])

1. Introduction

Number	Name	Latitude	Longitude	Classification
0	Salzburg	47.798850	13.048400	Start
1	Gaisberg	47.804133	13.110917	Signboard
2	Wagrain	47.274628	13.318581	Signboard
3	Kitzbühl Streif	47.426461	12.371147	Signboard
4a	Chiemsee	47.858077	12.500269	Radius 3000m
4b	Marquartstein	47.767503	12.457437	Signboard
5a	Zugspitze	47.421063	10.985517	Passed north
5b	Leermoos	47.401283	10.879767	Signboard
6	Säntis	47.249365	9.343238	Radius 2000m
7	Fiesch	46.40940	8.13688	Signboard
8	Dent d'Oche	46.352357	6.731626	Counterclockwise
9	Mont Blanc	45.830359	6.867674	Counterclockwise
10	Piz Palü	46.378200	9.958730	Radius 3500m
11	Kronplatz	46.737598	11.954900	Signboard
12	Schmittenhöhe	47.328744	12.737518	Signboard
13	Zell am See	47.325290	12.801694	Finish

Table 1.1.: Order, name, coordinates and classification of the turnpoints of the Red Bull Xalps 2021

The turnpoints must be reached in the given order, starting from the Mozartplatz in Salzburg until the goal in Zell am See. At the different turnpoints, the athletes either have to sign a board, have to enter a surrounding cylinder around given coordinates or in some occasions must only pass it anticlockwise. Table 1.1 shows the order, the coordinates of the turnpoints, as well as their classification. For example, at the forth turnpoint the athletes have to first enter a three kilometre cylinder located around the coordinates of the Chiemsee. This can be done either by flying or by hiking. Afterwards even the ones, who make the turnpoint 4a in the air, have to land at Marquartstein (number 4b) to sign a board. After signing the athletes can choose their way to the next turnpoint independently, trying to take the fastest way by either hiking or flying.

This race combines endurance hiking, trail running, mountaineering, and paragliding skills. Next to the competitor's physical strength and mental fortitude, strategic race management is an essential part for a successful

race. Therefore, the competitor is supported by at least one other person, the main supporter, or a whole team. This team is responsible for the physical well-being of the athlete, including cooking and bodily treatments like massages as well as mental support and route planning on the ground and in the air. Depending on the prevailing weather situation, it must be decided where and when to take off and fly or whether to stay on the ground and cover some distance through hiking. Whenever the athlete is in the air, the support team follows him/her on the ground, while simultaneously planning the next steps.

1.2.2. Bordairrace

Another type of a "Hike and Fly" race is the so called *Bordairrace*. Within a set time window of 33 hours, the competitors must hike and fly as far away as possible and return to the start before the time runs out. In order to win this race, a pilot has to set his return-point at the maximum distance from the start and make it back in time. As there is no pre-set turnpoint each competitor must decide for him/herself in which direction he or she heads out, and what the most promising plan is. This results in athletes spreading in all directions after the start. The further they go the longer the way back. So they need to keep an eye on the time left when they decide to turn back, as they should come back to the starting point within the 33 hours to get the homecoming bonus of 20%. The result calculates as the straight line distance to the furthest point away d_{out} where they turned back home, and the straight line distance from the turnpoint, to the point where they finish their race d_{return} . If they make it back to the start, d_{out} equals d_{return} , if not the remaining distance from where they stop to the start $d_{backtostart}$ is subtracted from the result.

$$result = (d_{out} + d_{return} - d_{backtostart}) + 20\%_{Home-Coming-Bonus}. \quad (1.1)$$

For getting a home-coming bonus of 20% it does not matter if the athlete returns early or just one or two minutes before the end of the race. But to win the competition it is often crucial to make it back home, so athletes tend to turn back rather too early to be on the safe side.

1.3. Research Aims

This thesis aims to develop a tool to optimize the routes during paragliding competitions. Those routes include the paths on the ground as well as flightpaths through the air. For the calculation of flightpaths, an algorithm will be developed, which will be able to detect optimal take-offs and landings. Furthermore, the reachable landings from each take-off will be computed and glidepaths will be generated and included in the street network. An additional algorithm will also be implemented to determine the ascent and descent for each path. All these vector paths will be included in a PostgreSQL database for routing calculations. Moreover, the costs for flying and hiking will be determined and the possibility of adding routing restrictions (for example, an impossible take-off due to hindering weather conditions) will be included. Corresponding to the two different types of "Hike and Fly" competitions, various routing options will be investigated. Finally, a webpage will be created which will offer the calculation tool for optimized routes and their display.

1.4. Status Quo

Currently, there is no tool available which combines both, the hiking and the flying component. Tools like BRouter [2] or the tourplanner of Alpenvereinaktiv [3] by the Deutscher Alpen Verein (DAV), the Österreichischer Alpenverein (ÖAV) and the Alpen Verein Südtirol (AVS) offer good navigational aid on the ground but do not take glidepaths into account. The swiss hike and fly planer for paragliders [4] makes it possible to determine the reachable distance around a point on the map but does not include any routing. It also does not check whether the point you clicked on on the map is actually in proper condition for a take-off. Other sites like ParaglidingMap [5] show official take-offs and whether the weather at the chosen point is currently flyable. During, for example the Red Bull Xalps, the athletes and supporters need to examine all those sources and additionally check satellite images and DEM in search for another unofficial take-off. When all the available information has been examined, they need to decide on

the best-looking option relying on their experiences. The chosen routes are often not optimized in their entirety, in the best case, the shortest path up to the take-off is calculated and the part which is covered through flying, is ignored. For a beginner it is therefore quite difficult to plan ahead.

1.5. Outline

This thesis consists of four parts. The first part deals with the theoretical foundations and is divided into three chapters: paragliding, graph theory and routing. The first gives a short introduction into paragliding, the second chapter presents the fundamentals of graph theory and the third chapter explains the basic principles of routing. In the second part of the thesis the data and tools used for developing and conducting research are listed. The first chapter introduces all the data used. The programs used for the development are introduced in chapter 6 and the last chapter gives basic information about the programming languages and used tools for the data interfaces. In the chapter 8 Algorithms of the third part the different calculation steps are outlined and in chapter 9 the realisation, a webpage as user interface, is introduced. The final part deals with the results and interpretation as well as the conclusion and provides an outlook for potential future fields of research.

Part I.

Theoretical Foundations

2. Paragliding

2.1. Aerodynamic Parameters

Glide Ratio

The glide performance or glide ratio of a paraglider is determined by the ratio of lift to drag. It can also be expressed as ratio of vertical to horizontal distance covered or the descent compared to the forward movement as shown in 2.1.

$$\frac{lift}{drag} \hat{=} \frac{d_{vertical}}{d_{horizontal}} \hat{=} \frac{v_{descent}}{v_{forward}} \quad (2.1)$$

For an average EN-A (certified after the DIN EN 926-2:2015-06 [6]) glider for beginners with a trim speed of $36km/h$ and a sink of $1.4m/s$ (corresponding to the Deutscher Hängegleiterverband e.V. (DHV) flight instructor examination sheet [7], part aerodynamic) this means

$$glide\ ratio = \frac{1.4[\frac{m}{s}]}{10[\frac{m}{s}]} = \frac{1}{7.14}. \quad (2.2)$$

As a result for every meter of sink, the paraglider covers a horizontal distance of $7.14m$. The actual glide ratio of a paraglider depends on various factors, such as the model or canopy shape, the smoothness of the air, the harness of the pilot, air-pressure, humidity or specifically the wind as can be seen in figure 2.1.

In the blog, published by Flybubble [9], the calculated glide ratio of paragliders is between $\frac{1}{7}$ and $\frac{1}{11}$ in smooth air. These calculations do not include the drag, which is produced by the harness or the pilot himself. In this master's thesis an average glide ratio of 7 is assumed. This guarantees that

2. Paragliding

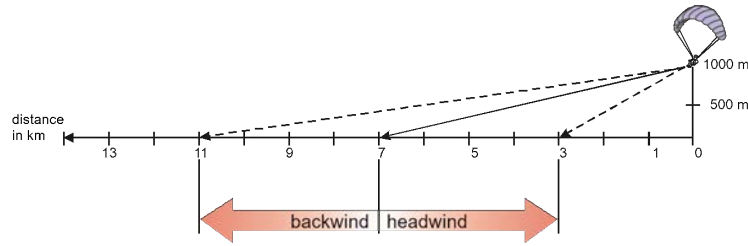


Figure 2.1.: Glide ratio of a paraglider with backwind or headwind [8]

the calculated glidepaths are possible for all types of gliders in smooth air. The glide angle is calculated as

$$\alpha = \arctan \frac{d_{vertical}}{d_{horizontal}} = \arctan \frac{1}{7} = 8,13^\circ \quad (2.3)$$

and describes the angle between the glide path and the ground.

2.2. Parameters Needed For A Take-off

For taking off with a paraglider, the canopy must first be filled with air. By taking several steps (depending on the headwind), the canopy will be pulled above the pilot's head. Only then the pilot is able to take off. As the pilot is approximately 7 to 9 m below the canopy (length of the lines) and the gliders have a flat area of about 16 to 30 m² by a wingspan of 10 to 14 m, a minimum take-off area of 112 m² is necessary.

With a glide angle of about 7 – 8°, the slope of the take-off field must be steeper, but not exceed 30° as this would be too steep for a safe take-off due to the risk of slipping.

The take-off direction must align with the aspect of the take-off area. In this master's thesis, take-offs are classified into eight take-off directions namely North, Northeast, East, Southeast, South, Southwest, West and Northwest. It must also be kept in mind that taking off is only possible with appropriate wind conditions. This includes headwind which must come from in front of

2. Paragliding

the pilot within a 45° angle from the intended take-off direction. The wind speed must be within the minimum criteria for a safe take-off and landing [10] or as the Flugbetriebsordnung für Hänge- und Paragleiter (FBO) by the DHV writes, the wind must be below two thirds of the maximum speed of the aircraft. For a beginner's paraglider with a maximum speed of approximately 45 km/h the maximum take-off wind is 30 km/h . Therefore in this master thesis a glide is considered as impossible if the wind exceeds 30 km/h . Perfect conditions are with a light headwind of 5 to 15 km/h .

To be useful in route optimization, a take-off must also be close to an existing road or track. Additionally, as a paragliding pilot it is mandatory to follow the rules of air. This means that a paragliding pilot is only allowed to fly in airspaces class G or E [11], and in class D only with an Air Traffic Control (ATC) clearance which is usually not given. Besides these restrictions, national parks must also be taken into account as zones in which it is forbidden to take-off or land with a paraglider. The last, but most important criteria is the absence of obstacles. The best take-off area is useless if you crash into a tree or building right after taking off. The photograph 2.2 taken by the author shows what an optimal take-off would look like.



Figure 2.2.: Perfect paragliding take-off site

2.3. Parameters Needed For A Landing

For landing a paraglider, a flat area without obstacles is needed. This includes a slope with less than 3° and a safety distance to possible obstacles within 30 m. The safety distance is needed because wind can cause lee-turbulences behind objects, for example a tree or building, which can endanger the pilot during landing. Turbulences can occur up to ten times the obstacle height behind it, depending on the wind strength. The illustration 2.3 [8] shows such possible turbulences on the wind-sheltered or lee side of an obstacle. Due to the low wind speed in which a paragliding pilot is flying when approaching, a safety distance of 30 m to the next obstacle is set as standard in this master's thesis.

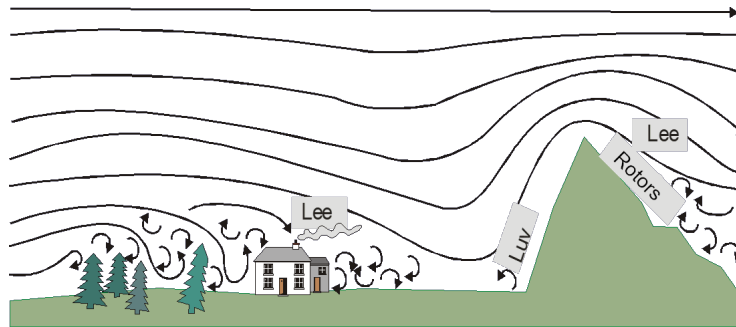


Figure 2.3.: Influence of obstacles on the wind flow [8]

3. Graph Theory

In order to model real world navigational road networks, the so called graph theory is used. Graph theory deals with node-edge structures mathematically, and is used in various contexts such as in electronics, informatics or in navigational routing either on land, at sea or in the air. A simple definition of a graph can be found in [12, p. 1]:

"A graph is a collection, or set, of very simple objects, namely, a set of line segments terminated by dots as depicted by Figs. 1.1.1(a) and 1.1.1(b). These line segments and dots are the sole objects of concern in the graph and have no properties other than their visual objectivity. No line length, or curvature, or point content, or position of line segments is considered significant."

Figure 3.1 shows the mentioned illustration. As Lee M. Maxwell states, each graph consists of dots also called nodes or vertices v and lines often referenced as edges e . Graph theory was first used by the Swiss mathematician Leonhard Euler to solve the problem of "the bridges of Königsberg". In this case, a route across all seven bridges had to be found which does not use one bridge twice. With the help of graph theory - he modelled the watersides and isles as nodes and the bridges as edges - Euler was able to theoretically prove that such a route is not possible [13]. Some important properties of a graph, if used in a navigational context, are explained in the next sections.

Finite Graph

In a model of the real world a graph is always going to be finite, with a finite number of edges and nodes. If it is a very large graph, it will be split up in subgraphs to reduce the computation-time when calculating a route.

3. Graph Theory

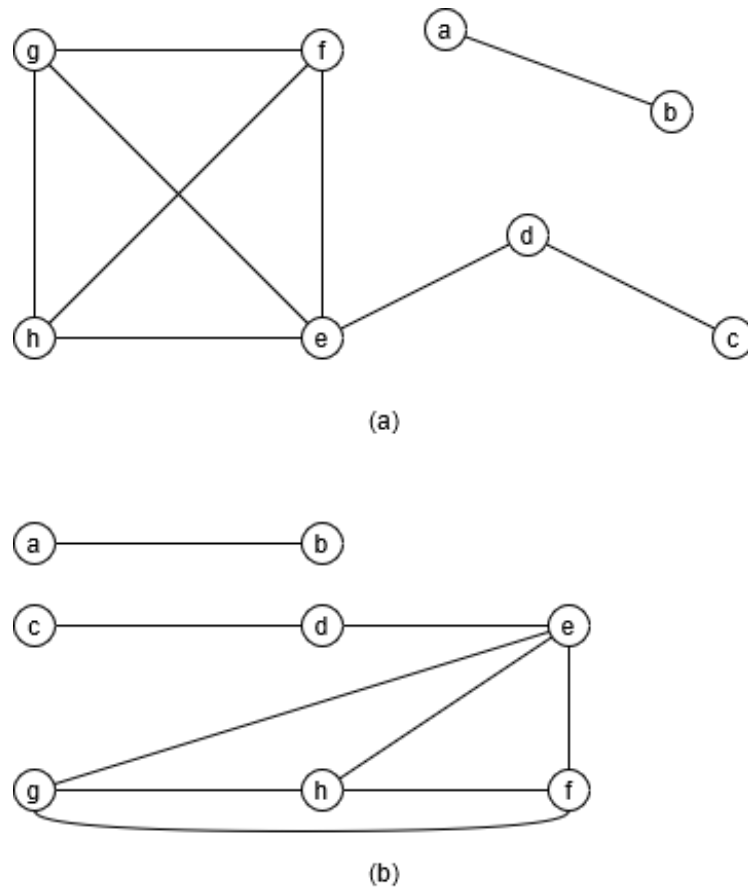


Figure 3.1.: Basic graphs as explained in [12, p. 1]

Irregular Graph

The real navigational networks consist for example of intersections with three roads ("T-crossing") or classical crossings with four meeting roads. The first intersection would be represented as a node with three incident edges (degree three) and the second as a node with four incident edges (degree four). As there occur different degrees within the graph it is called an irregular graph.

Planar Graph

A graph is stated as planar, if you can draw it in 2D and no edges overlap without a node at every intersection. Another term for a planar graph is *noded network* as at every intersection there is a node. This means that at every intersection of the real street network, a node is inserted and the roads get split up into separate road segments. In the case of tunnels or bridges across another street within a network it turns into a non-planar graph. Glidepaths which lead through the air from a take-off to a landing act as bridges across other streets, paths or tracks. The used graph or network in this master thesis is partially noded on all ground intersections (except bridges and tunnels) but aerial crossings are not noded.

Directed Graph

If a graph consists of edges with a specified direction, so called *arcs*, it is called a directed graph [14, pp. 85-87]. In a directed graph the connection or arc a_{AB} from vertex A to B is different from the arc a_{BA} from B to A . This type of graph is an ideal choice for many road networks as one-way options can be modelled. Within this master thesis, for example, glide-paths are only usable in one direction, namely from the take-off to the landing.

Valuated Graph

In a network of arcs a cost function $c(a_{ij})$ assigns a real valuation number c_{ij} to an arc a_{ij} . This valuation number or cost describes the effort needed to travel over this arc. It can either be the geometric distance, or the travelling time across this specific arc, or any other measure which does not need to have a direct connection to the geometry of the arc. To model roads which should be avoided when hiking (for example highways) high cost are assigned to the arc. Generally, the higher the costs the more unlikely it is to use this path. In some cases, the direct route is not the optimal one as can be seen in equation 3.1, where the direct costs c_{ik} from vertex v_i to vertex v_k are higher than the sum of the costs c_{ij} and c_{jk} when taking the route from v_i via v_j to v_k .

$$c_{ij} + c_{jk} < c_{ik} \quad (3.1)$$

4. Routing Or Route Planning

Routing on the one side describes the process of finding a singular route (the shortest or optimal path from a start-vertex v_A to a destination vertex v_B , as needed in the Red Bull X-Alps 1.2.1). On the other side also more complex tasks like finding the best out-and-return option (as searched for in a Bordairrace 1.2.2) is another question within routing. The first routing-problem would be referred to as a single-pair shortest path problem (detailed information can be found in section 4.3), the second problem must be examined in two steps: firstly it raises the need to construct a spanning tree out from one source node, a so called single-source shortest path problem [14] (which is explained in section 4.3), secondly, to find the shortest path from anywhere to a fix destination the routing-problem is called a single-destination problem and is discussed in section 4.3. Within this master thesis, two routing algorithms are used: Dijkstra (see 4.1) and A* (see 4.2).

4.1. Dijkstra

One of the most famous routing algorithms is the one designed by E.W.Dijkstra in 1959 [15]. It calculates the shortest paths from one node v_s to every other node in a graph (which consists of nodes and vertices with positive costs c_{ij}). Within the search algorithm either permanent or temporary labels are assigned to the nodes. A set of nodes with a permanent label is defined as P while the set of nodes with a temporary label as T [14, p. 303]. The label l_j indicates the costs or distance from the start node v_s to the destination node v_j . Within the search process it is running through some temporary values until it is set to a permanent value. This equals the shortest path distance d_{sj} . The node sequence along the path can be reconstructed using predecessor

4. Routing Or Route Planning

Table 4.1.: Symbols for pseudocode

Symbol	Explanation
V	Set of all nodes
T	Set of nodes with a temporary label
P	Set of nodes with a permanent label
N	Set of adjacent nodes (neighborhood)
v_s	Start node
l_j	Distance label of a node v_j
p_j	Predecessor of a node v_j
c_{ij}	Cost of an arc a_{ij}
f_j	Total evaluation number of a node v_j
h_j	Heuristic evaluation number of a node v_j
$:=$	Assignment to a label or node
$\leftarrow$	Assignment to a set
$\{x\}$	Set consisting of a node x
$\{\}$	Empty set
$X \setminus \{x\}$	Set X except node x
$X \cup \{x\}$	set X plus node x
$X \leftarrow X \setminus \{x\}$	Removing node x from set X
$X \leftarrow X \cup \{x\}$	Adding node x to set X
$\in$	Element of
$\notin$	Not element of
$\wedge$	Logical "and"

nodes, where p_j is the predecessor of v_j . Algorithm 4.1 shows the pseudo code of the Dijkstra algorithm with some improvements in computational efficiency. Table 4.1 gives an explanation on the symbols used.

Before entering the for-loop the label of the start node is set to zero, as the distance to itself is zero. Then the start node v_s is added to the temporary set of nodes. For all the other nodes the label l_j is set to infinity (see lines 4 and 5). As long as the temporary set of nodes T is not empty the steps 8 to 21 are computed. At first, the node with the minimal temporary cost is searched within the set T , and in the next steps is added to the permanent

Algorithm 4.1 Shortest-path algorithm based on Dijkstra's approach

```
1:  $l_s := 0$ 
2:  $T \leftarrow \{v_s\}$ 
3:  $P \leftarrow \{\}$ 
4: for  $v_j \in V \setminus \{v_s\}$  do
5:    $l_j := \infty$ 
6: end for
7: while  $T \neq \{\}$  do
8:    $v_i := v_j \in T$  with min.  $l_j$ 
9:    $P \leftarrow P + \{v_i\}$ ;
10:   $T \leftarrow T \setminus \{v_i\}$ 
11:  for  $v_j \in N(v_i)$  do
12:    if  $(v_j \notin T) \wedge (v_j \notin P)$  then
13:       $l_j := l_i + c_{ij}$ 
14:       $p_j := v_i$ 
15:       $T \leftarrow T + \{v_j\}$ 
16:    end if
17:    if  $(v_j \in T) \wedge (l_i + c_{ij} < l_j)$  then
18:       $l_j := l_i + c_{ij}$ 
19:       $p_j := v_i$ 
20:    end if
21:  end for
22: end while
```

set and deleted from the temporary one (line 9 and 10). For every node v_j in the neighborhood $N(v_i)$ of the node v_i the steps 12 to 21 are taken. A vertex or node is in the neighborhood, if an arc a_{ij} exists which directly connects the nodes v_i and v_j . Two different options can be performed then: either step 12 to 16 or 17 to 20.

The first option is performed, if the node v_j is not part of the set of nodes with a temporary or a permanent label. Then the label l_j is set to the sum of the predecessor node's label l_i and the cost c_{ij} from node v_i to v_j . The node v_i is set as the predecessor of v_j . v_j is in the last step added to the set of nodes with a temporary label T .

The other option (step 17 to 20) is performed, if the node v_j is part of T and the cumulated costs $l_i + c_{ij}$ are smaller than the current label l_j . The label l_j gets corrected and the predecessor is set to v_i .

Once the algorithm is finished, a permanent label will be assigned to all nodes of the digraph with a possible path from the start node v_s . All other nodes remain with a label set to infinity (which was set in step 5). The label represents the shortest path distance. The shortest path itself is obtained by traversing the predecessors starting from node v_j back to v_s .

If only the shortest path from one starting point v_s to one specific end point v_d is needed, the while loop can be interrupted as soon as v_d gets a permanent label.

This algorithm hence generates a tree which grows iteratively by adding one labelled node after the other radially around the start node. This is then called a breadth-first search [16]. The resulting search-tree expands symmetrically around the start node (as shown in figure 4.1).

In contrast to a breadth-first search strategy, it is often wanted that the search tree grows predominantly into a certain destination and is thus much closer to a shortest path result. Such a search is called a depth-first search and is often used within the A^* (pronounced A star) algorithm (section 4.2).

4.2. A^*

With the Dijkstra algorithm as a basis, the A^* algorithm can easily be implemented as shown in the pseudo code algorithm 4.2. All modified or

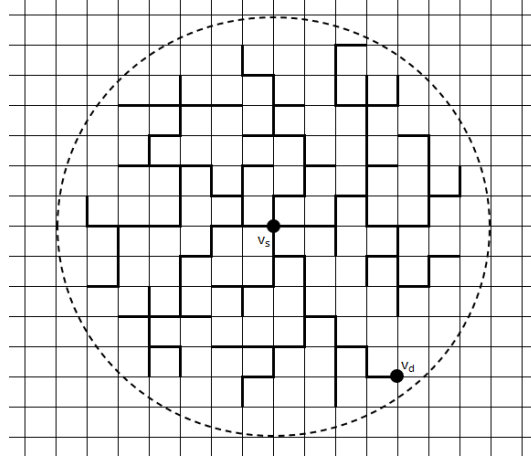


Figure 4.1.: Search tree of a breadth-first search

added parts in the code are marked with a "*" (the pseudocode symbols are listed in table 4.1). The biggest difference between the two algorithms lies in the decision, which node is set to a permanent label next (step 9). Instead of the one with the smallest distance label l_j , it is the one with the smallest total evaluation number f_j . This total evaluation number calculates as

$$f_j = l_j + h_j \quad (4.1)$$

whereas h_j is the heuristic evaluation number of a node v_j . When using an A* algorithm, it is crucial to find the right heuristic evaluation number. This number must improve the time complexity of the algorithm while keeping the risk of a suboptimal path as low as possible. One option for a heuristic evaluation number is shown in equation 4.2.

$$h_j = \sqrt{\Delta x^2 + \Delta y^2} \quad (4.2)$$

h_j is calculated as the straight distance between two points a and b with

$$\Delta x = x_b - x_a \quad (4.3)$$

and

$$\Delta y = y_b - y_a. \quad (4.4)$$

Including these heuristics leads to a stretched search tree towards the destination node v_d as seen in figure 4.2. Besides the risk of getting a

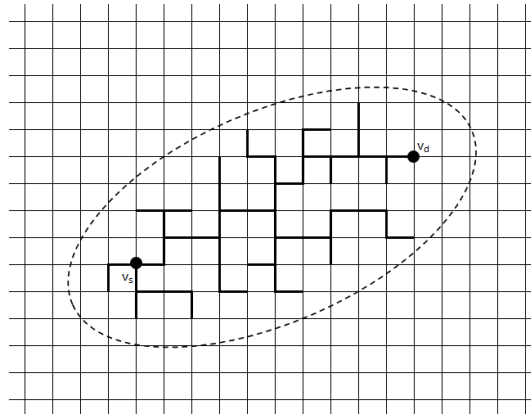


Figure 4.2.: Search tree of a depth-first search

suboptimal solution, the original label-setting procedure becomes a label-correcting procedure (steps 24 to 30). Within these steps it might occur, that an already permanent label needs to be set to a temporary again.

4.3. Routing Options

One-To-One

The most common problem in navigation is the single-pair shortest path problem from a vertex v_A directly to a vertex v_B . If the route from v_A to v_B has to bypass a given set of nodes in between (for example v_X, v_Y and v_Z) in a given order, it is called a sequential shortest-path problem [14, p. 301]. A sequential shortest-path problem consists of several single-pair shortest path problems in a row (v_A to v_X , v_X to v_Y , v_Y to v_Z and v_Z to v_B) whereas the costs accumulate. This type of problem is given during the Red Bull X-Alps 1.2.1, where the athletes must complete the turnpoints in a given order.

Algorithm 4.2 A* algorithm based on Dijkstra's algorithm

```

1:  $l_s := 0$ 
2:  $T \leftarrow \{v_s\}$ 
3:  $P \leftarrow \{ \}$ 
4:  $f_s := l_s$  ▷ *
5: for  $v_j \in V \setminus \{v_s\}$  do
6:    $l_j := \infty$ 
7:    $f_j := \infty$ 
8: end for
9: while  $T \neq \{ \}$  do ▷ *
10:   $v_i := v_j \in T$  with min.  $f_j$ 
11:   $P \leftarrow P + \{v_i\};$ 
12:   $T \leftarrow T \setminus \{v_i\}$ 
13:  for  $v_j \in N(v_i)$  do
14:    if  $(v_j \notin T) \wedge (v_j \notin P)$  then
15:       $l_j := l_i + c_{ij}$ 
16:       $p_j := v_i$ 
17:       $f_j := l_j + h_{ij}$  ▷ *
18:       $T \leftarrow T + \{v_j\}$ 
19:    end if
20:    if  $(v_j \in T) \wedge (l_i + c_{ij} < l_j)$  then
21:       $l_j := l_i + c_{ij}$ 
22:       $p_j := v_i$ 
23:       $f_j := l_j + h_j$  ▷ *
24:    end if
25:    if  $(v_j \in P) \wedge (l_i + c_{ij} < l_j)$  then ▷ *
26:       $l_j := l_i + c_{ij}$  ▷ *
27:       $p_j := v_i$  ▷ *
28:       $f_j := l_j + h_j$  ▷ *
29:       $P \leftarrow P \setminus \{v_i\};$  ▷ *
30:       $T \leftarrow T + \{v_i\};$  ▷ *
31:    end if
32:  end for
33: end while

```

One-To-Many

The One-to-Many problem, also called single-source shortest path problem, analyses a graph in search for the shortest path from one source to many or all others. That is exactly what is asked for at a Bordairrace (see 1.2.2), where all the options in any direction need to be investigated from one specific starting point.

Many-To-One

For the inclusion of the return time to the starting point in a Bordairrace, the Many-to-One or single destination problem is also part of this master thesis. In a Many-to-One situation, the shortest paths from anywhere to a single destination are calculated.

Part II.

Data and Tools

5. Data

The calculations done within this master's thesis rely on several different data sets as described on the next pages.

5.1. Digital Elevation Model (DEM)

The information about the elevation, slope and aspect is taken from the digital elevation model of Austria as shown in picture 5.1[17].



Figure 5.1.: Digital 10m elevation model from airborne laser scanning data Austria [17]

Published by Geoland.at[18] this dataset is licenced under *CC-BY-4.0: Land Kärnten - data.gv.at* and can be found in the Open Data Österreich database[17]. It contains the elevation of Austria in a resolution of ten-times-ten metres. More detailed information can be found in table 5.1.

5. Data

Table 5.1.: Statistics of the DEM Austria

Projection	EPSG-Code: 31287
Minimum	111,98 m
Maximum	3795,41 m
Pixelsize	10 x 10 m

The European Petroleum Survey Group Geodesy (EPSG)-Code 31287 describes the Coordinate Reference System (CRS) and the projection used. Specifications for the EPSG-Code can be found in table 5.2.

Table 5.2.: Specification of EPSG:31287

Unit	metre
Geodetic CRS	Militär-Geographisches Institut (MGI)
Datum	MGI
Ellipsoid	Bessel 1841
Prime meridian	Greenwich

5.2. Land use Raster

Figure 5.2 shows the land use of Austria and Lichtenstein with a resolution of ten-times-ten metres.

This information is gained from the satellite sensor data of Sentinel-2A L2A (BoA) during the time period 2015-12-08 to 2017-05-21. It is licensed under *Creative Commons Namensnennung 3.0 Österreich Lizenz* and published by the Umweltbundesamt GmbH[19]. Like the DEM it is in the reference system EPSG 31287. Each pixel is classified as either

- Build-up,
- Flat sealed surfaces,

5. Data

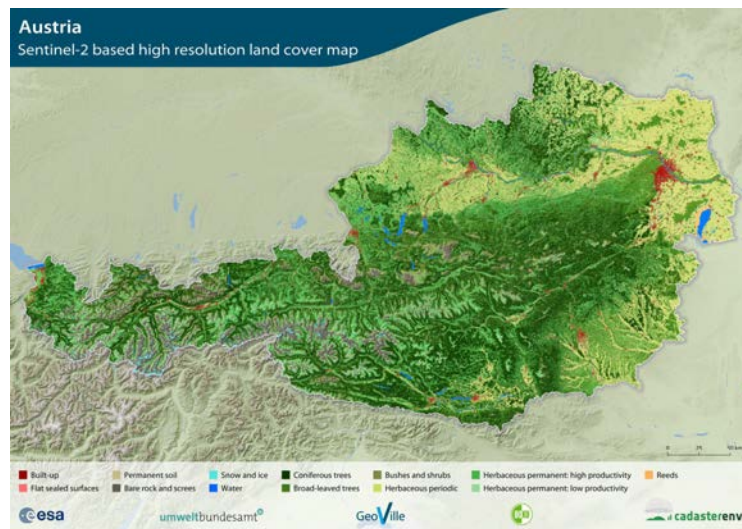


Figure 5.2.: High Resolution Land Cover Map Austria[19]

- Permanent soil,
- Bare rock and scree,
- Snow and ice,
- Water,
- Coniferous trees,
- Broad-leaved trees,
- Bushes and shrubs,
- Herbaceous periodic,
- Herbaceous permanent: high productivity,
- Herbaceous permanent: low productivity,
- or reeds.

This information is used to distinguish between areas where it is possible to take off (for example herbaceous permanent or permanent soil), and areas where it is not possible to take off or land.

5.3. Open Street Map

The Open-Source database OSM[20] is used as underlying road network. This collection of roads, tracks and paths is licensed under Open Data Commons Open Database License (ODbL)[21]. The data of Austria can be downloaded as ESRI Shapefiles from Geofabrik[22]. Figure 5.3 shows the OSM website.

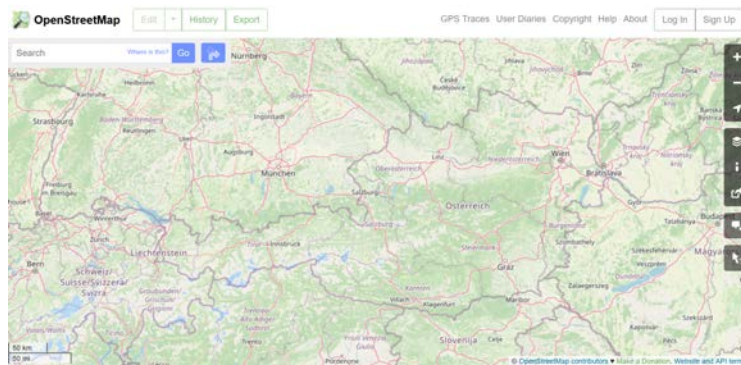


Figure 5.3.: Open Street Map Website[20]

5.4. DHV-Geländedatenbank

The Deutscher Hängegleiterverband e.V. (DHV) offers a collection of well-known take-offs in various formats for download[23]. This database is called "DHV-Geländedatenbank". This data is free to use for non-commercial purposes (more information to be found at [24]). The downloaded data (in the format Garmin Comma Separated Values (CSV)) contains the latitude, longitude, name, usability for paragliders and/or hanggliders and take-off directions for many official take-offs and landings. The data is stored as follows:

```
10.199617,46.987042,"Adamsberg SP, GS und HG, O-S"  
13.656317,47.096083,"Speiereck, Aineck LP, GS und HG"  
13.656317,47.096083,"Jocham, Aineck LP, GS und HG"
```

13.656317,47.096083,"Aineck LP, GS und HG"
 13.656317,47.096083,"Hollaberg, Aineck LP, GS und HG"
 13.637064,47.057008,"Aineck SP, GS und HG, W-NW"
 ...

The used abbreviations are explained in table 5.3. *SP* indicates a take-off,

Table 5.3.: Abbreviations used in the DHV-Geländedatenbank

Abbreviation	German expression	English expression
SP	Startplatz	take-off area
LP	Landeplatz	landing area
GS	Gleitschirm	paragliders
HG	Hängegleiter	hanggliders

LP indicates a landing area. *GS* gives the information, that this take-off or landing area can be used with a paraglider, *HG* on the other side shows that it is possible to take-off and land with a hangglider. The last column comprises the take-off direction.

5.5. Airspaces

The Austrian airspaces are updated annually and are published by the Austro Control GmbH[25] in OpenAir format. Airspaces in Austria are divided in the controlled classes C, D and E and the uncontrolled airspace is classified as G. Paragliding is only allowed in class E and G. An extraction of the provided content is shown as follows:

AC D
 AN MATZ AIGEN
 AH 5000 ft msl
 AL SFC
 DP 47:32:19 N 014:04:58 E

5. Data

DP 47:34:15 N 014:11:43 E
DP 47:33:20 N 014:12:25 E
DP 47:31:09 N 014:12:16 E
DP 47:29:24 N 014:05:56 E
DP 47:31:10 N 014:04:56 E
DP 47:32:19 N 014:04:58 E
...
AC CTR
AN CTR LOWG (GRAZ TOWER:118.200)
AH 4500 ft msl
AL SFC
DP 47:05:52 N 015:14:56 E
DP 47:06:14 N 015:17:44 E
DP 47:07:47 N 015:29:40 E
DP 47:08:06 N 015:32:02 E
DP 47:07:23 N 015:34:34 E
DP 46:46:10 N 015:40:25 E
DP 46:43:40 N 015:21:10 E
DP 46:59:05 N 015:16:51 E
DP 47:05:52 N 015:14:56 E
...
AC W
AN TRA ALMENLAND
AH 9500 ft msl
AL 7500 ft msl
DP 47:28:08 N 015:37:58 E
DP 47:14:27 N 015:41:47 E
DP 47:13:20 N 015:32:54 E
DP 47:19:16 N 015:31:14 E
DP 47:20:09 N 015:26:47 E
DP 47:19:42 N 015:23:12 E
DP 47:23:53 N 015:15:47 E
DP 47:25:42 N 015:15:22 E
DP 47:26:46 N 015:25:13 E
DP 47:28:08 N 015:37:58 E
...

5. Data

The first line per airspace declaration *AC* defines the classification of the airspace, the second line *AN* the name, third *AH* the upper boundary height, fourth *AL* the lower boundary height. Each line starting with *DP* defines the coordinates of one point of the airspace polygon. The used abbreviations are explained in table 5.4.

Table 5.4.: Abbreviations used in the OpenAir format

Abbreviation	English expression
D	Controlled airspace class D
CTR	Control zone
W	Wave Window (used for Temporary Reserved Airspace (TRA))
SFC	Surface
MSL	Mean Sea Level

These files include all controlled airspaces (except airspaces of class E, which do not require a ATC clearance and can therefore be entered with a paraglider) as well as TRA, danger and restricted areas. In controlled airspaces of class D, C or Control Zone (CTR) paragliding is prohibited. Furthermore it is forbidden to enter a restricted or danger area within its duty hours. TRAs can be activated by the relevant flight operation office. During this period airspace class G (uncontrolled and hence open for paragliding) is allocated to this area.

5.6. National Parks

For the protection of flora and fauna certain areas are classified as national parks. Depending on the individual rules per national park, paragliding is completely prohibited (taking off, landing and flying across) or only parts (for example taking off and landing) are forbidden. In this master's thesis the restrictions of the *Nationalpark Hohe Tauern* are included. The nationalpark area is received from the Open Data Portal Austria and is published by Nationalparks Austria[26] in GeoJSON format.

5.7. Test Area

The investigation of routing possibilities is done in the area of Salzburg. The borders are retrieved from Open Data Austria and published by Land Salzburg[27] under the Creative Commons Namensnennung 4.0 International license.

An overview of the testarea Salzburg, the restricted airspaces and the nationalpark Hohe Tauern is shown in figure 5.4.

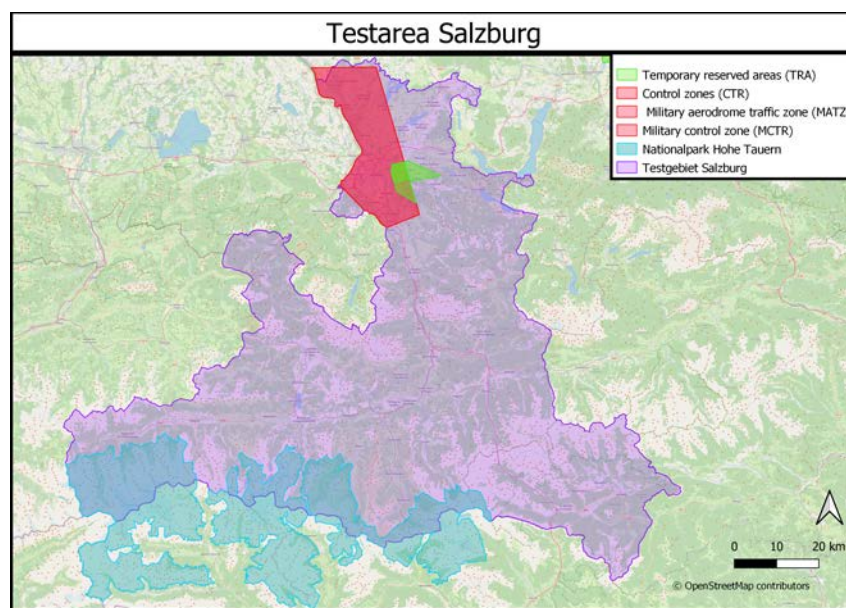


Figure 5.4.: Testarea Salzburg (graphic by the author)

6. Programs

This chapter elaborates on the programs needed for full-filling the programming.

6.1. ArcGIS

ArcGIS is the Desktop Geographic Information System (GIS) application developed by ESRI[28]. It offers various options for data analysis, migration, visualisation and more. The work of this thesis is achieved with ArcMap 10.5.1.

6.1.1. ArcGIS Toolboxes

ArcGIS offers different toolboxes depending on the customers' needs. The *Spatial Analyst* extension offers tools for modelling and analysing spatial raster and vector data. A students or full license is needed for the usage.

6.2. QGIS

QGIS[29] is a free Open-Source GIS software. Unlike ArcGIS it offers less tools for spatial analysis, but data-conversion and display are easier. In this thesis it is mostly used for output analysis.

6.3. pgAdmin

pgAdmin is the browser application used for the database setup, programming of Structured Query Language (SQL) functions and final data manipulation. It hosts the database server on which route calculations are done.

6.4. Visual Studio Code

For developing the website, Visual Studio Code[30] is used. The Server-Client architecture is also set-up within the Visual Studio Code application. Syntax highlighting, terminal real-time output and debugging tools are very helpful for efficient programming.

7. Programming Languages And Tools For Data Interfaces

This section focuses on a list of programming languages used for developing a "Hike and Fly" - routing platform.

7.1. Python

Take-off and landing evaluation, computation of the possible flight-paths and extracting ascent and decent information from raster data is done with python[31]. Python is an interpreted, high-level and general-purpose programming language used for educational and commercial projects likewise. One focus, when programming in Python, is the readability. Code is supposed to be easily understood in both, object-oriented and functional programming. Version 2 (discontinued in 2020) and 3 (released in 2008) are not compatible. As ArcMap 10.5 uses Python 2, the master thesis' code is written in Python 2, using different libraries as for example *arcpy*, *math* or *time*.

7.2. PostgreSQL

PostgreSQL or Postgres is named the World's Most Advanced Open Source Relational Database[32]. Relational databases consist of several tables which are interconnected, or in other words *related*. Each table consists of rows, each entry is named a tuple and columns are often addressed as attributes[33]. These tuples or entries can be imported, manipulated, deleted and inquired

using SQL and pgAdmin. Within this master thesis' database each path, road or track is represented as one row in the table. Attributes like the source and target vertex, length, costs, type of road and the geometric representation are stored in different columns. A second table represents the vertices.

7.2.1. PostGIS

PostGIS is a spatial database extender for PostgreSQL object-relational database[34]. With these extensions, geographic objects can be stored in a PostgreSQL database, location queries can be made and spatial connections can be investigated. PostGIS is licensed under the GNU General Public License (GPL) (version 2 or later)[35].

7.2.2. PgRouting

PgRouting extends the PostGIS/PostgreSQL geospatial database. It provides geospatial routing and additional network analysis functionality[36]. Like PostGIS, it is licensed under GPLv2. Some core features of the pgRouting library are:

- Shortest path A* and Dijkstra
- Bi-directional A* and Dijkstra shortest path
- K-Dijkstra, One to Many Shortest Path
- ...

7.3. HTML5

Hypertext Markup Language (HTML) in its fifth version is the core language used in the World Wide Web (WWW). It is used to connect text and other content in electronic documents. The latest specification was published on 28th of October 2014 by the World Wide Web Consortium (W3C)[37] and

includes more functionality like video, audio and dynamic graphics. Most common webpages are built with HTML5.

7.3.1. JavaScript

In order to add more complex content, calculations, and functions to a webpage, JavaScript (JS) is used. It expands the features of HTML and makes it possible to respond to user interaction, change or load additional content on-the-fly.

OpenLayers

One of many JS libraries is OpenLayers. It includes all types of functions in order to add interactive maps to an existing webpage. Starting with basic functionality like showing for example a OSM map, it also offers the option to display individual geo-referenced vector or raster data. But also map-actions can be defined, like what happens if the user clicks on the map, drags a marker (pinpoint location on the map) somewhere else, zooms in and zooms out and many more. A rich explanation can be found in the book *OpenLayers* by Marc Jansen[38].

7.4. Tools For Data Interfaces

There are two data interfaces within this master thesis: first, an interface to import the Shape-files generated with ArcGIS into the PostgreSQL database, second, an interface to retrieve data from the database from the website.

7.4.1. Command Line Tools

To import Shape-files into a PostgreSQL database the command line tools *shp2pgsql* and *psql* are used. They are part of the Postgres installation. *shp2pgsql* converts Shape-file entries to SQL statements. These generated

code can be stored into an output-file with the ">" sign. The command *psql* establishes a connection to the PostgreSQL database and runs the commands.

Export database entries into Shape-files are possible with the tool *pgsql2shp*.

7.4.2. **express.js**

Express.js[39] is a fast, unopinionated, minimalist web framework for Node.js[40]. Node.js is an asynchronous event-driven JavaScript runtime, designed to build scalable network applications. Express.js is a minimal and flexible Node.js application framework which provides a robust set of features, like retrieving data from a PostgreSQL database using the *pg-promise* module.

Part III.

Algorithms and Realisation

8. Algorithms

In this chapter the invented algorithms, calculations and evaluations are explained. This master thesis was designed on three levels which build upon each other as shown in the diagram 8.1. First, the underlying graph for routing was generated using ArcGIS, second, the generated data was implemented into a PostgreSQL database, which is storing the data and doing the path cost, as well as the routing calculations, and third, a prototype of a webpage was set up to simulate the users' input and display the results of the routing calculations (see chapter 9).

8.1. Preparatory Work

As the first stage of this project a routing graph was created which draws from various data sources. In the end, this routing graph should include edges of the OSM street network as well as additional calculated edges that represent the glidepaths from take-offs to landings, and their connection to the existing street network. All of the above-mentioned sources need to be imported and transformed into the same coordinate system EPSG:31287 (MGI-Coordinates used in Austria). This coordinate system was chosen as the main data source for take-off and landing calculations. The two raster data sets for the elevation (see subsection 5.1) and the land use (see subsection 5.2) are given in this projection.

Although the land use and the DEM rasters are given in the same coordinate system, they have a five meter offset to each other. Therefore the DEM was interpolated so that the raster pixel are overlaying with the land use raster. This was done using the built in raster interpolation tool from ArcGIS with the *snap-raster* option and a linear interpolation. The data flow can be seen in diagram 8.2.

8. Algorithms

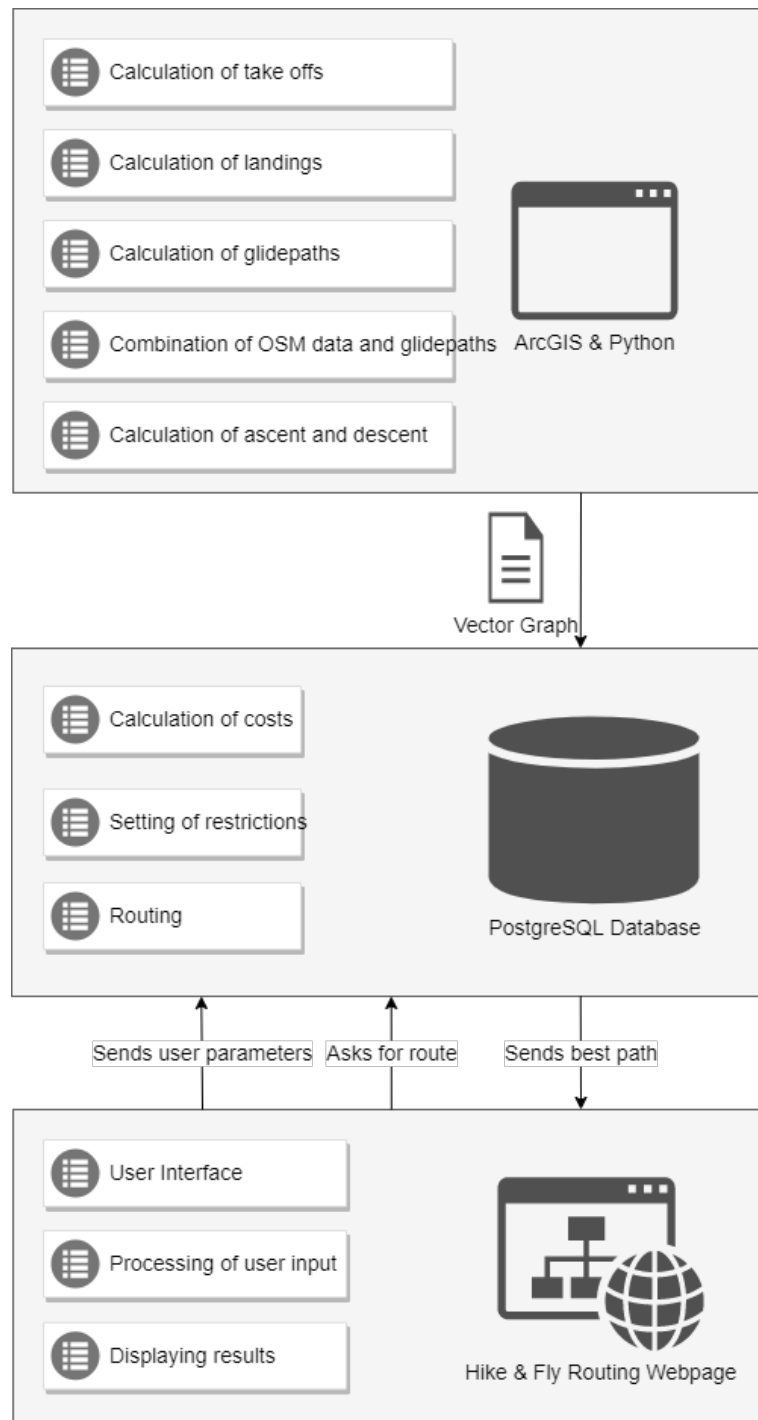


Figure 8.1.: Development levels

8. Algorithms

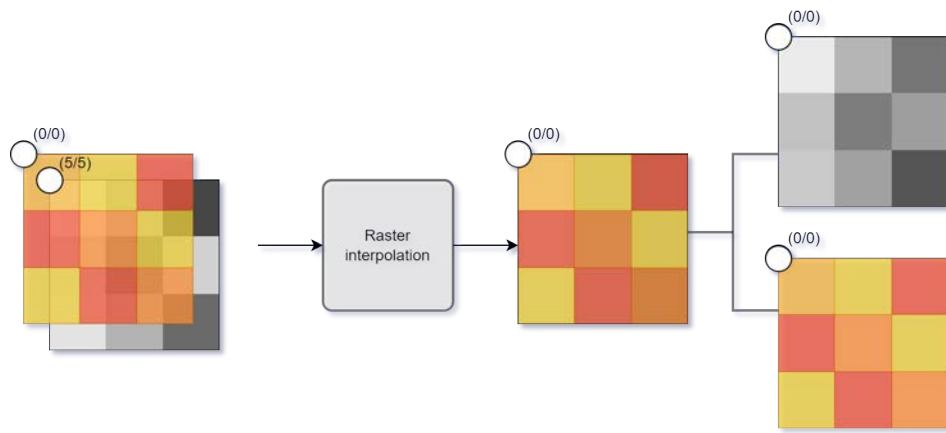


Figure 8.2.: Raster interpolation

One of the other import data sources is the *DHV-Geländedatenbank* (explained in subsection 5.4) which stores information about official take-offs and landings in text-format. Another data source is the airspace file (see subsection 5.5) published by the Austro Control GmbH. Both need to be converted to ESRI-Shapefiles for further use within ArcGIS (see section 6.1) for follow-up calculations. This is achieved by using the import function of QGIS (see section 6.2). When imported, the data is transformed into the same coordinate system EPSG:31287 (MGI-Coordinates). This is achieved by using the built-in transformation and reproprojecting tool of QGIS. The data flow is shown in diagram 8.3.

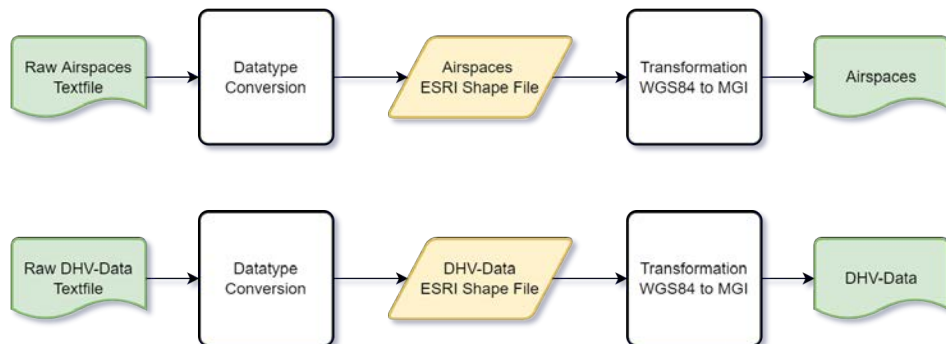


Figure 8.3.: Data import

The OSM vector data for the country of Austria (see in subsection 5.3) is already given in a suitable file-format and can be imported into ArcGIS directly. This data includes all highways, streets, paths and other kinds of roads. The test area outline of Salzburg and the boundary of the national park Hohe Tauern are in ESRI-shapefile format too (see subsections 5.7 and 5.6). During the import, all of the files are transformed into the EPSG:31287 coordinate system.

Finally, each data sets extent is set to the outline of the test area Salzburg (see subsection 5.7).

8.2. Take-Off Calculation

The diagram 8.4 shows the different steps to finding suitable points for taking-off with a paraglider.

The first step in detecting a take-off is to calculate the slope and aspect out of the given DEM raster (named "Elevation" in the diagrams) by using the built-in functions of ArcGIS. Second, these calculated rasters as well as the land use raster are scanned for possible take-off pixels. Each of the following steps is calculated individually for every take-off direction north, north-east, east, south-east, south, south-west, west and north-west. As stated in section 2.2 the hillside needs to fulfill the following criteria

$$7^\circ < slope < 30^\circ, \quad (8.1)$$

$$take - off\ direction - 45^\circ < aspect < take - off\ direction + 45^\circ, \quad (8.2)$$

and

$$land\ use = [31, 125, 126] \quad (8.3)$$

with the class 31 refers to *permanent soil*, 125 and 126 to *herbaceous permanent (high or low productivity)* (see figure 5.2). Figure 8.5 shows the raster evaluation.

A majority-filter is applied to the output raster afterwards. This step is executed to reduce the number of small take-off areas which are in most cases unusable as obstacles are likely to prohibit taking off. The shape of the majority filter is a 90 degree arc with a radius of 30 meters in the direction

8. Algorithms

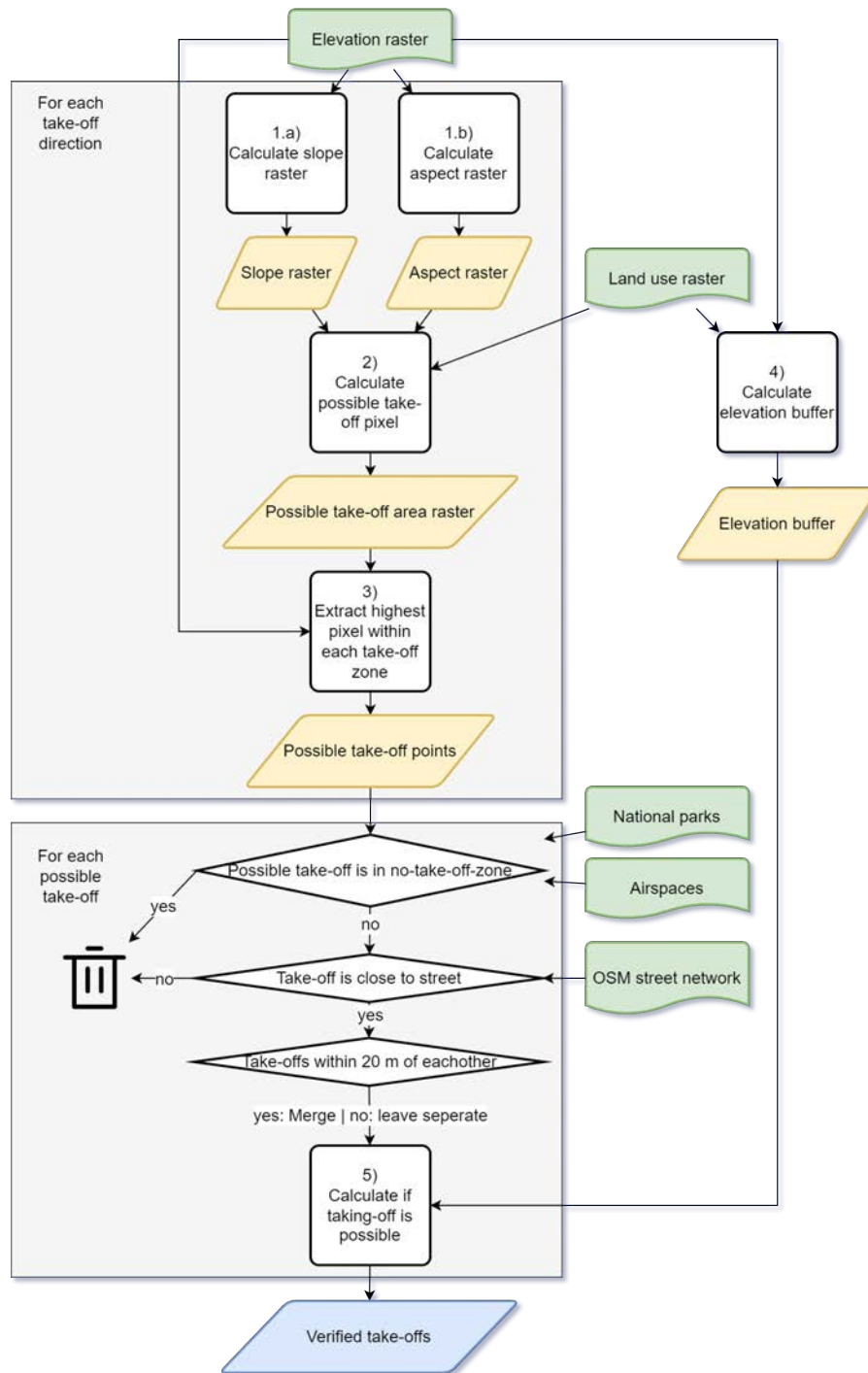


Figure 8.4.: Flow Chart of the take-off calculation

8. Algorithms

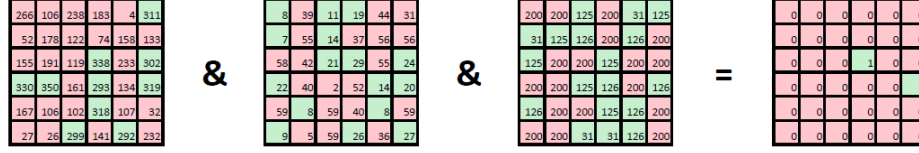


Figure 8.5.: Evaluation whether a raster pixel is a potential take-off area. From left to right: aspect (take-off direction north-west or 315°), slope, land use, potential take-off

of the take-off and represents the space needed to launch. In case there is zero wind during the take-off, the pilot needs to accelerate on the ground to reach the take-off velocity of the paraglider before lifting into the air. This can be seen as the *runway* for a paraglider. The arc-shape is used due to the fact, that a paragliding pilot is more flexible on the ground than an airplane. After the majority filter is applied only pixel with a *runway* in front remain for the next calculation steps.

Within these generated possible take-off areas, the highest (and therefore most likely) point for a take-off is determined using an inverse flow analysis as the third step. The different calculation steps within this process are shown in diagram 8.6. First, for all pixels, which are possible for take-off the value calculates as

$$value = 10000 - Elevation \quad (8.4)$$

while all others are set to 10000. This leads to a raster, in which the points with the highest elevation (within a take-off zone) have the smallest values. This output raster can then be analysed using the ArcGIS flow analysing tools **flow direction** and **sink**. In order to get a vector file of those sink-pixels they must be converted from raster to point.

Furthermore, each potential take-off must not be placed within a no-take-off-zone (restricted airspaces, national parks) and must be at maximum 50 m off a path, track or road from the OSM network. These parameters are evaluated using the ArcGIS functions **near** and **clip**. Potential take-offs which are within 20 m of each other are merged.

Step number four of the take-off calculation (see diagram 8.4), is to create an elevation buffer raster. This raster determines the minimum height above a given land use which is needed for a safe flight over. This buffer value is added to the DEM values. Table 8.1 shows the added buffer values per land

8. Algorithms

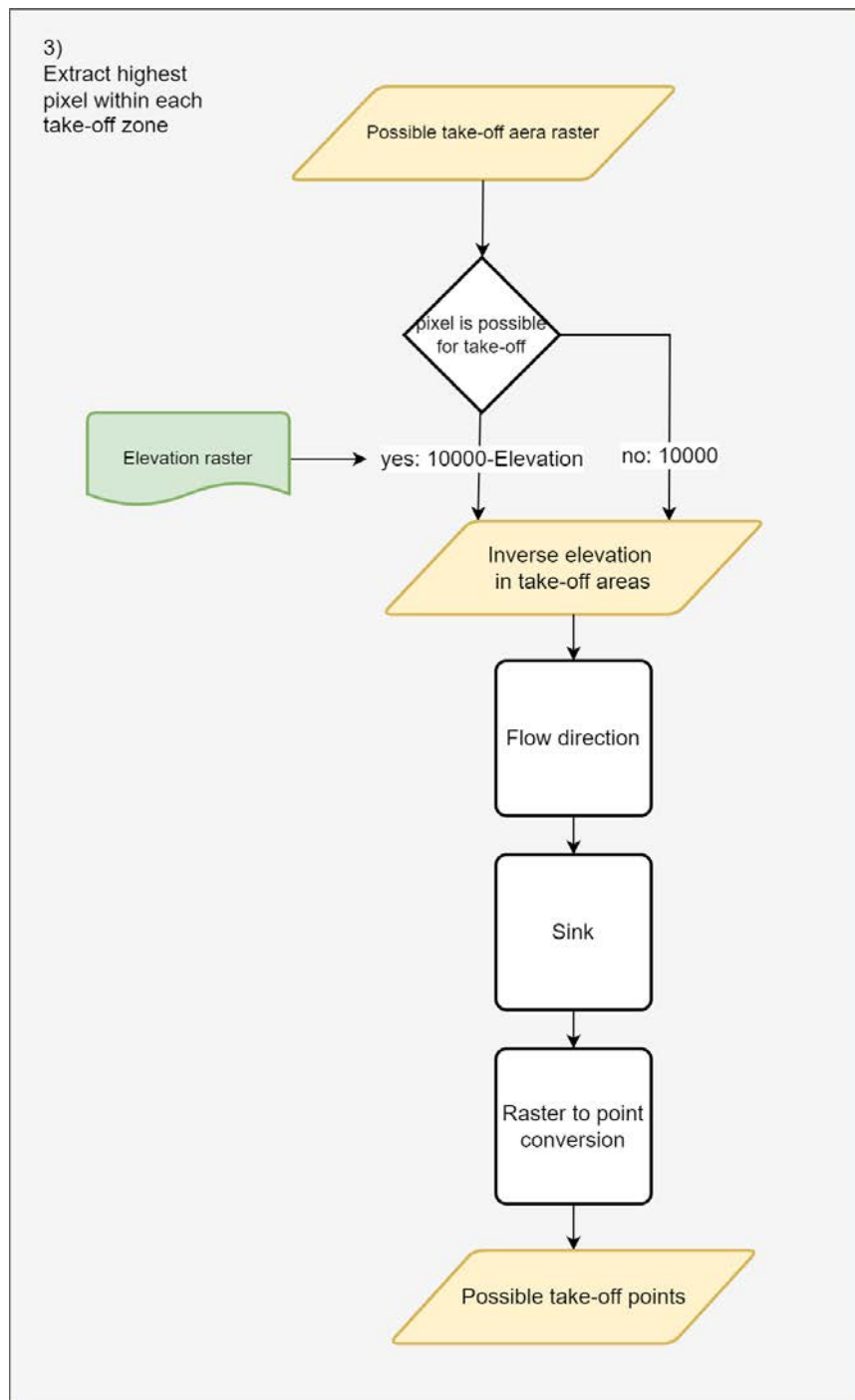


Figure 8.6.: Flow chart of the extraction of the highest point within a take-off zone

land use class	added buffer value [m]
Build-up	30
Flat sealed surfaces	0
Permantent soil	0
Bare rock and screes	0
Snow and ice	0
Water	0
Coniferous trees	35
Broad-leaved trees	25
Bushes and shrubs	10
Herbaceous periodic	0
Herbaceous permanent: high productivity	0
Herbaceous permanent: low productivity	0
Reeds	3

Table 8.1.: Buffer values for different land use classes

use class. This buffer represents obstacles emerging from the ground that make a take-off impossible. The final step is the calculation, of whether any obstacles or the DEM itself hinder a successful take-off. This calculation can be executed in two ways: vector-based or raster-based.

Vector-Based Check Of Absence Of Obstacles

The different steps in the vector-based analysis are shown in diagram 8.7. First, an imaginary 3D line 500 m away from the take-off is calculated. This line is orthogonal to the take-off direction and spans an isosceles triangle with an angle of 40° at the take-off. The height of this line is calculated as

$$h_l = h_{to} - \frac{d}{gr} \quad (8.5)$$

with h_l being the elevation of the imaginary line to be seen, h_{to} being the elevation of the take-off, d the distance from the take-off to the line and gr being the glide-ratio. Second, sight-lines to this imaginary line are constructed (shown in figure 8.8). ArcGIS offers the functions **Construct**

8. Algorithms

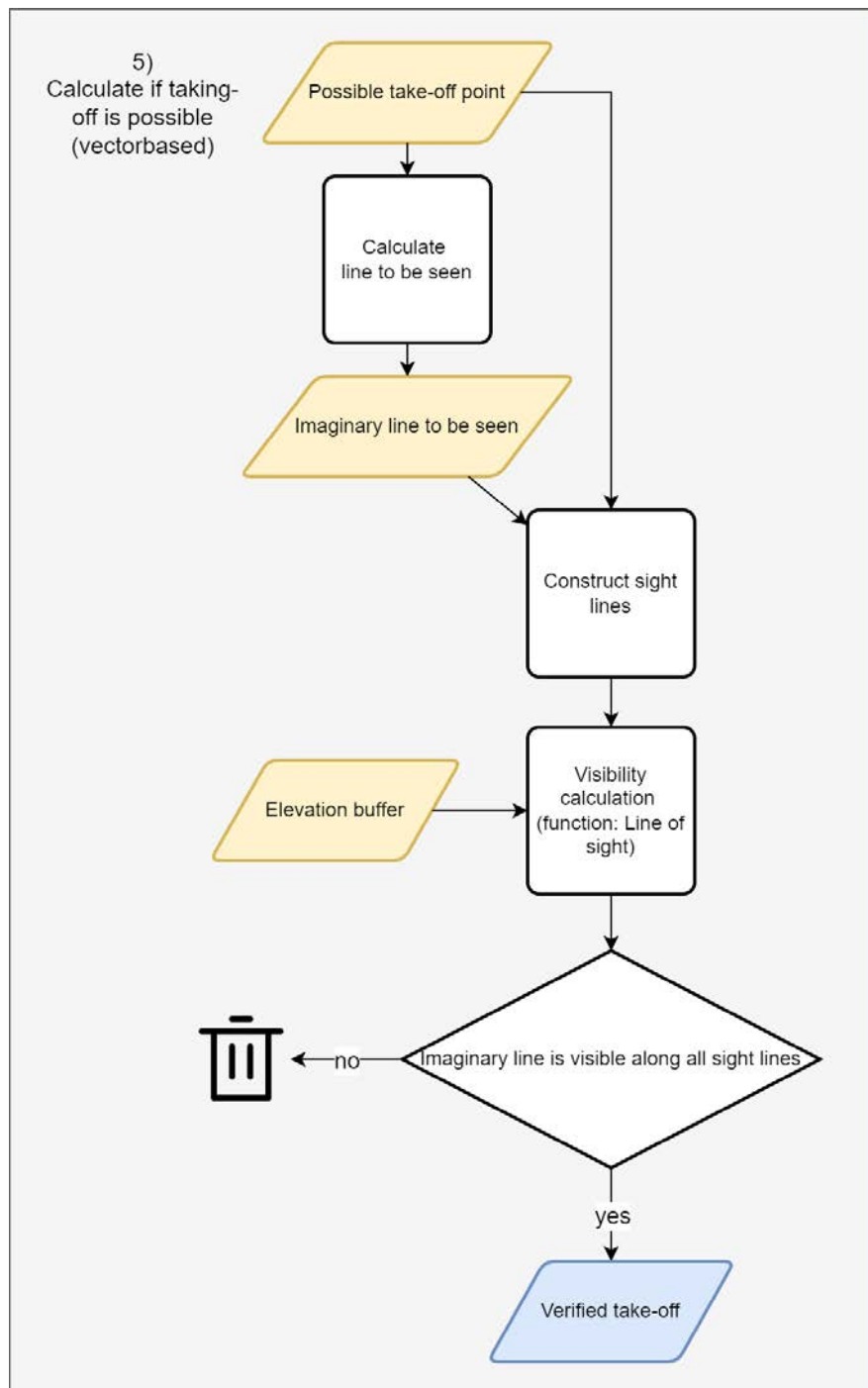


Figure 8.7.: Flow chart of the vector-based check of absence of obstacles

Sight Lines to create the lines between the take-off and the imaginary line. It also offers the function **Line of Sight** for the evaluation, if the given target (imaginary line, colored red) is visible along the constructed sight-lines (violet) from the take-off over the DEM with the added elevation buffer.

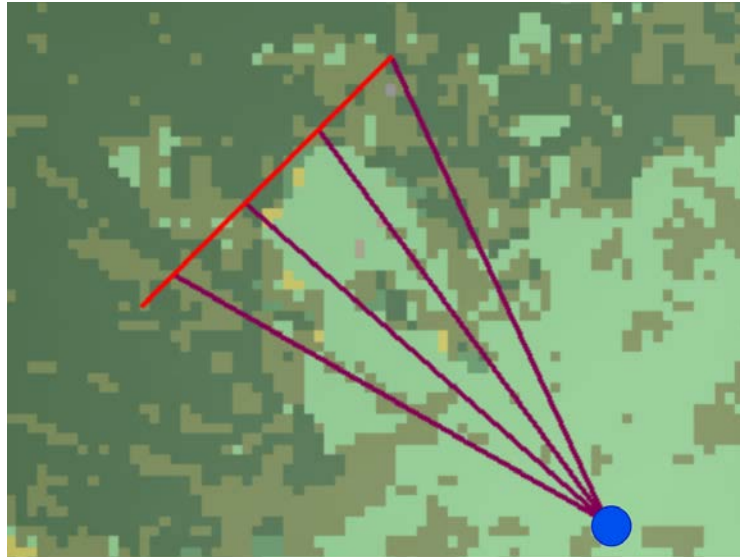


Figure 8.8.: Vector based analysis, whether it is possible to take-off from a certain point. The imaginary line (red), must be visible from the take-off (blue point) along the four representative sight-lines (violet).

Only when the imaginary line can be seen along all four sight-lines, which means there is no tree, building or rising ground ahead, will the point be classified as a safe take-off.

Raster-Based Check Of Absence Of Obstacles

Another option is to examine the triangular area in front of the take-off with the help of a raster-based check. This isosceles triangle is defined by the take-off in the acute angle and the imaginary line-to-be-seen (represented as red line in the graphic 8.8) as opposing side. The different steps for the raster-based analysis are listed in diagram 8.9.

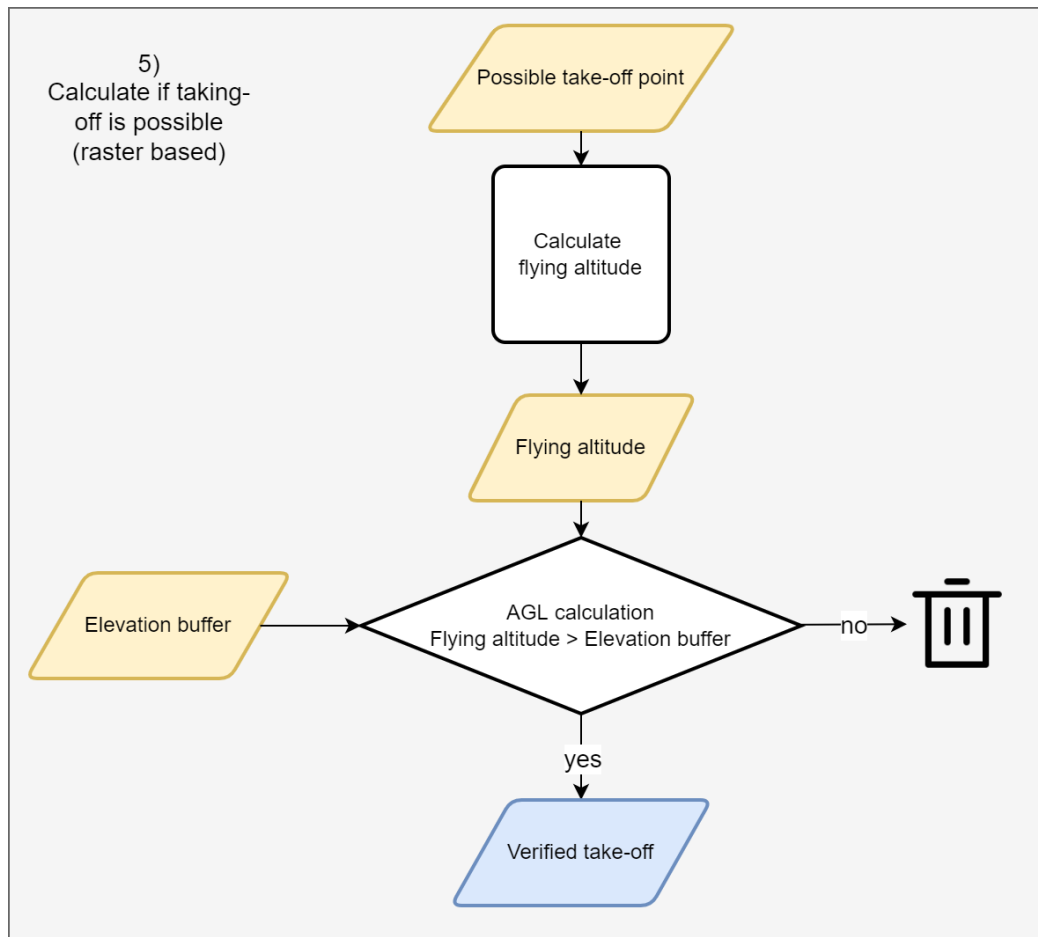


Figure 8.9.: Flow Chart of the raster-based check of absence of obstacles

First, a new raster must be calculated for each take-off. The pixel value in this raster is calculated in the same way as for the "line to be seen" in equation 8.5 and indicates the height above mean sea level at which the paraglider would be flying over this point (named flying altitude in diagram 8.9). In other words it represents the glide plane of a paraglider taking-off from this certain possible take-off. Taking this into the process, the AGL height can be calculated for each pixel, using a matrix subtraction of the flying altitude minus the elevation buffer raster. If any negative AGL heights occur inside the triangle in front of the take-off, it indicates that a collision would be taking place, as the paraglider would be flying below the surface or into an obstacle. Consequently, the examined take-off is classified as unsuitable and is therefore deleted.

As this calculation needs more computational power and thus also longer time to process, the vector-based evaluation was used for this thesis. This decision was based on the scope of this thesis while being aware of the fact that the vector-based evaluation shares a higher risk of wrongly classified take-offs, as only a subsection of the area in front of a take-off is evaluated.

8.3. Landing Calculation

In diagram 8.10 the different calculation steps are shown. As first step, raster algebra is used to determine suitable pixels for landing. The criteria each pixel must fulfill are the following:

$$\text{slope} < 3^\circ \quad (8.6)$$

and

$$\text{land use} = [31, 125, 126]. \quad (8.7)$$

These criteria make sure, that the landing is flat (slope less than three degrees) and either grass or soil (same as for take-offs). If both criteria are met, the pixel is set to one, which identifies it as suitable for landing, else its value is set to zero, determining that it is not possible to land there. Additionally, a buffer-zone of 30 m is given by using the tool **focal statistics**. First, with a **Minimum**, second with a **Maximum** analysis. The **Minimum** sets the investigated pixel to the smallest value within the buffer zone of

8. Algorithms

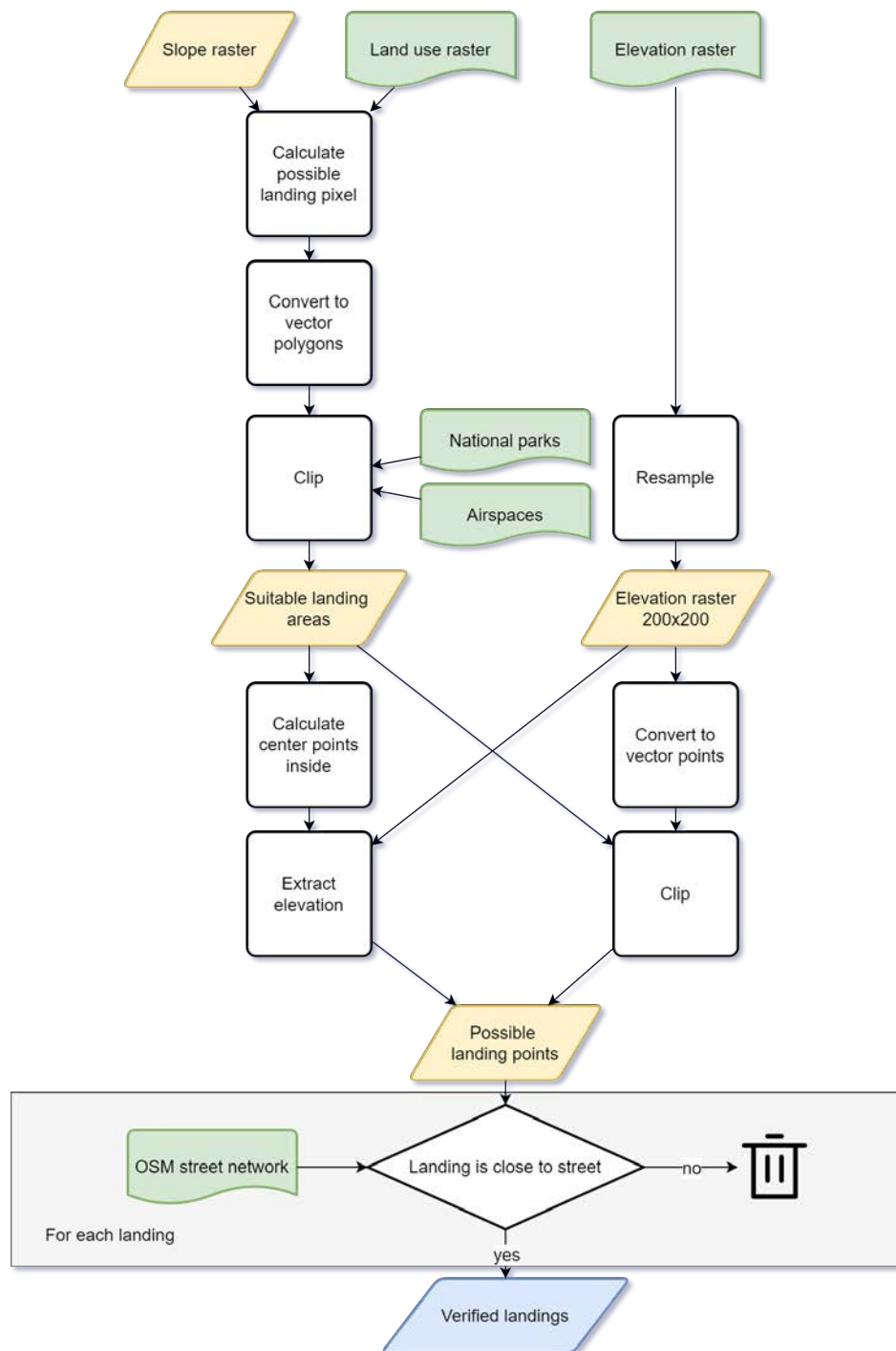


Figure 8.10.: Flow chart of the landing calculation process

30 m around. If any pixel in the buffer zone is not suitable for landing and has the value zero, the investigated pixel is set to zero. To regain the actual landing area the **Maximum** is applied afterwards by setting the investigated pixel to one if any of the surrounding pixels has the value one after the **Minimum** calculation. Afterwards these landing areas are converted to vector polygons. As a reminder, areas in restricted airspaces or national parks are erased due to unsuitability. Then the DEM raster is resampled to a 200 m times 200 m raster and then converted to point features. This creates a 200x200 m grid of possible landing points. If they are located within the landing zones calculated in the previous step and are within reach of a path, track or road, they are classified as landings. In addition to the grid points each landing polygon center is analysed by reachability and then classified as potential or unsuitable landing. An overview is given in figure 8.11 Both,

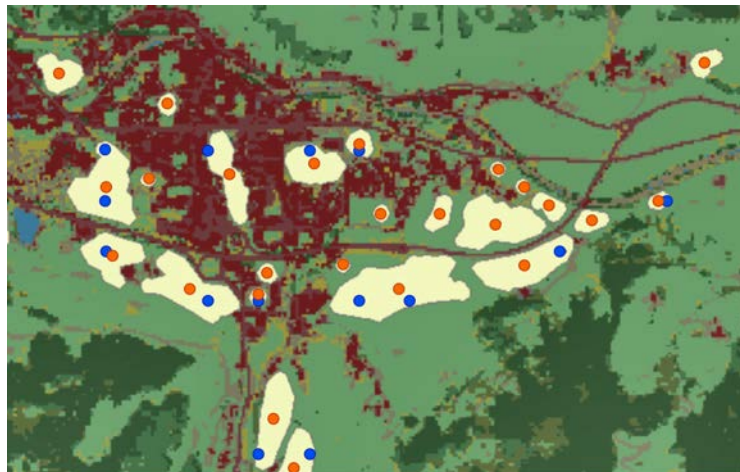


Figure 8.11.: Overview of possible landing points and areas. The yellow polygons represent the areas which are suitable for a landing, the orange dots are the landing points in the center of the landing areas and the blue dots the landing points out of the grid.

the grid points and the polygon center points, are necessary as a single point would not be enough in huge landing areas (more than 200x200 m). Small landing zones, however, might be overseen if only the grid points are used.

8.4. Calculation Of Glidepaths

Once all take-offs and landings are calculated and the take-offs and landings from the DHV-Geländedatenbank (referenced as *official* take-offs and landings) are imported, connections between them must be established. These connections are needed for the graph network later. The different calculation steps are shown in diagram 8.12. For each take-off the reachable area is calculated in a 100 m times 100 m raster. This can also be seen as the area, in which the AGL height

$$AGL = glideplane - elevation \quad (8.8)$$

is more than zero meters. A two dimensional sketch of the calculation of the AGL value is shown in figure 8.13. This calculation is achieved by calculating a cost raster out from the take-off, whereas the travelling costs are the altitude loss per pixel. With a glide-ratio of one to seven the cost for travelling one pixel of 100 m is 14.28 m altitude loss. Therefore the value of the glideplane calculates as

$$value_{glideplane} = elevation_{take-off} - travellingcosts \quad (8.9)$$

Then the intersection with the DEM is calculated and all pixels of the cost-raster, where a collision would take place, are set to *null*. Then the calculations are done again with the changed cost-raster. This ensures that higher travelling costs are calculated for when a detour must be made around an obstacle. This is shown in figure 8.14. The red arrow indicates the lowest cost path of the first calculation step, the grey areas identify where a collision would take place. Therefore, in the second calculation step the shortest path to the destination is marked with a green arrow. As the green arrow is a little longer than the red one, higher costs apply and this pixel might not be reachable anymore. The reachable area is illustrated in graph 8.15. All the pixels with a red to green shading are reachable from this specific take-off. Green areas indicate high AGL values, red ones indicate values close to zero. For each reachable landing spot a connection from the take-off to the landing is established, and stored as straight line vector referenced as *glidepaths* in this thesis. The stored classification-attribute of these vectors is the *glidepath* and indicates a path which is flown with a

8. Algorithms

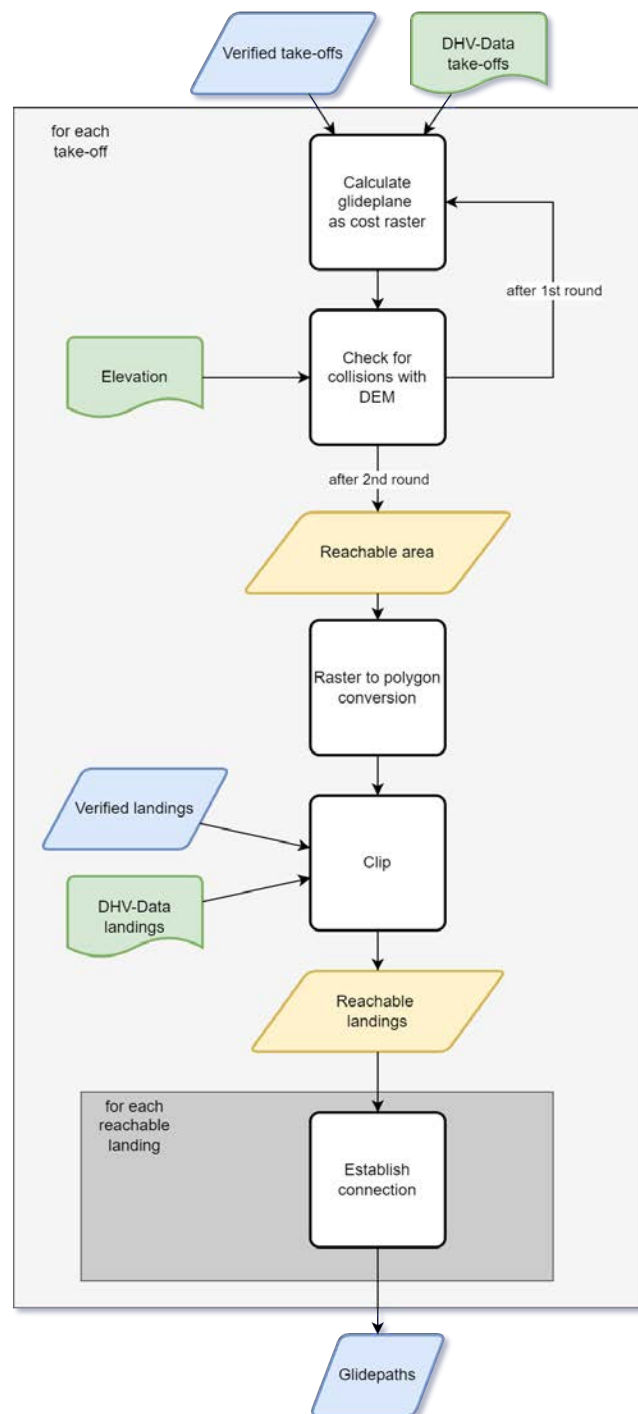


Figure 8.12.: Flow chart of the glidepath calculation

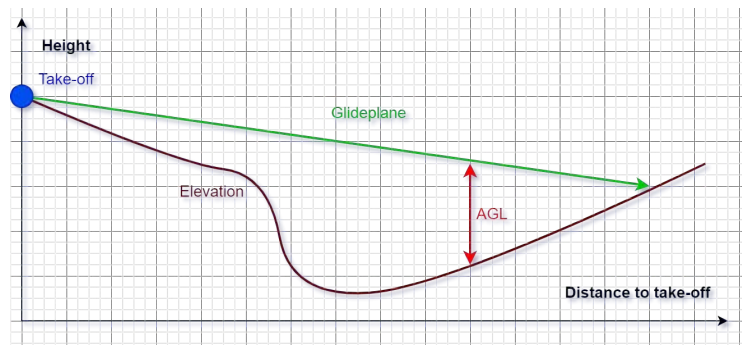


Figure 8.13.: 2D sketch of the AGL value. The blue dot represents this specific take-off, the green line the glide plane and the brown line indicates the elevation.

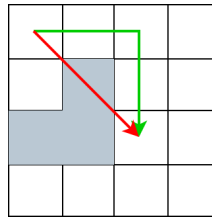


Figure 8.14.: Different shortest paths in the two calculation steps of the reachable area calculation.

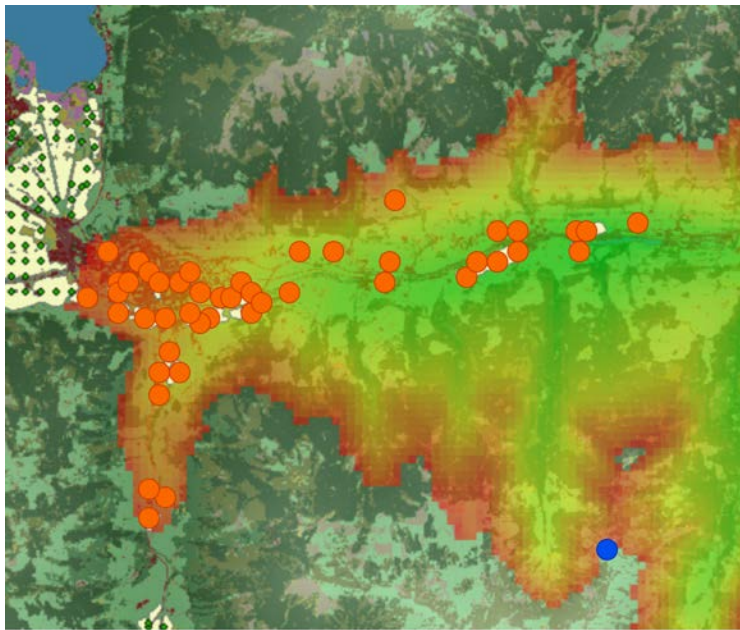


Figure 8.15.: Determination of reachable landings from a specific take-off. The investigated take-off is shown as a blue dot, reachable landing points as orange dots, unreachable landing spots as small green dots. The red to green shaded region indicates the reachable area.

paraglider. Additionally, the following information is stored as attributes of the vector as well:

- the arrival AGL height
- the altitude difference between take-off and landing
- if the take-off is an official one published by the DHV
- if the landing is an official one published by the DHV
- the elevation of the take-off

This information is needed for the determination of cost (explained in section 8.8), as well as when setting restrictions (see section 8.9) for routing purposes.

8.5. Combination Of OSM Data And Glidepaths

To ensure the interconnection of glidepaths and the OSM network additional paths must be included in the routing network (see diagram 8.16). This is necessary as without these, there would be a gap between the street network and the take-off and landing points and thus routing would not be possible. For each take-off and landing the shortest straight distance to the official OSM road network is stored as polyline vector. In graph 8.17 a schematic overview about the different edges of the created vector graph is shown. This vector graph is then used for the routing calculations in the PostgreSQL (PSQL) database.

8.6. Calculation Of Ascent And Descent

For each network segment on the ground, except bridges and tunnels, the ascent and descent are calculated by examining the height profile along it (see flow chart diagram 8.18). Each intersection in the road network defines a node. The connection between two nodes then forms a line segment. The height profile along each segment is generated using the command **stackProfile_3d** which returns the elevation of each raster cell which is touched by a vector line. The vector shown as red line in graph 8.19 will

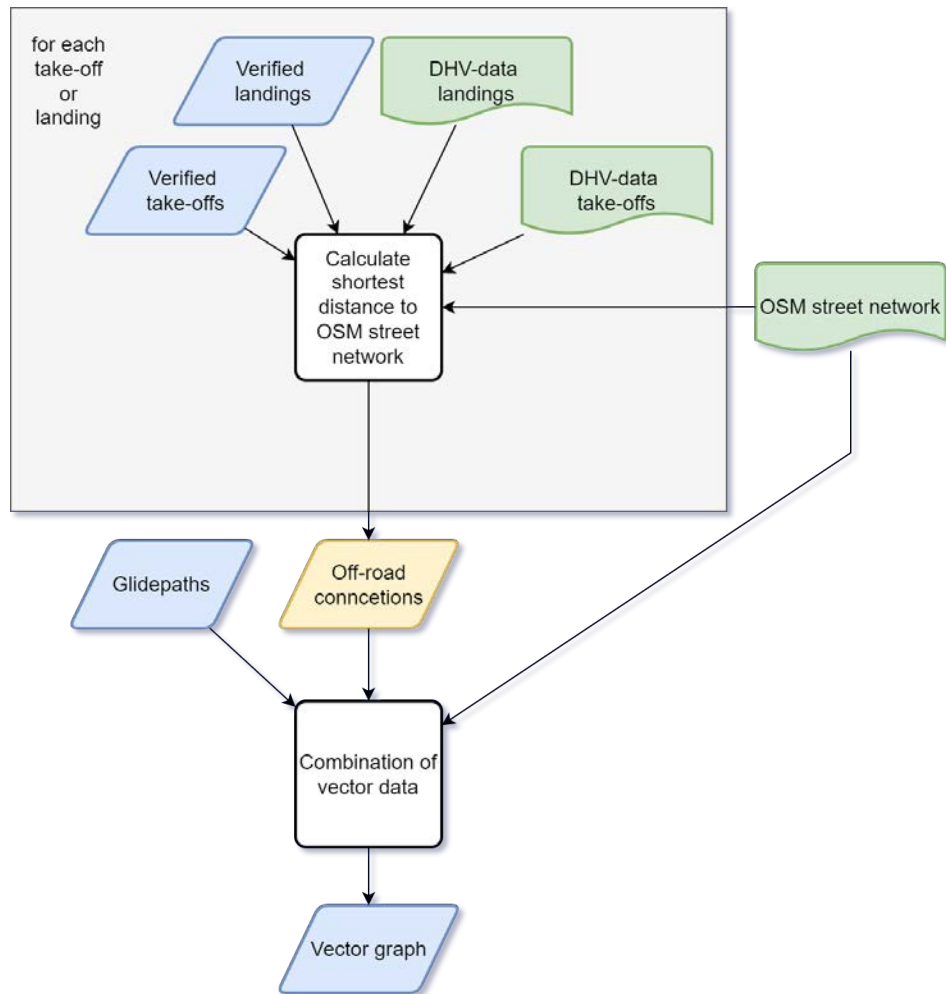


Figure 8.16.: Flow chart of the calculation of the interconnecting paths between the aerial part of the network and the OSM street network.

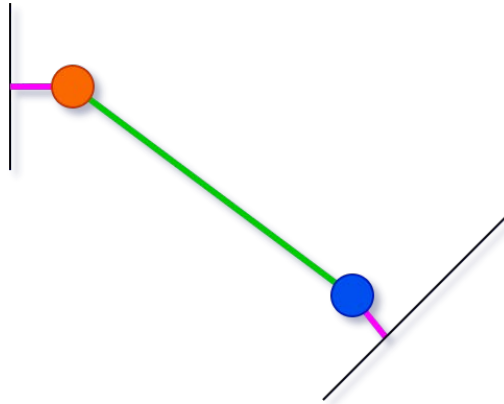


Figure 8.17.: Schematic overview about the different types of connections in the graph network. The blue dot represents a take-off the orange one represents a landing. The connection between them (glidepath) is shown as green line, the off-road connection to the OSM street network in pink, and the roads of the street network themselves are marked in black.

then return the elevation values h : 860, 872, 876, 872, 875, 880, 886, 881, 885.

The elevation value is taken from the surface of the DEM (re-sampled to a pixel size of 30 times 30 m). Taking into account this information the ascent and decent for each segment can be calculated as demonstrated in algorithm 8.1. A visualisation is given in figure 8.20.

Algorithm 8.1 Calculation of the ascent and descent along a given path

```

1: for each Linesegment do
2:   for  $i = 1; i < \text{count}(\text{rows}); i++$  do
3:     if  $h(i-1) \geq h(i)$  then
4:        $dsc = dsc + (h(i-1) - h(i))$ 
5:     else
6:        $asc = asc + (h(i) - h(i-1))$ 
7:     end if
8:   end for
9: end for

```

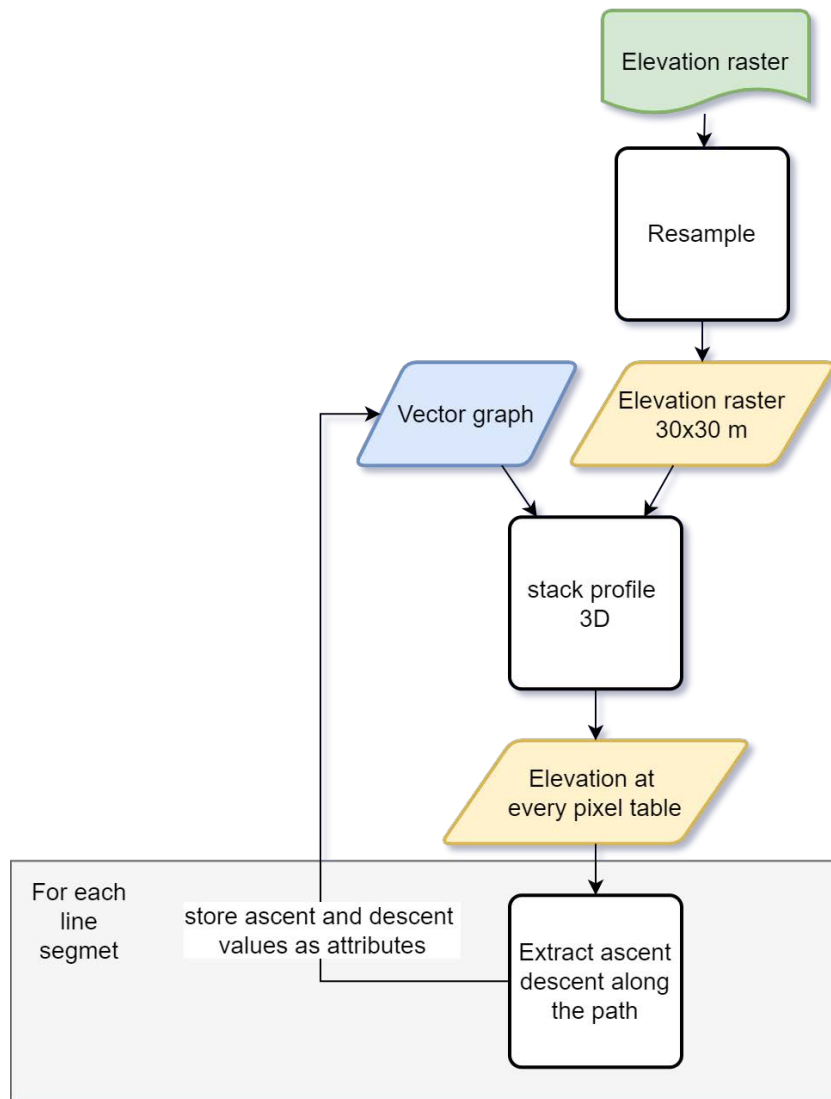


Figure 8.18.: Flow chart ascent descent calculation

8. Algorithms

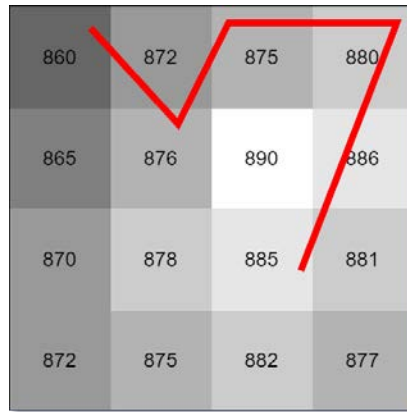


Figure 8.19.: Schematic overview how the command stack-profile is working

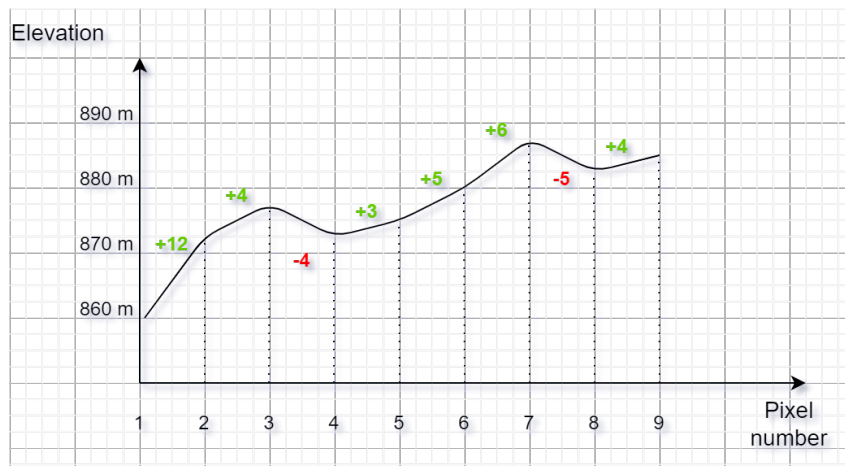


Figure 8.20.: Visualisation of how the accumulated ascent and descent for a specific road segment is calculated. All green values sum up to the ascent along the path. All negative red values sum up to the descent (e.g. in this example the stored ascent value would be 34 meters, the descent value nine meters).

8.7. Data Migration Into The PostgreSQL Database

Once all of the calculations are done, the OSM road network, including the calculated vectors to and from take-offs and landings to the road network as well as all the glidepaths, is imported into a PostgreSQL database and additional preparatory work needs to be done before routing is possible (see flow chart diagram 8.21). This indicates the start of the second developmental level of this master's thesis. First, to import the raw data into the database, the command line tool **shp2pgsql** (see subsection 7.4.1) is used. Second, the imported multilinestrings must be converted to linestrings to be usable within the PgRouting extension (see subsection 7.2.2) for route-analysis. Furthermore, to create a noded network, the command **pgr_nodeNetwork** is used. This tool ensures that, wherever two or more vector lines intersect, a node (also called vertex) is set and the line is split up into individual segments. As the OSM street network was not invented for routing purposes in the first place, this step is necessary to create a vertex at every intersection or t-crossing. Only within a noded network also called planar graph (explained in section 3) routing algorithms can be used. Beforehand tunnels, bridges and glidepaths must be excluded from the road network, otherwise wrong intersections, such as jumping from a bridge to use the underlying road, would be detected. Furthermore, additional columns for routing are added such as cost and coordinates of the source and target of each edge. The preceding steps finish with the PgRouting command **pgr_createTopology** and as result a database, which holds two main tables **roadsandglides** and **roadsandglides_vertices_pgr** as shown in the diagram 8.22, is set up. The table **roadsandglides** includes every single edge of the routing network with the id of the source vertex stored in column *source*, the id of the target vertex in column *target* and the geometric representation stored in column *geom_linestring*. If the edge is a glidepath, the column *arrival_agl* represents the altitude above ground at which the landing will be reached. Column *to_direction* gives information about the suitable wind direction for taking-off. The column *to_lines* displays the number of visible lines from the step 8.2, and in column *to_elevation* the elevation of the take-off is stored. Furthermore the boolean values *official_t* and *official_l* indicate whether the take-off and/or landing is official. The other columns

8. Algorithms

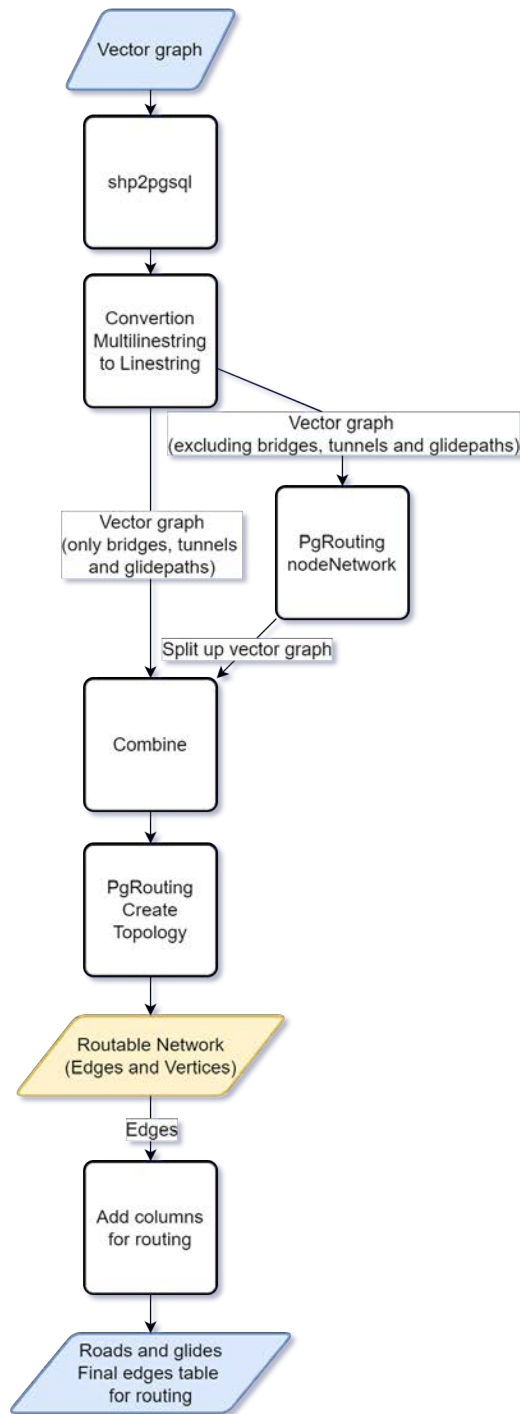


Figure 8.21.: Flow Chart of the data migration into the PostgreSQL database

8. Algorithms

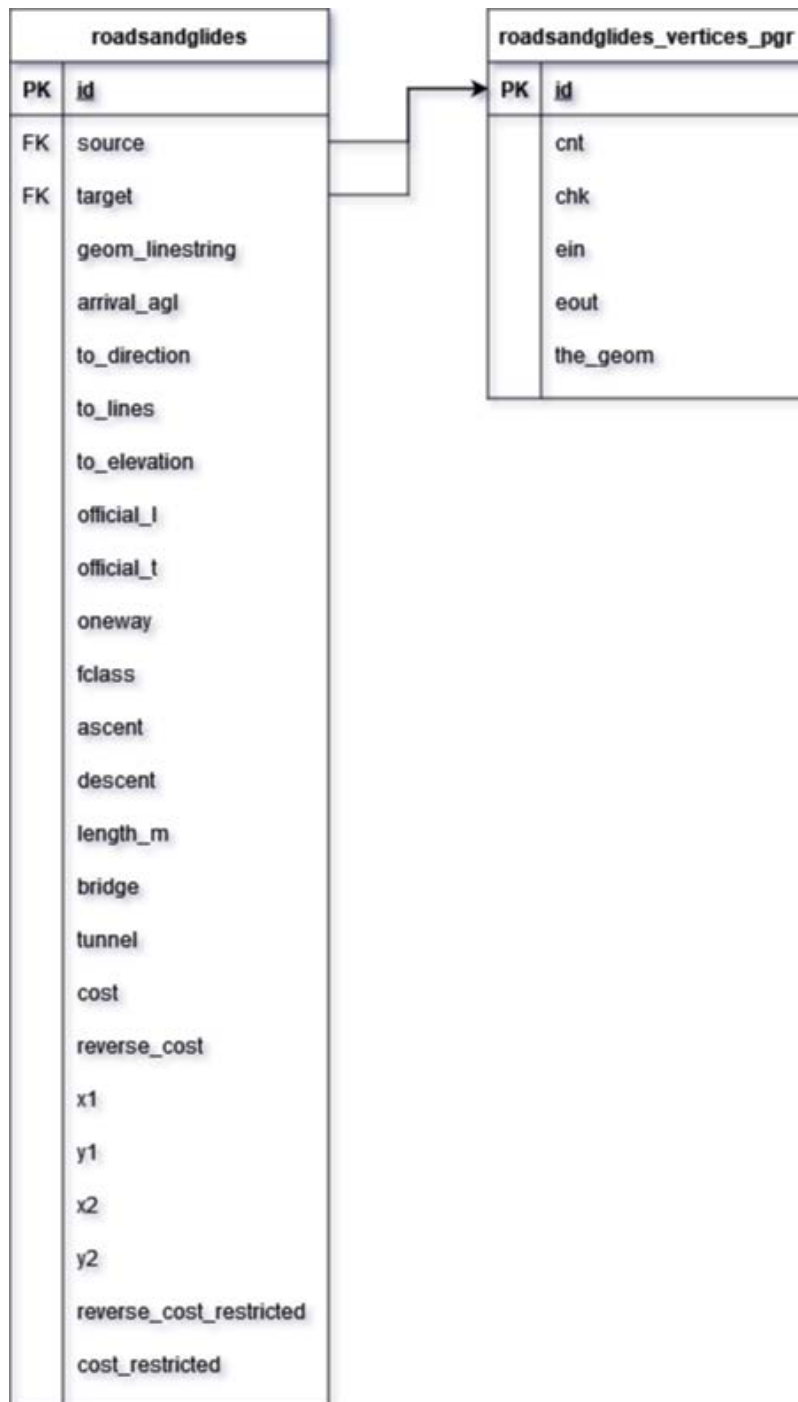


Figure 8.22.: Entity Relationship Diagram (ERD) of the tables used for routing

are filled for each edge of the network. The column *oneway* indicates paths which are only usable in one direction; *fclass* stands for the type of path. The columns *ascent* and *descent* indicate the vertical gain and loss along this path with the length stored in *length_m*. If the edge happens to be a bridge or tunnel the information can be found in *bridge* and *tunnel*.

For routing, the columns *cost*, *cost_restricted*, *reverse_cost* and *reverse_cost_restricted* hold the information about the travelling cost in either direction with or without restrictions applied. For the use of the A* algorithm the columns *x1* and *y1* represent the coordinates of the source vertex, *x2* and *y2* the coordinates of the target vertex.

The second table *roadsandglides_vertices_pgr* includes the information of every single vertex, with *cnt* representing the number of edges which connect to this vertex, whereas *ein* indicating the number of edges which have this vertex listed as incoming and *eout* as outgoing. If there is a problem with this vertex, it is visible in the column *chk*. The point geometry of the vertex is stored in *geom*.

8.8. Determination Of Cost

Each edge of the routing network is assigned with cost representing the minutes it takes to travel along it. The parameters

- *time_{5km0m}*
time needed for five kilometres of hiking in the flat (zero meters of ascent)
- *time_{5km1000m}*
time needed for five kilometres of hiking and 1000 m ascent
- *time_{to}*
time needed at take-off for the preparation of the glider before getting in the air
- *time_l*
time needed to pack the paraglider after landing

can be set individually and the path cost are calculated as shown in diagram 8.23 with different cost calculation for hiking and flying as defined in section 8.8.1 and 8.8.2.

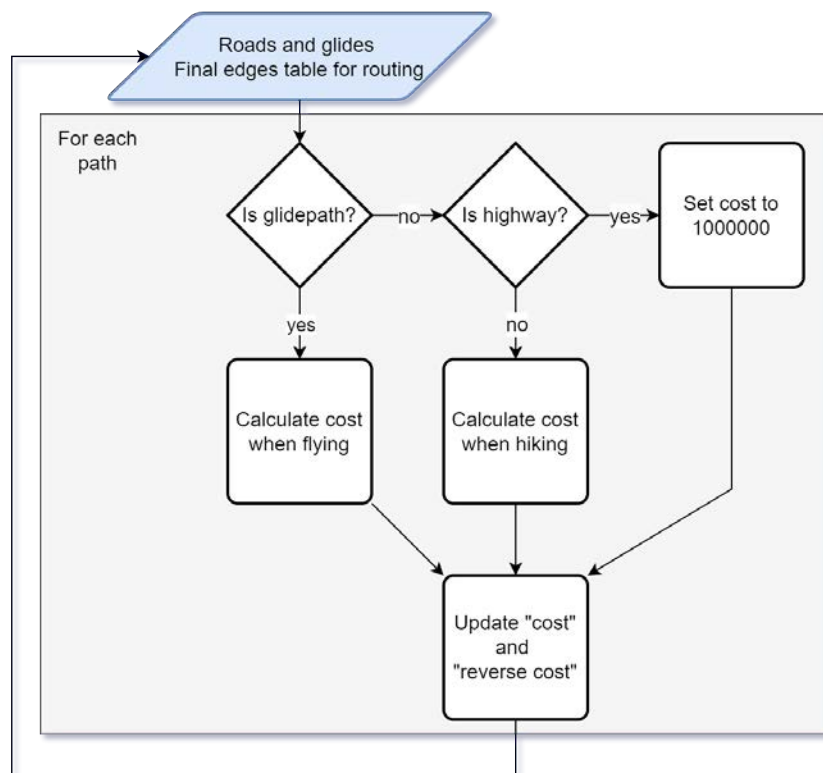


Figure 8.23.: Flow Chart of the determination of cost.

8.8.1. Hiking

For each path on the ground, the cost to travel across it is calculated as demonstrated in equation 8.10.

$$cost = \left(\frac{\frac{time_{5km1000m} - time_{5km0m}}{5km}}{ascending\ slope\ difference} \frac{a}{l} + \frac{time_{5km0m}}{5km} \right) l \quad (8.10)$$

l is the length of the path and a the calculated ascent from step 8.6. As the input values are given for a distance of five kilometres, the values must be normalised accordingly. The ratio of 1000 vertical meters and five kilometres distance results in an ascending slope of 20%, for zero meters of altitude gain it is an ascending slope of 0%. Therefore the "ascending slope difference" is 20%. The corresponding cost function is visualised in figure 8.24 for the distance of five kilometres. The reverse cost for a path is calculated

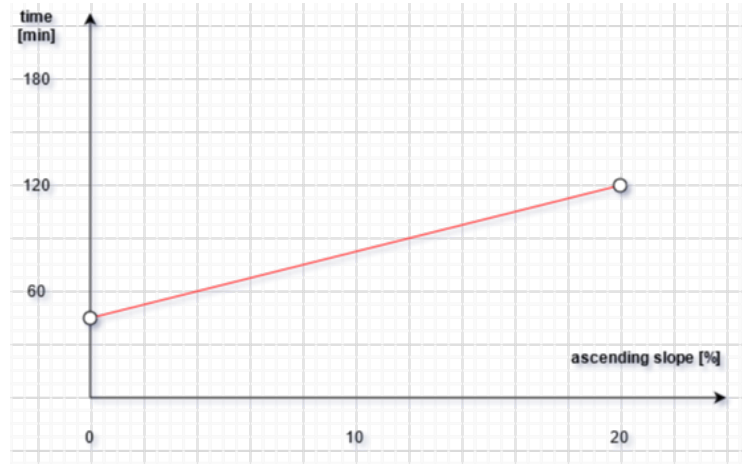


Figure 8.24.: Cost function for hiking paths for a given distance (5 km with no altitude gain would take 45 Minutes, 5 km with 1000m vertical gain would take two hours or 120 minutes.)

similarly by using the descent instead of the ascent. Additionally, highways are classified as unusable by foot and therefore the cost for highways is set to one million.

8.8.2. Flying

The cost for a glidepath is calculated as

$$cost = time_{to} + \frac{d}{1.4 \cdot 60} + time_l \quad (8.11)$$

where d represents the altitude difference between the take-off and the landing area. $1.4[m/s]$ is the average sink rate of an EN-A glider as described in section 2.1. This value times 60 results in the meters of sink per minute. As it is not possible to fly from a landing zone to a take-off, the reverse cost is set to one million.

8.9. Setting Of Restrictions

There are five different restrictions which can be applied to all glidepaths: wind restrictions, restrictions to only use official take-offs and/or landings, a minimum arrival altitude over a landing and a minimum of altitude difference between the take-off and the landing. A schematic overview is given in the flow chart 8.25.

Wind Restrictions

To address different wind situations, both the wind speed and its incoming direction are crucial. The stronger the wind, the more demanding it gets to launch from a take-off. Additionally, if the wind speed exceeds three kilometres per hour, the wind direction needs to be taken into account as well. Therefore, depending on the wind speed w_s the cost for travelling along a glidepath is multiplied by a factor w_f . A factor of one indicates it is easy to take-off, a factor of 1.2 already tells that it is more demanding and therefore is going to take more time. A wind speed between twenty and thirty kilometres per hour can lead to several attempts before a pilot can make it into the air. This problem is addressed by setting the factor to 1.4. If the wind is even stronger than that, it is not possible or not safe to

8. Algorithms

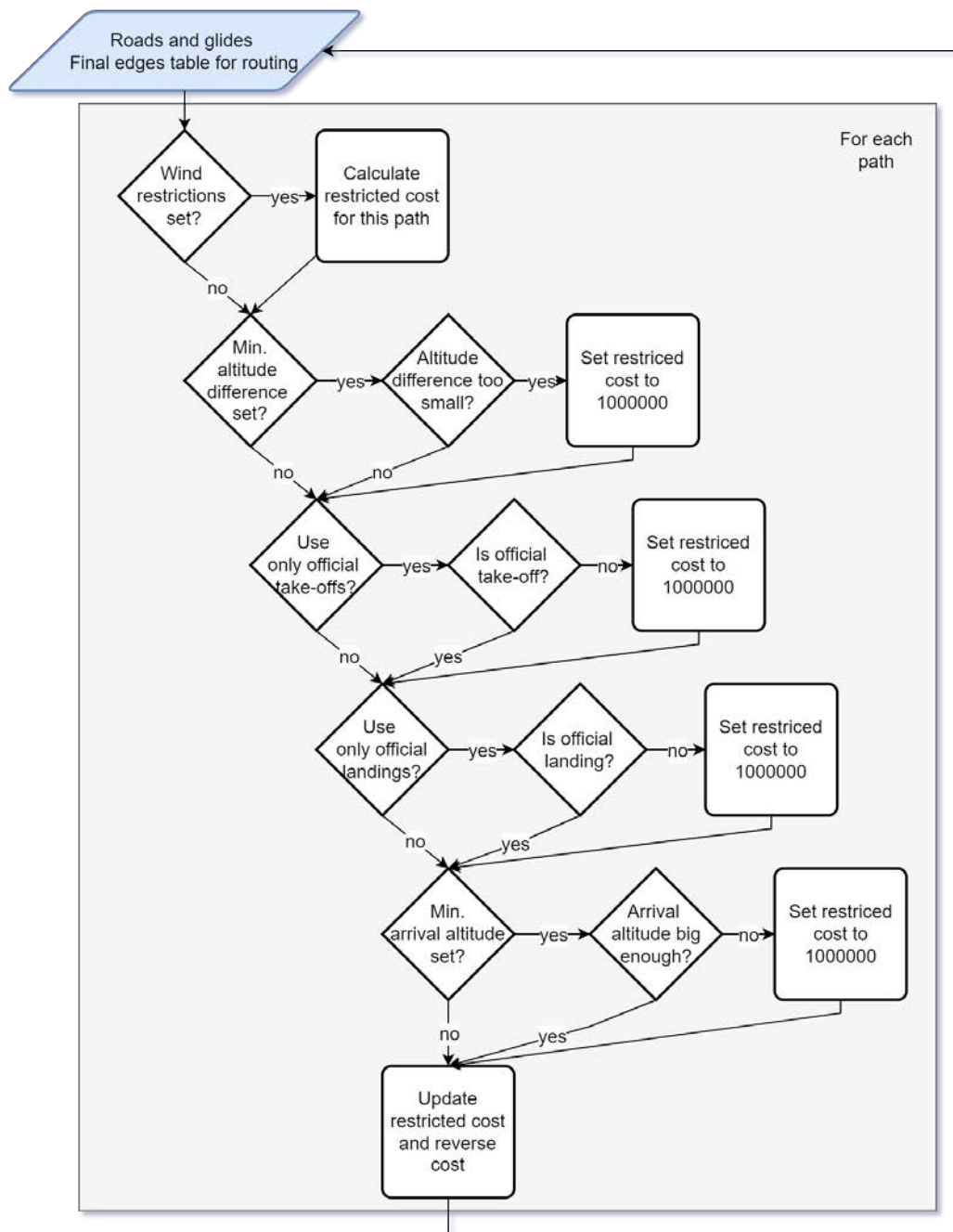


Figure 8.25.: Flow Chart of the setting of restrictions.

launch and therefore the factor is set to one million. The calculation of this factor w_f can be seen in the algorithm 8.2, and is used in the algorithm 8.3 to estimate the time needed to fly along a glidepath.

Algorithm 8.2 Calculation of the wind factor

```

1: switch  $w_s$  do
2:   case  $w_s \leq 3.0$ 
3:      $w_f = -1.0$ 
4:   case  $3.0 \leq w_s \leq 15.0$ 
5:      $w_f = 1.0$ 
6:   case  $15.0 \leq w_s \leq 20.0$ 
7:      $w_f = 1.2$ 
8:   case  $20.0 \leq w_s \leq 30.0$ 
9:      $w_f = 1.4$ 
10:  case  $30.0 \leq w_s$ 
11:     $w_f = 1000000.0$ 

```

In algorithm 8.3 to_d describes the possible take-off direction for the glidepath and w_d defines the wind direction.

If the wind-factor w_f is set to minus one, the wind direction does not need to be taken into account. In case the wind-factor is set to one million it indicates that it is not flyable at all and thus all glidepaths get their cost set to one million. In any other case the cost is calculated as the initial cost times the wind factor, if the take-off direction coincides with the wind direction (it is only possible to launch with headwind), else the cost is set to one million.

Algorithm 8.3 Re-Calculation of the cost for glidepaths in respect to a given wind situation

```
1: switch  $w_f$  do
2:   case  $-1.0$ 
3:      $cost_{restricted} = cost$ 
4:   case  $1000000.0$ 
5:      $cost_{restricted} = 1000000.0$ 
6:   case  $1.0||1.2||1.4$ 
7:     switch  $to_d$  do
8:       case  $N$ 
9:         if  $315.0 < w_d || w_d < 45.0$  then
10:           $cost_{restricted} = cost \cdot w_f$ 
11:        else
12:           $cost_{restricted} = 1000000.0$ 
13:        end if
14:       case  $NE$ 
15:         if  $0.0 < w_d < 90.0$  then
16:           $cost_{restricted} = cost \cdot w_f$ 
17:        else
18:           $cost_{restricted} = 1000000.0$ 
19:        end if
20:       case  $E$ 
21:         if  $45.0 < w_d < 135.0$  then
22:           $cost_{restricted} = cost \cdot w_f$ 
23:        else
24:           $cost_{restricted} = 1000000.0$ 
25:        end if
26:       case  $SE$ 
27:         if  $90.0 < w_d < 180.0$  then
28:           $cost_{restricted} = cost \cdot w_f$ 
29:        else
30:           $cost_{restricted} = 1000000.0$ 
31:        end if
32:       ...
```

Other Restrictions

In case only official take-offs should be used, the costs for all glidepaths, which lead away from an unofficial take-off, are set to one million. If only official landings should be approached, the costs for all glidepaths leading to an unofficial landing are set to one million.

In order to have a safety altitude over the approached landing, a minimum arrival AGL can be set. If this limit is not full-filled, the cost is set to one million.

Glidepaths which indicate short flights can be excluded from the computations too by setting a minimum altitude difference for all flights. Accordingly, when the altitude difference is lower than the set minimum, costs are set to one million. This restriction is implemented as for some pilots shorter flights are unappealing.

8.10. Calculating The Closest Vertex

The closest vertex can be extracted from the database by using the build-in function `st_distance` from the `pg_routing` toolbox. This tool calculates the distance from a pair of coordinates to a given list of points, in this case the vertices of the routing network. The results are ordered by the distance as well as limited to one entry to get the closest vertex to the point, where the routing should start from (see diagram 8.26). The extracted vertex *id* is then needed for the routing functions.

8.11. Xalps - Routing (A To B)

The first developed routing algorithm deals with sequential routing, a routing analysis that is needed in races as the Red Bull Xalps. These races require their athletes to reach a certain number of waypoints in a given order. All invented algorithms for the use in competitions (the routing algorithm for sequential routing explained in this section and the algorithm for out and return calculations (explained in section 8.12)) simulate, that

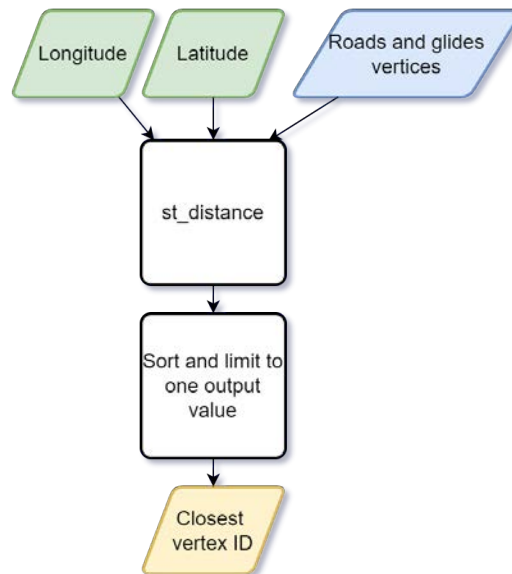


Figure 8.26.: Flow Chart of extracting the closest vertex

a pilot is only gliding down without gaining altitude during the flight. Calculations for these specific glidepaths around a take-off can be seen in section 8.4. Diagram 8.27 gives an overview on the most important steps when calculating a route via several waypoints. First of all, the users can decide whether they want to use restrictions or not. Depending on their choice, either the cost with or without restrictions (see section 8.8 and 8.9) is used in the routing algorithm. Furthermore, depending on the users' input, the optimized route between two vertices can be calculated by using the Dijkstra or A* algorithm. Depending on the number of input vertices it is a simple route or a sequential route calculation, meaning that firstly the individual route sequence between vertex v_1 and v_2 , then v_2 and v_3 and so on, is calculated, before secondly, the routes get combined as one big route between the first and the last point via several waypoints in between. The important thing when using sequential routing is to add the aggregated cost of the previous sequence to the next part of the route. This will lead to cumulative cost of the entire route in the end, rather than receiving just the cost of the last sequence. For the route calculation itself the **pg_routing** functions **pgr_dijkstra** or **pgr_astar** are used. For faster computation a bounding box with a 0.05 degree buffer around the straight line between

8. Algorithms

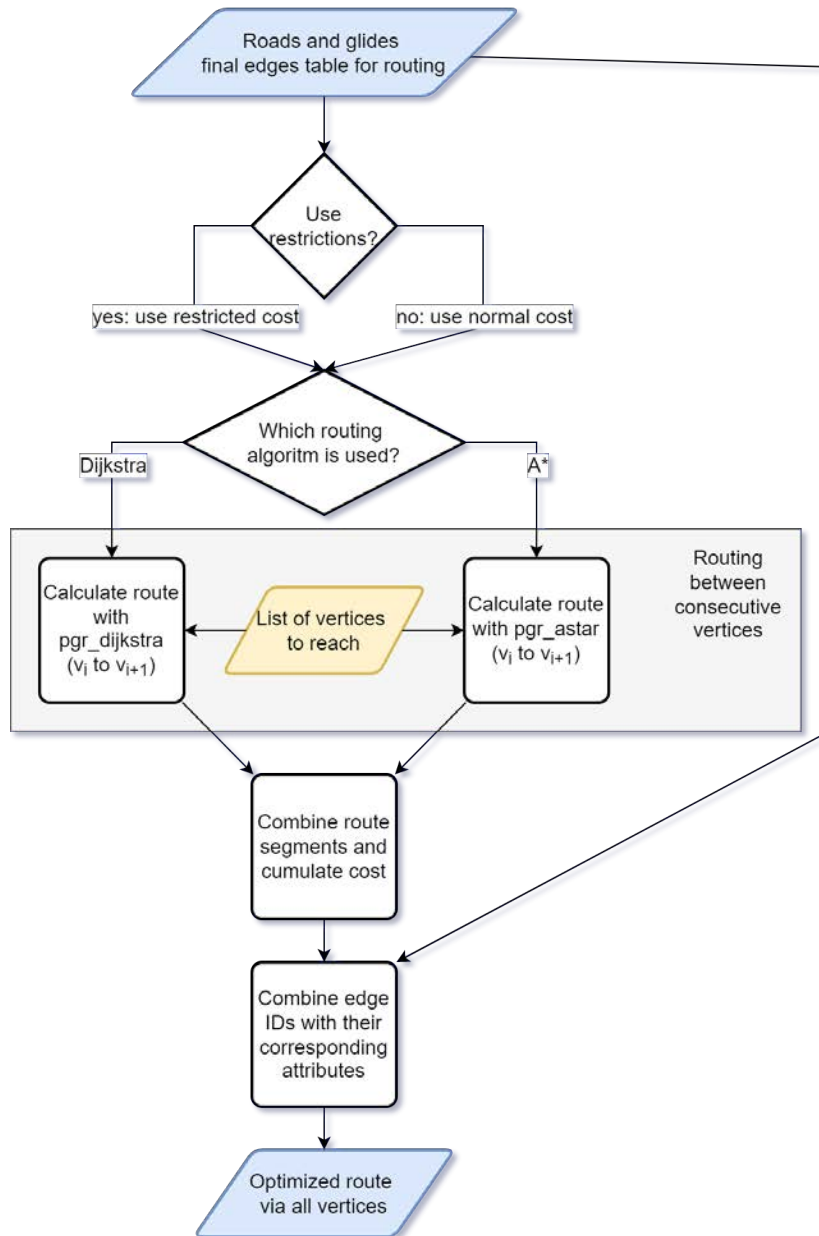


Figure 8.27.: Flow Chart for Xalps - Routing (A to B)

the start and end vertex of each sequence is used. This does not affect the quality of the route calculation as the travelling pace of a hiker is relatively slow regardless of the type of pathway. Therefore, big detours from the line-of-sight connection always result in a slower route. The result, a list of edge-ids, is then joined with the **roadsandglides** table for connecting the attributes and stored in a temporary table for being visualised and analysed on the website.

8.12. Bordairrace - Routing (Out And Return)

Another type of hike and fly race is the so called Bordairrace (explained in section 1.2.2). In this race, the goal is to reach the furthest point within a set time window and return to the start. Therefore, a second routing option was invented.

As for the Xalps - Routing, the users can decide, if they want to use restrictions and which algorithm (Dijkstra or A*) should be used for the calculations (first two steps of the calculations are the same as in diagram 8.27). Given this information, the process of finding the optimized route, and therefore the furthest vertex reachable with the need to come back within a set time frame, is shown in diagram 8.28.

The first step, when looking for the best option for a Bordairrace, is to calculate the line-of-sight distance from the source vertex to all other vertices using the **pg_Routing** tool **st_distance**. An example output is shown in table 8.2. As the next step a One-to-Many calculation is performed from the start vertex to all surrounding vertices. For performance reasons only the vertices within a given bounding-box are taken into account. The bounding-box is a buffer around the straight line distance between the start and the end vertex. The distance from this straight line is set in arc-degrees which calculate as

$$d[arc - degree] = 0.05 \left[\frac{arc - degree}{minute} \right] \cdot max_{time}[minutes] \quad (8.12)$$

with max_{time} being the time allowed for out and return. $0.05 \left[\frac{arc - degree}{minute} \right]$ is the travelling velocity which defines the bounding-box extend and represents an average velocity of 240 kilometres per hour, which is more than six times

8. Algorithms

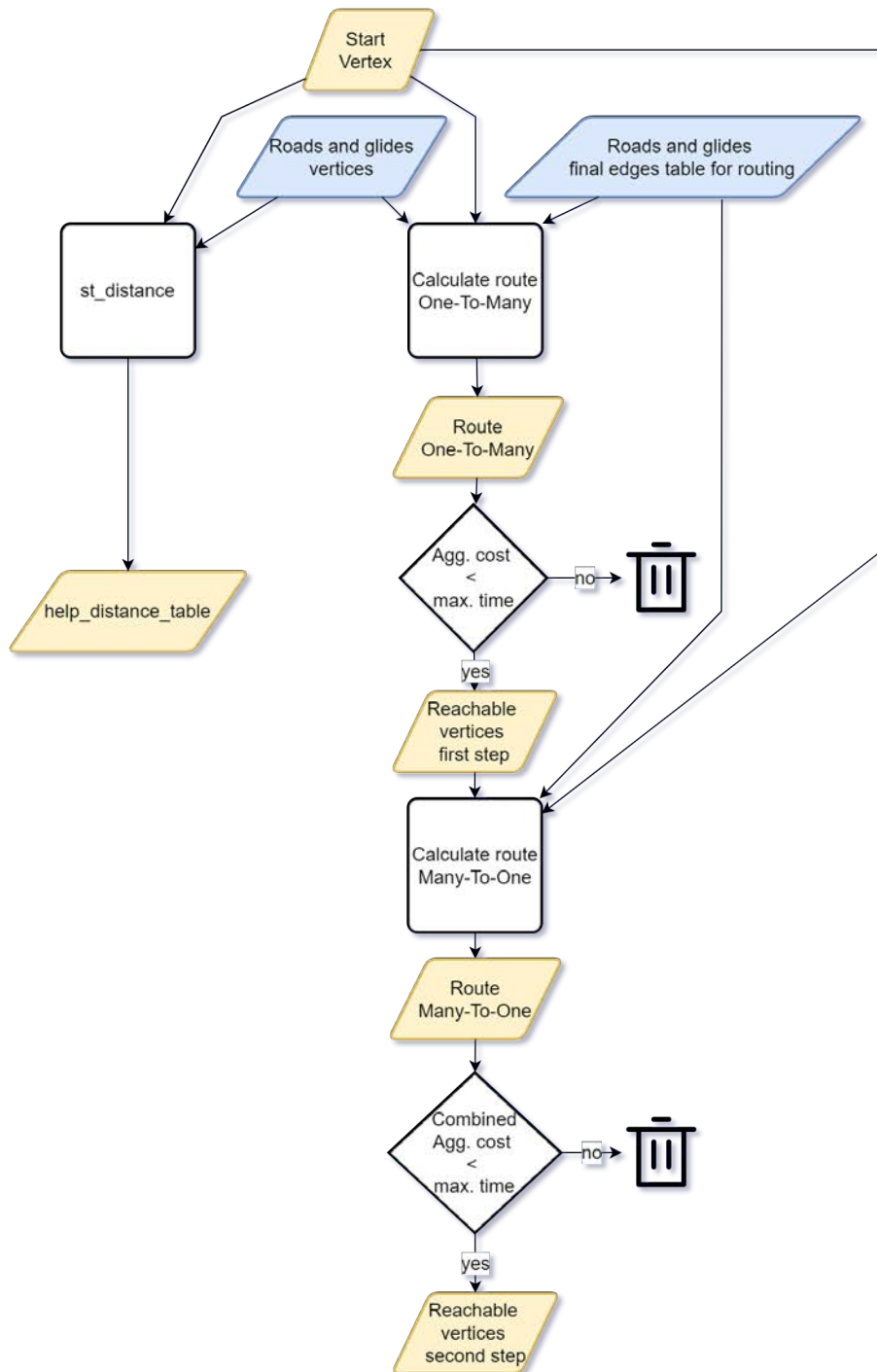


Figure 8.28.: Flow Chart for Bordairrace - Routing (Out and Return), part 1

id	distance
412195	0
136928	10,3677315
412194	28,8301397
412196	30,7356671
136927	32,3591367
226992	33,1335568
136929	34,41877
...	...

Table 8.2.: Line-of-sight distance to other vertices

the speed you can achieve with a paraglider.

After that, the route back to the source is calculated from all vertices which are reachable within the set time-frame. This data is taken from the first calculation (Many-to-One calculation). Thus, we get two result sets for each combination, namely the route and cost from source v_S to the target v_T and the route and cost from the target v_T to the source v_S . If the combined cost of both lies within the set time frame, this pair is possible and therefore the vertex v_T becomes a potential candidate. Out of these possible combinations the best option is chosen based on the furthest distance from v_S to v_T that was calculated during the first step (calculating the straight distance to every vertex). This evaluation steps are shown in diagram 8.29. Figure 8.30 schematically shows the different calculation steps. All vertices are represented as little circles. The big purple shaded circle indicates the bounding-box. The source vertex is filled in purple. The vertices that are within the bounding box but not reachable within the set time frame are red. If the connection between the source and the target vertex is a normal red arrow, it indicates that already in the first step the time needed to travel to that vertex exceeded the time frame and hence this vertex is excluded from the next step (calculating the return-route). In case it is a red double arrow, the time for travelling there and back exceeds the allowed time frame. Green double arrows indicate that it is possible to travel to this specific vertex and come back within the allotted time. All those vertices are coloured yellow. As mentioned before, the best option out of these possibilities is chosen based on the longest distance calculated in the very first step. In this graph

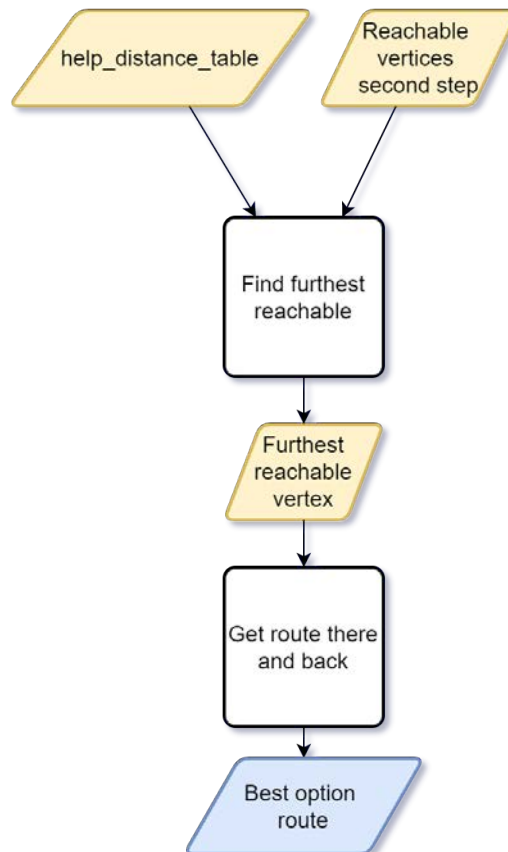


Figure 8.29.: Flow Chart for Bordairrace - Routing (Out and Return), part 2

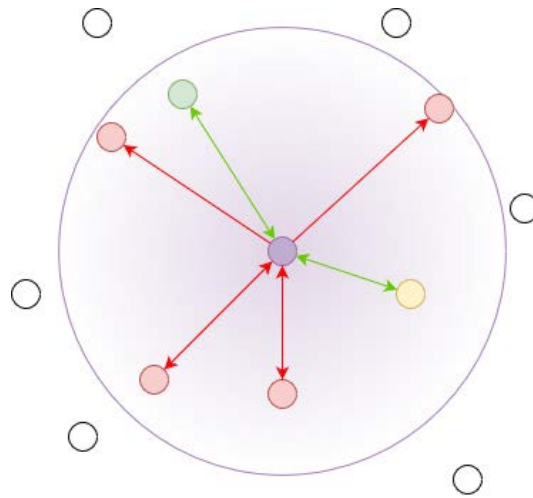


Figure 8.30.: Schematic representation of the out and return calculation

it is shown as green circle.

8.13. Routing To Reachable Take-Offs

Despite the assumption that a pilot is only gliding down from a take-off, it is sometimes possible to climb higher with the help of thermals and continue on to a cross country flight of several hundred kilometres through the air. As such special flights exceed the routing possibilities invented in the first two algorithms (see 8.11 and 8.12) a third routing problem was invented, namely a search for reachable take-offs nearby with the idea of starting a cross country flight from there. This is helpful in a situation as being in the valley on a day with good flying conditions and the need to find the fastest reachable take-off nearby, or just get the information what options there are to get airborne again.

This calculation can furthermore be used by leisure pilots if they want to find take-offs for a hike and fly around their own position.

To find possible take-offs around the current position it is first and foremost necessary to have a list of all vertices which are take-offs. This information as well as take-off specific attributes can be extracted from the **roadsandglides**

and **roadsandglides_vertices_pgr** table and then stored in a separate table for further use (see diagram 8.31). By extracting the take-offs in this way, only take-offs from which at least one landing can be reachable are taken into account. This can be seen as a safety measure, as in case there is no thermal activity a safe landing is still guaranteed. Example data is shown in table 8.3. A set of potential take-off candidates is extracted from the data,

id	the_geom	to_direction	official_t	to_elevation	max_alt_diff
8	...	N	FALSCH	1842,72	881,082
10	...	N	FALSCH	2248,31	1413,25
12	...	NE	FALSCH	884,125	133,887
14	...	SE	FALSCH	1164,96	221,548
16	...	E	FALSCH	1793,92	1074,03
18	...	E	FALSCH	1913,67	334,467
...	...	...	...	...	...

Table 8.3.: Take-offs view

depending on the users' preferences such as to use only official take-offs, the elevation of the take-off, the take-off direction or the minimum altitude difference between the take-off and the landing. The diagram 8.32 illustrates the different calculation steps.

As in section 8.12 a One-to-Many route calculation is done in the next step. If the user wishes to come back to the starting position afterwards (a functionality added for leisure pilots which only want to glide down) a Many-to-One calculation follows, which is similar to section 8.12. As for the Bordairrace routing calculation, the user can set a maximum time allowed for the routes. In contrast to the Bordairrace-route, the user can get more than one result, if several take-offs are reachable within this time frame and meet the users' preferences. As also the statistics of each option are listed, the users can then decide which option they prefer.

8. Algorithms

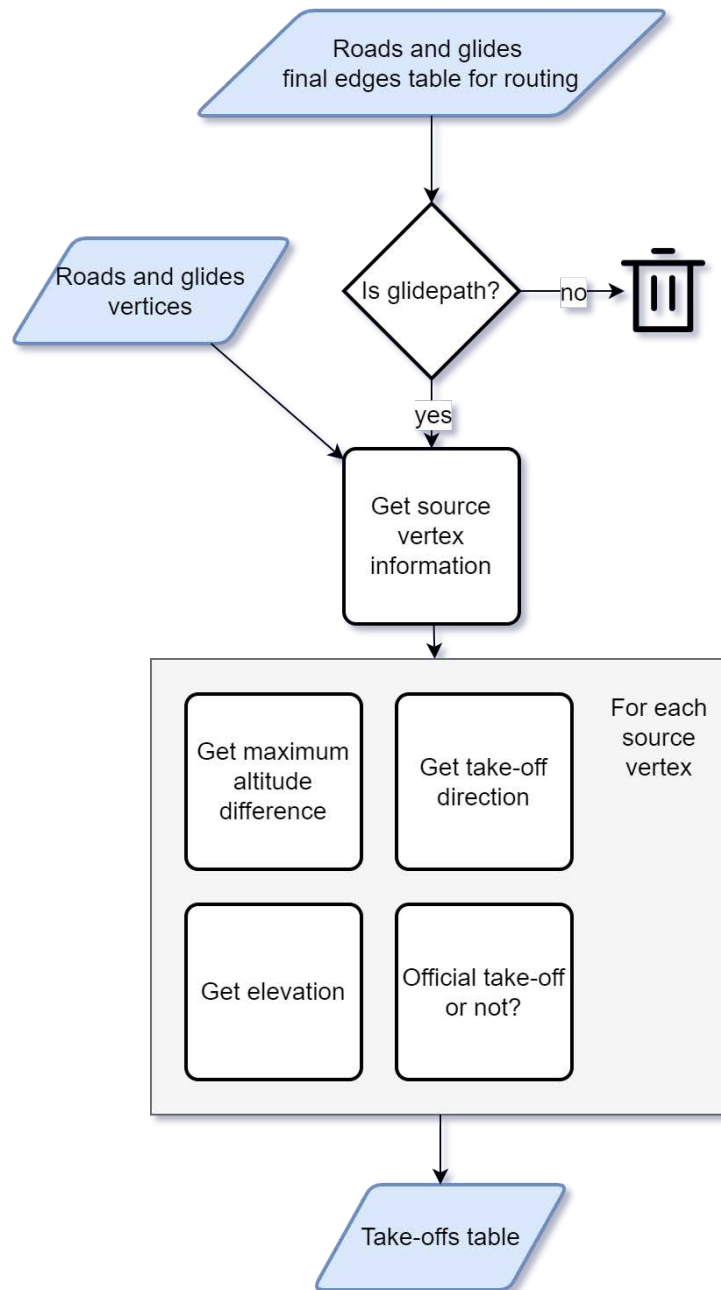


Figure 8.31.: Flow Chart extracting the take-offs from the database

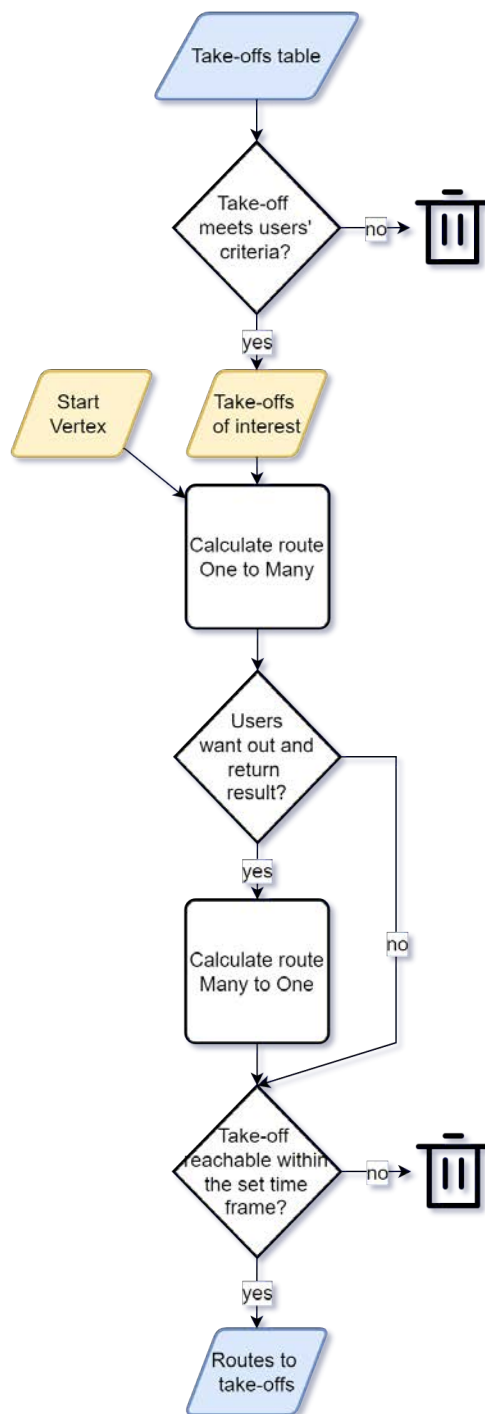


Figure 8.32.: Flow Chart of calculating the routes to reachable take-offs

9. Realisation

On the third developmental level of this master's project a webpage was programmed on which the users can calculate the shortest-paths, set their individual routing parameters, and inspect their routing results. The different features are listed in section 9.1. This webpage, as well as the PostgreSQL database, are prototypes only running on a local machine and therefore are not accessible on the internet.

9.1. Webpage: Hike And Fly Routing

When opening the *Hike and Fly Routing* webpage an empty map is shown (see figure 9.1). On this page, route calculations can be done, the parameters for the routing cost can be set as well as restrictions can be applied. The page offers the option to choose between the three different routing options *Xalps-Routing (A to B)*, *Bordairrace-Routing (Out and Return)* and *Routing to reachable take-offs* as well as to decide whether the Dijkstra or A* algorithm should be used. Routing waypoints can easily be set by clicking on the map. In order to change their position users must activate the *Modify* radio button (visible in figure 9.2) and then drag the waypoints along the map. When using *Xalps-Routing (A to B)* at least two waypoints must be set on the map, for the other two routing options only one starting point is necessary. As soon as a waypoint is set on the map, its coordinates are displayed below the map. Furthermore, if a route calculation is done, the statistics of this route are shown in a table (see figure 9.2). The route on the ground is displayed as solid line, the flying parts are shown as dotted lines as shown in figure 9.3. Waypoints are displayed as red markers.

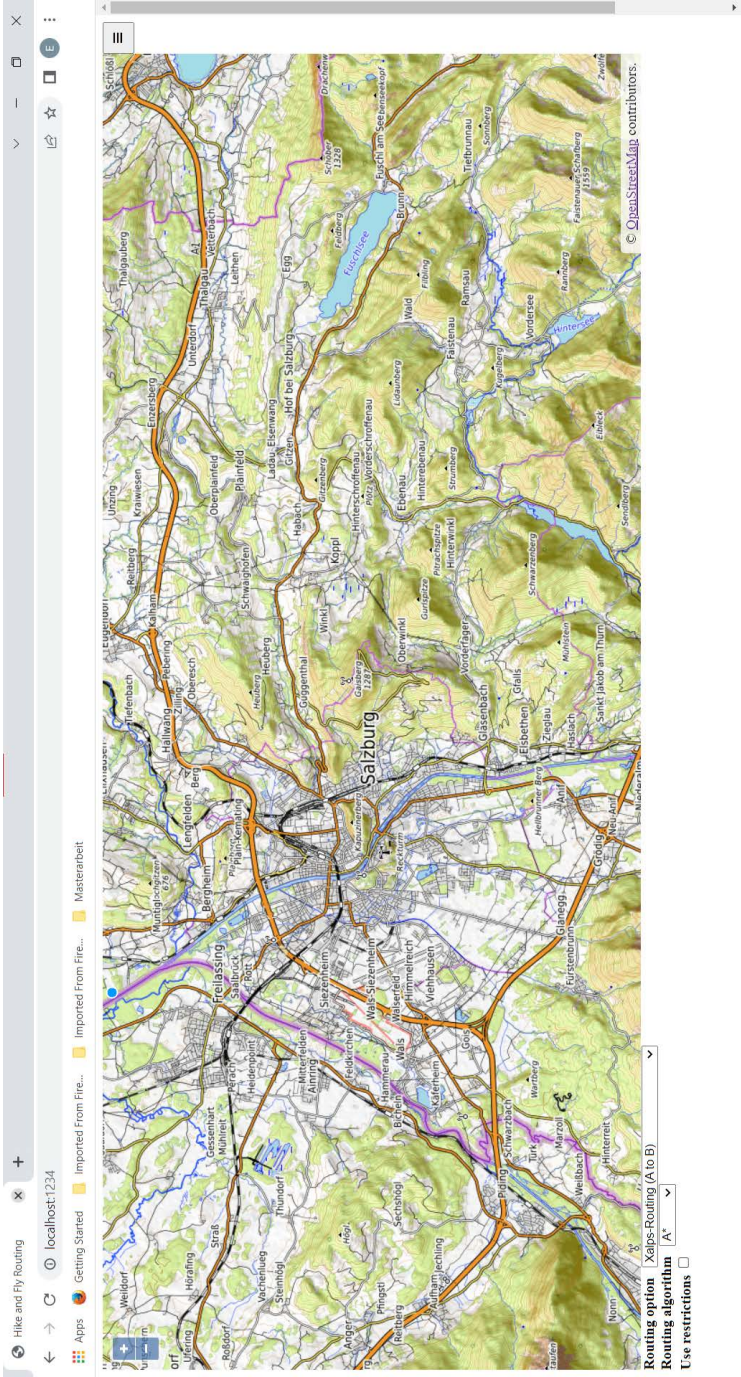


Figure 9.1.: Main window of the Hike and Fly Routing webpage

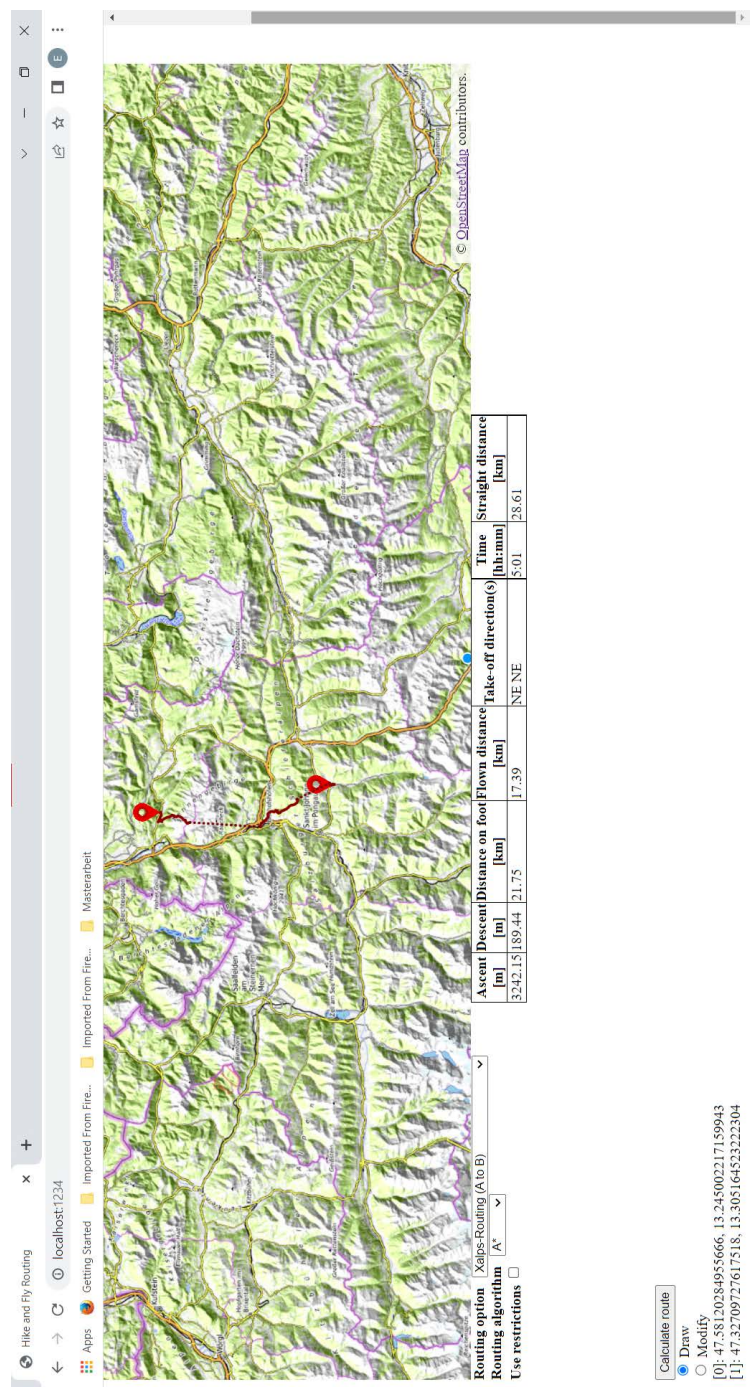


Figure 9.2.: Main window of the Hike and Fly Routing webpage with a calculated route

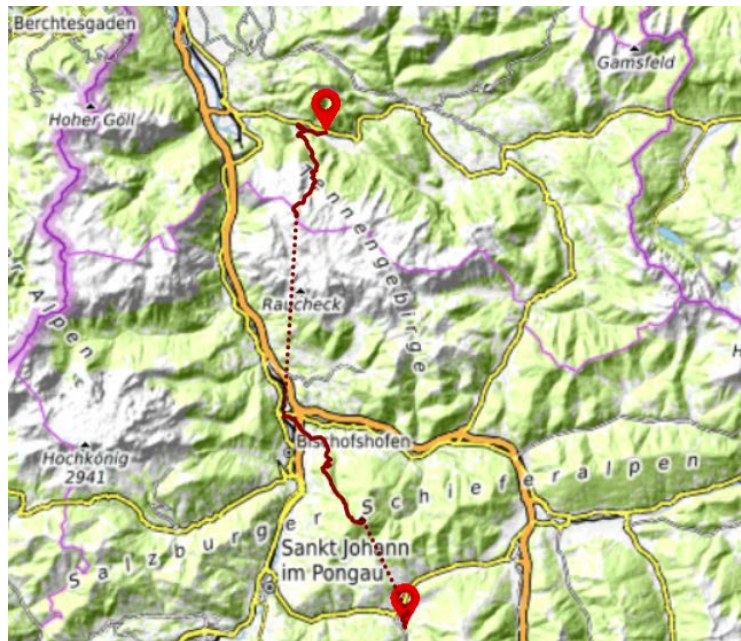


Figure 9.3.: Close up of a displayed route

In the upper right corner of the webpage a menu (displayed as a gray box with three dashes in figure 9.1) can be found. The routing parameters as well as the restrictions can be set there. The left graph in figure 9.4 displays the menu which opens up upon a click on the button. When hovering over a certain parameter a hint text is shown which can be seen in the right image. For the time needed for five kilometres with or without altitude gain, the users can set a value between one minute and five hours. Values between one minute and one hour are options for the time needed at the take-off and at the landing. When clicking on the **Recalculate cost** button the set values are interpreted by the website and sent to the PostgreSQL database for doing the calculations as shown in diagram 9.5. Furthermore, the cost and reverse cost for each path are updated in the **roadsandglides** table. In this menu, users thus have the chance to set restrictions (see figure 9.6). The different options for setting restrictions are explained in section 8.9. The users can decide whether or not they want to set a single restriction or multiple at the same time. In the example shown, the users expect a wind of fifteen kilometres per hour from the west and only want to use official take-offs and landings where the altitude difference between them is more

Figure 9.4.: Cost parameters

than 500 meters. Additionally, for safety reasons only glidepaths should be used which guarantee that the athlete reaches the landing with more than 100 meters above the ground.

The dataflow is similar to the recalculation of the cost as shown in diagram 9.5 except that the data concerning the restrictions is sent to the database instead of the cost parameters. In this scenario the restricted cost are recalculated and the corresponding columns are updated.

Depending on the chosen routing option, additional information is needed for the route calculation. For *Xalps-Routing (A to B)* a minimum of two waypoints need to be set on the map. For *Bordairrace-Routing (Out and Return)* and *Routing to reachable take-offs* only one waypoint but additional input is needed from the users (see figures 9.7 and 9.8). In both cases the allotted time for the entire route must be set.

In case the users are looking for a reachable take-off, the search can be narrowed down with the help of these criteria:

- **Official take-off**, if only official take-offs should be taken into account

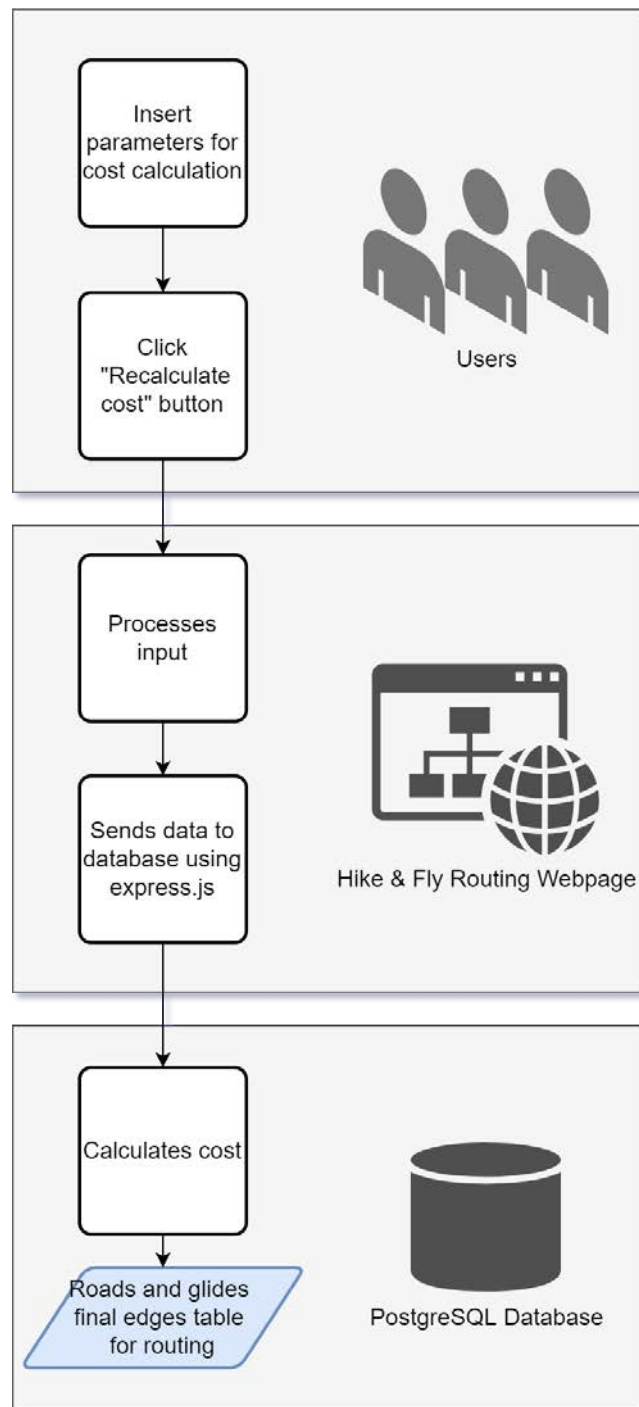
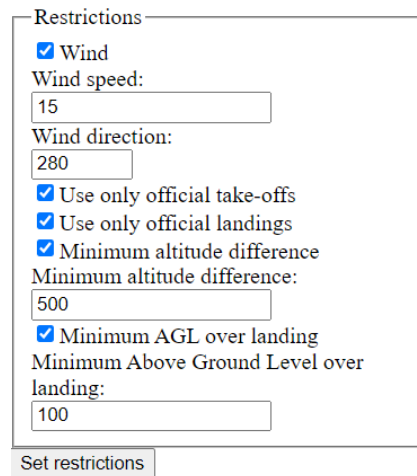


Figure 9.5.: Data flow for cost calculation (Users - Webpage - Database)

9. Realisation



Restrictions

☒ Wind

Wind speed:

Wind direction:

☒ Use only official take-offs

☒ Use only official landings

☒ Minimum altitude difference

Minimum altitude difference:

☒ Minimum AGL over landing

Minimum Above Ground Level over landing:

Set restrictions

Figure 9.6.: Set restrictions menu on the webpage

- **Take-off direction**, if the users are searching for a certain aspect; for example south-east take-offs offer the best thermal activity in the morning
- **Minimum elevation** and
- **Maximum elevation**, to set an elevation range in which the take-off should be within; for example if there is still snow above 2000 m those take-offs must be excluded
- **Minimum altitude difference**, to guarantee that even without thermal activity a long glide down is possible
- **Out and Return**, should be clicked if the users want to come back to the same point where they started from at the beginning. In case this is clicked, the route to the take-off and back must be achievable in the allotted time.

When all the parameters are set, a route can be calculated and the data is processed as shown in diagram 9.9. More than one result is possible when calculating the routes to reachable take-offs (see figure 9.10). The paths are coloured individually in order to distinguish possible take-offs more easily. The statistics table also includes more information about each take-off, that includes whether it is an official take-off or not, which take-off directions are possible, the elevation of the take-off as well as the maximum altitude difference. When clicking on one of the take-off markers, this

information is shown in a pop-up window. All of these features lead to a better understanding of the calculated routes and support the users need to find the best route during a race. It also supports them when searching for the optimal take-off for a leisure hike and fly.

The variety of options offered by this webpage prototype makes it possible to individualise the routing process as not every user has the same physical abilities, tactical questions, and context when it comes to routing.

Routing option

Routing algorithm

Use restrictions ☐

Time available Minutes

Figure 9.7.: Set parameters for Bordairrace-Routing

Routing option

Routing algorithm

Use restrictions ☐

Time available Minutes

Official take-offs ☐

Take-off direction

Minimum elevation m

Maximum elevation m

Minimum altitude difference m

Out and Return ☐

Figure 9.8.: Set parameters for routing to take-offs

9. Realisation

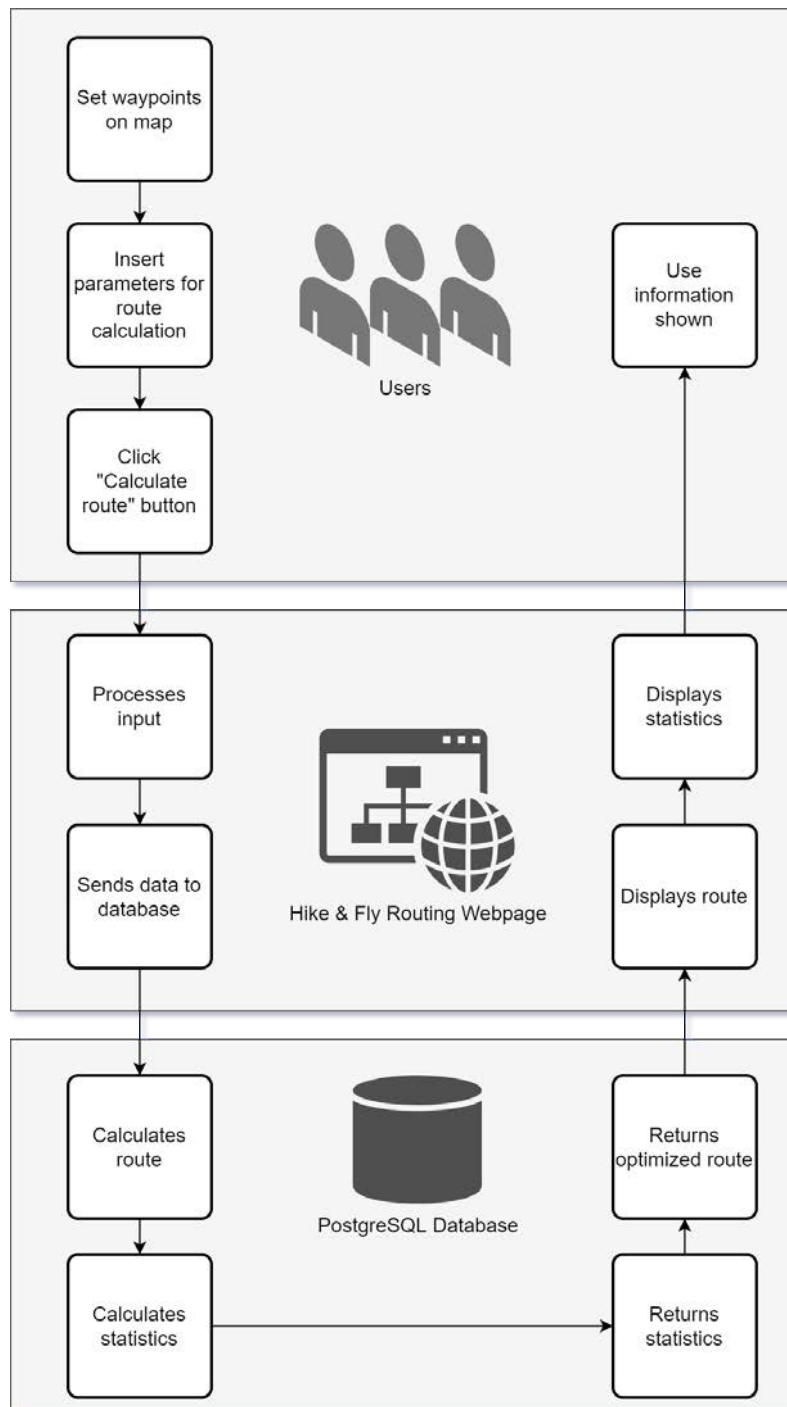


Figure 9.9.: Flow chart of route calculation (Users - Webpage - Database)

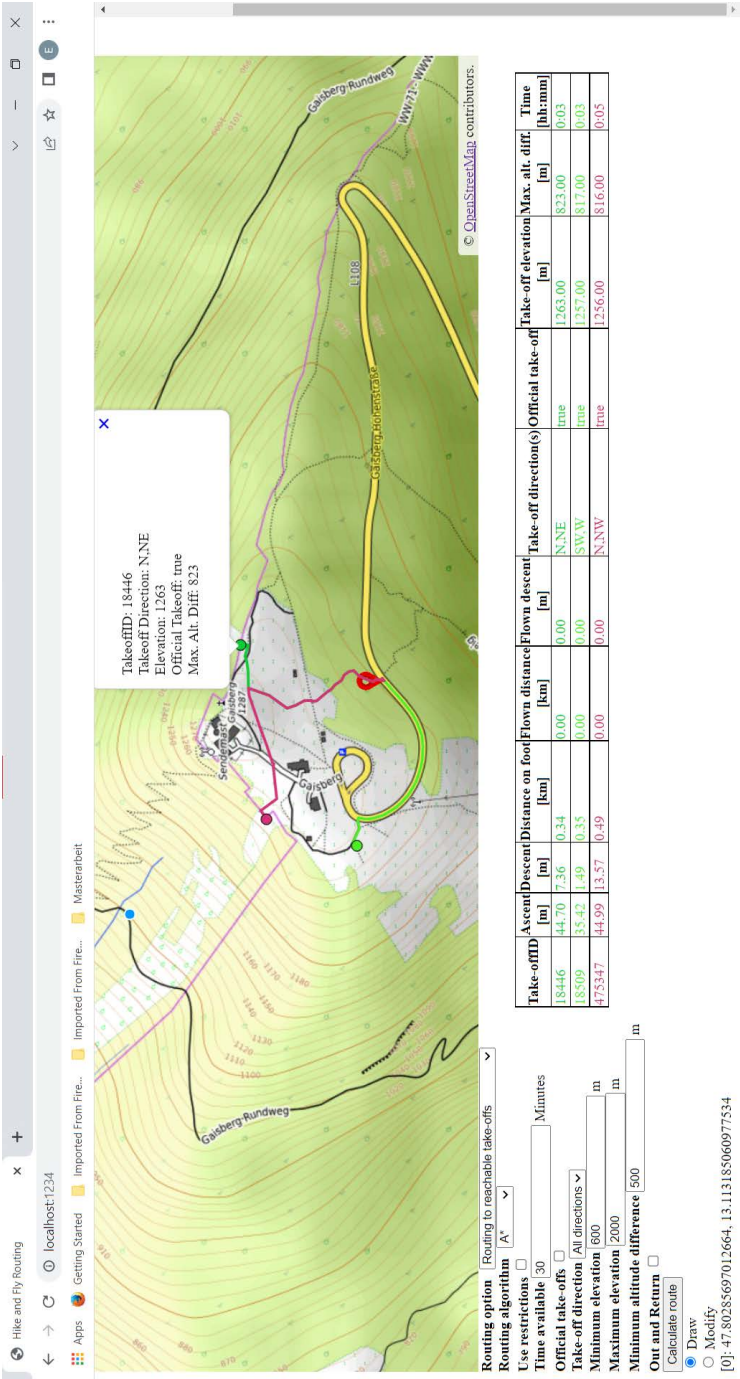


Figure 9.10.: Main window of the Hike and Fly Routing webpage with calculated routes to various take-offs

Part IV.

Results

10. Routing Graph

This chapter will give a statistical overview on the generated routing graph based on the OSM street network, the calculated take-offs and landings, and the glidepaths. The number of take-offs and landings interconnected by glidepaths and used in the routing graph are shown in table 10.1. It is clearly

Unoff. landings	Off. landings	Unoff. take-offs	Off. take-offs
5406	24	12971	79

Table 10.1.: Number of take-offs and landings

shown that there are more unofficial take-offs and landings than officially declared ones. Table 10.2 shows the number of different edges in the routing graph which sum up to 1621970 edges used for the routing calculations. An edge classified as glidepath describes the connection between a take-off and a landing. The offroad edges indicate the connection between the OSM street network and all take-offs and landings even if some of them are dead ends where no glidepath starts or end. Such dead ends occur when there is a suitable take-off but no landing in reach. These statistics prove that there are boundless possibilities when flying with a paraglider as you have more

Number of edges	Edge type	Official take-off	Official landing
987603	"glidepath"	false	false
6019	"glidepath"	false	true
10816	"glidepath"	true	false
141	"glidepath"	true	true
60318	"offroad"		
557073	"osm network"		

Table 10.2.: Number of edges in the routing graph

than double the options than when using the street network on the ground only. Furthermore, if taking any suitable grass for a take-off or landing into account, and not only the officially declared ones, there are about 200 times the possibilities than before.

Especially for landing a paraglider there are a vast number of fields which are suitable (big enough, flat and no obstacles around). Many of those areas are even reachable from official take-offs, e.g. 10816 glidepaths start at an official take-off but end at an unofficial landing. Only 141 glidepaths connect an official take-off with an official landing. For each edge on the ground, the accompanying ascent and descent have been calculated. Likewise, every glidepath includes information about the altitude difference between take-off and landing, the possible wind direction, the arrival altitude over the landing field, the take-off elevation, and whether it connects an official or unofficial take-off with an official or unofficial landing. All of this information is necessary for the cost calculation and to adapt to restrictions or users' needs.

11. Test Cases

In this thesis, three routing possibilities were examined: First, A to B routing that is useful in competition such as the Red Bull Xalps (section 11.1); second, the usefulness of out and return routing for competitions such as the Bordairrace (section 11.2); and third, the search for take-offs (section 11.3) which can be used in every competition during flyable days. For the third option the flight-route can not be calculated on the basis of this thesis as it only takes glides down into account and no long-distance cross country flights. The routing provided in this thesis, however, is useful for amateur pilots too, if they are in a new area and want to know which nearby options are available for flying.

11.1. A To B

As first testcase, the first part of the Red Bull Xalps route was chosen, more precisely the section between the turnpoints TPo Mozartplatz, TP1 Gaisberg and TP2 Wagrain-Kleinarl. An overview is given in graph 11.1. The coordinates of the turnpoints are listed in table 11.1. This section of the competition was similar in the editions of 2017 (unflyable due to heavy wind), 2019 (moderate flying conditions but low winds) and 2021 (strong

Turnpoint	Latitude [°]	Longitude [°]
TPo Mozartplatz	N47,7988	E13,0477
TP1 Gaisberg	N47,8044	E13,1107
TP2 Wagrain-Kleinarl	N47,2746	E13,3186

Table 11.1.: Turnpoint coordinates

11. Test Cases



Figure 11.1.: Map of the turnpoints TP0 Mozartplatz, TP1 Gaisberg and TP2 Wagrain-Kleinarl

11. Test Cases

Parameter	Eli Egger	Aaron Durogati
time 5km, flat	1:00 [h:mm]	0:45 [h:mm]
time 5km, 1000m altitude difference	2:00 [h:mm]	1:00 [h:mm]
time needed at take-off	0:10 [h:mm]	0:05 [h:mm]
time needed at landing	0:10 [h:mm]	0:08 [h:mm]

Table 11.2.: Cost parameters Eli Egger and Aaron Durogati

northern winds which narrowed down the take-off options). Besides these varying conditions, namely wind direction and speed, the differences in the optimised route, when only using official take-offs, was also examined. Furthermore, the effect of the cost parameters for myself and for Aaron Durogati (shown in table 11.2) were investigated. These cost parameters were set by experience. Aaron Durogati is a professional paragliding pilot who took part at the Red Bull Xalps as athlete several times. In 2019 and 2021 I had the opportunity to support him during the race. It is quite apparent that there are vast differences between the time I need for uphill hiking or walking and the time it takes Aaron Durogati to cover the same distance and vertical meters. Equally interesting is the fact that the more experience a paragliding pilot has, the less time is needed for preparations at take-off, but the time needed for packing the glider stays almost the same throughout the different skill-levels. Figure 11.2 shows the result path for the first section, figure 11.5 shows the second section. As there is no difference between the results of the A* and the Dijkstra algorithm (as can be seen in table 11.3) only the path calculated with the A* algorithm is shown in those figures. For the heuristic evaluation number within the A* algorithm the line-of-sight distance between the start- and the endpoint of each edge is used. This can in some cases lead to longer calculation times than with Dijkstra, as those distances need to be calculate in every routing process again. Due to the narrow speed range when walking or running, the most direct route in most cases is usually the best route possible. Therefore, the possible disadvantage of having a suboptimal route, when using the A* algorithm compared to using Dijkstra's shortest-path algorithm, is negligible. It takes equally long to compute the route regardless of using A* for the whole graph or Dijkstra with a sub-graph. Without the use of sub-graphs, calculations with the Dijkstra algorithm take several hours as each connection in this graph of

11. Test Cases

several-million-edges needs to be examined.

Section	TP0-TP1				TP1-TP2			
	E. Egger		A. Durogati		E. Egger		A. Durogati	
Algorithm	A*	Dijkstra	A*	Dijkstra	A*	Dijkstra	A*	Dijkstra
Calc. Time [ms]	3122	3079	3150	3121	5247	5389	5467	5396
Ascent [m]	880,21	880,21	880,21	880,21	3664,5	3664,5	3664,5	3664,5
Descent [m]	28,85	28,85	28,85	28,85	510,38	510,38	510,38	510,38
Dist. on Foot [km]	7,08	7,08	7,08	7,08	51,24	51,24	51,24	51,24
Flown Dist. [km]	-	-	-	-	20,19	20,19	20,19	20,19
Take-off Direction(s)	-	-	-	-	NW NE N	NW NE N	NW NE N	NW NE N
Time [hh:mm]	02:17	02:17	01:16	01:16	15:35	15:35	09:56	09:56
Straight Dist. [km]	4,75	4,75	4,75	4,75	60,96	60,96	60,96	60,96

Table 11.3.: Results A to B routing for the section Mozartplatz (TP0) to Gaisberg (TP1) and Gaisberg (TP1) to Wagrain-Kleinarl (TP2)

11. Test Cases

Even on this short route the higher speed of Aaron Durogati is clearly visible. Utilizing his parameters, the calculated time is $1 : 16[h : mm]$ compared to $2 : 17[h : mm]$ when looking at my personal pace hiking uphill. The route itself has a length of $7.08km$ and along that path $880.21m$ altitude gain is achieved by only $28.85m$ of altitude loss. The straight distance represents the line-of-sight distance between the two turnpoints.

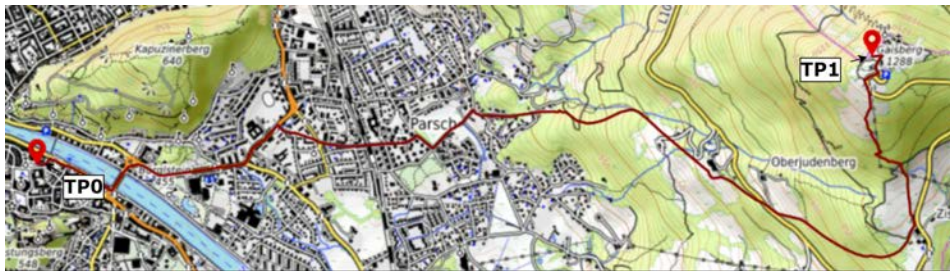


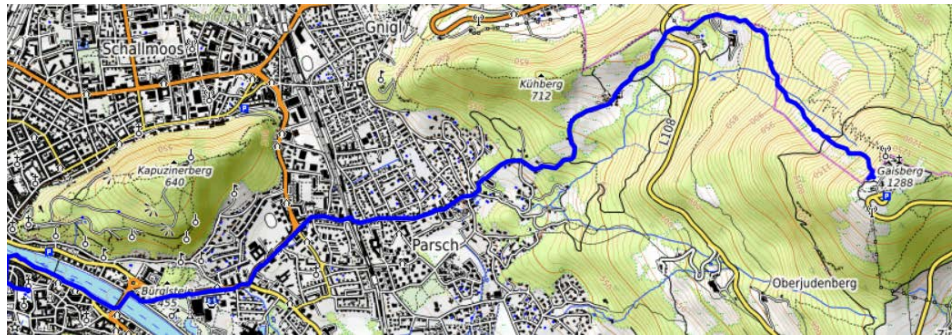
Figure 11.2.: Optimum path from Mozartplatz (TP0) to Gaisberg (TP1)

The first part of the route in this race is mandatory, which leads to a slightly different route shown in figure 11.3. To achieve similar results within these calculations a waypoint (shown figure 11.3 (b) in green) was inserted between Mozartplatz and Gaisberg.

The GPX-Tracklog shown in figure 11.3 (a) as blue line was logged by Aaron Durogati during the Red Bull Xalps 2021 with his Suunto 9 Baro sports-watch. During the last part he hiked off-road straight up a grassy slope which resulted in for a $600m$ shorter route than the one calculated along the road-network. Considering the $600m$ longer distance, the used parameters lead to a very realistic result with an estimated time of $1 : 21[h : mm]$ compared to the logged $1 : 16[h : mm]$. Nevertheless, a difference can be seen in the ascent and descent values. One explanation would be that the Suunto 9 Baro is averaging the barometric altitude and does not count small peak values and thus, the calculated ascent and descent out of a $30m \times 30m$ raster (explained in section 8.6) sums up to a higher value. The absolute altitude difference with $845m - 3m = 842m$ compared with $890.12m - 38.96m = 851.16m$ has an error margin of less than 2%. Those statistics are shown in table 11.4.

Graph 11.4 shows the different routes between Gaisberg and Wagrain-Kleinarl when restrictions were set for Aaron Durogati. Table 11.5 shows

11. Test Cases



(a) GPX-Tracklog of Aaron Durogati during the Red Bull Xalps 2021



(b) Calculated route

Figure 11.3.: Comparison of real tracklog vs. calculated path

	Tracklog	Calculation
Ascent [m]	845	890,12
Descent [m]	3	38,96
Dist. [km]	6,96	7,58
Time [hh:mm]	01:16	01:21

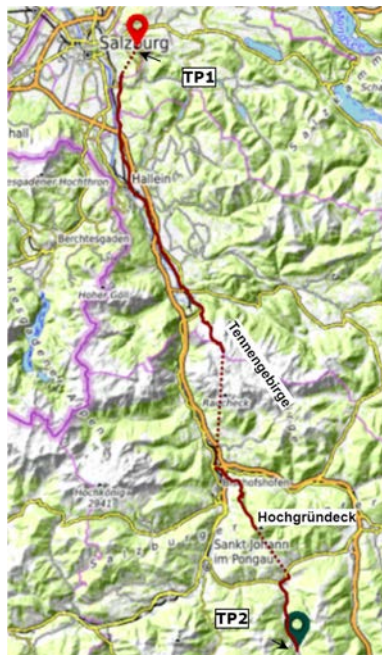
Table 11.4.: Statistics tracklog vs. calculation

the statistics for each route. Without restrictions the optimal route would take 09 : 56 [hh : mm] and cover 51.24 km walking, displayed as solid line, and 20.19 km flying a paraglider, displayed as a dotted line. Three different take-offs would be used: first direction NW, second NE and finally N. With a total of 3664,5 m ascent, the most altitude gain would be made compared to the optimised routes with set restrictions. But as it is supposed to be flyable,

only 510,38 *m* would be covered hiking down-hill. This concludes that more than 3000 *m* can be flown down with a paraglider. Compared to that, in the case of unflyable weather conditions 2419,82 *m* ascent and 2738,71 *m* descent would have to be covered instead. The reason for a higher descent value than ascent lies within the fact that the first turnpoint at Gaisberg lies higher, with an elevation of 1288 m, than Wagrain-Kleinarl at 1010 m. When hiking only, it would take two hours and eleven minutes longer than during flyable weather conditions. Considering strong northern winds, the flight passage down from Tennengebirge (shown as dotted line in the left upper image of figure 11.4) would not be suggested and the route would lead around the mountain range through Pass Lueg (can be seen on the upper right image of figure 11.4). Strong northern winds would make a take-off impossible from the Tennengebirge and only the second flight down from Hochgründeck (to be seen as dotted line next to St. Johann im Pongau of figure 11.4) would be safe.

As a result, this route would take one hour and 15 minutes longer and would cover twelve kilometres more of walking. The lower right graph shows the optimized route if only official take-offs and landings were to be used. The only official take-off available is located at Gaisberg and consequently, a flight is only possible there. This clearly shows that by sticking to official areas a race like the Red Bull Xalps would become an ultra-trail rather than a paragliding competition.

11. Test Cases



(a) no restrictions



(b) with 20km/h north wind



(c) unflyable



(d) Only official take-offs and landings

Figure 11.4.: Optimum path from Gaisberg (TP1) to Wagrain-Kleinarl (TP2)

	No Restrictions	Northwind	Unflyable	Only official take-offs
Ascent [m]	3664,5	2265,59	2419,82	2418,70
Descent [m]	510,38	857,96	2738,71	1906,87
Distance on Foot [km]	51,24	63,97	75,79	71,64
Flown Distance [km]	20,19	16,04	-	-
Take-off Direction(s)	NW NE N	N N	-	NW
Time [hh:mm]	09:56	11:11	12:07	11:51
Straight Distance [km]	60,96	60,96	60,96	60,96

Table 11.5.: Statistics, no restrictions vs. restrictions

11. Test Cases

As good weather cannot be guaranteed at an outdoor race like the Red Bull Xalps, it has also happened, that athletes had to hike the entire distance from Gaisberg to Wagrain. In order to optimize routes for athletes during those unflyable conditions, the varying hiking speed of athletes must be taken into consideration (as shown in 11.5). The influence of the different cost parameters can also be seen in table 11.6.

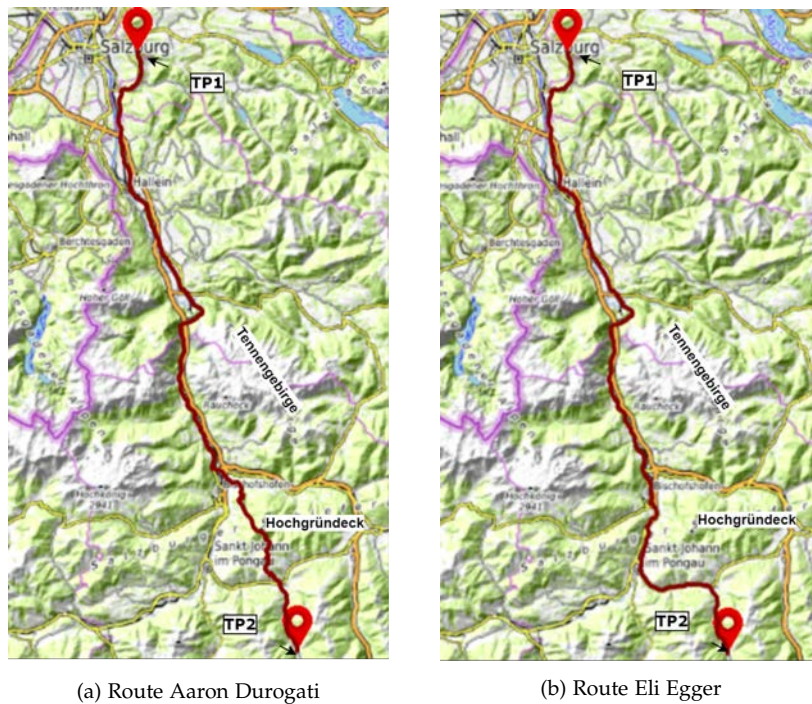


Figure 11.5.: Optimum path from Gaisberg (TP1) to Wagrain-Kleinarl (TP2) if it is unflyable

Seen in the parameters in table 11.2, the required time for crossing the flats is smaller than when hiking uphill. The time, a person like me needs for five kilometres and 1000 m vertical climb, doubles compared to the time, an athlete like Aaron Durogati needs. This leads to the conclusion that it is faster for me, to hike around Hochgründeck (the mountain east of St. Johann im Pongau) compared to the suggestion for Aaron Durogati to climb the mountain and follow the most direct route. About 900 m of elevation gain are saved by me but I have to cover two kilometres of additional distance.

11. Test Cases

	Eli Egger	Aaron Durogati
Ascent [m]	1532,17	2419,82
Descent [m]	1841,26	2738,71
Distance on Foot [km]	77,78	75,79
Flown Distance [km]	-	-
Take-off Direction(s)	-	-
Time [hh:mm]	15:15	12:07
Straight Distance [km]	60,96	60,96

Table 11.6.: Statistics unflyable routes

The time needed for reaching Wagrain-Kleinarl still is 15 : 15 [hh : mm] for me in comparison to only 12 : 07 [hh : mm] for Aaron Durogati.

11.2. Out And Return

The second routing option addresses the task given in a Bordairrace competition, namely: to find the furthest point in reach that still allows you to return to the starting point in time. During a real Bordairrace, competitors get a total of 33 hours for hiking and flying. As the distance covered in these competitions, however, exceeds the extent of the test area, only a time frame of two and a half hours is allotted for out-and-return-maneuvers in this testcase. Starting at Gründeck, a peak west of Kleinarl, the furthest reachable point lies up north as shown in figure 11.6.

This furthest reachable point within two and a half hours is 8,14 km line of sight distance away from the starting point as shown in table 11.7. Looking at the result for Mitterhofen, the starting point lies in the valley and the optimum path would be to hike up to Schmittenhöhe and then glide back down. But it can also be the case that it is better to stay on the ground (see in the lowest figure of 11.6) and cover as much distance as possible by hiking. This can be the case, if there is no suitable take-off around or a long detour has to be done to reach it. In both cases in which flying is suggested, the hiking route is fairly straight. Accordingly, it is crucial that the hiking path to the take-off is as direct as possible.

11. Test Cases

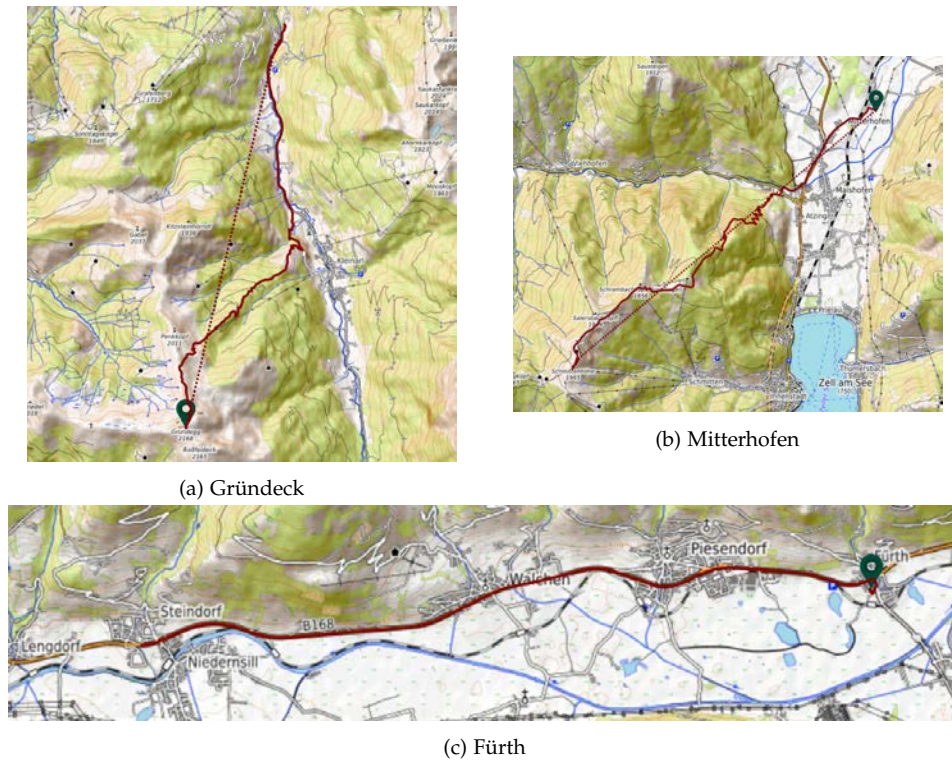


Figure 11.6.: Best options for out and return

Table 11.7, however, also shows that with the use of a paraglider, a longer distance can be covered compared to when hiking only. The straight distance is still half a kilometre shorter when hiking only than when flying as well. However the allowed time frame is used to its full capacity in all three cases, no matter if only hiking or flying as well.

11.3. Take-Offs Around

The third testcase deals with the situation of a paragliding pilot visiting a new place and wanting to know all the take-off options in this area. Depending on the pilot's skill level it can only include official take-offs and landings or all possible take-offs. The altitude difference can also be

11. Test Cases

	Gründeck	Mitterhofen	Fürth
Ascent [m]	1359,3	1343,21	171,61
Descent [m]	125,95	146,05	171,61
Distance on Foot [km]	11,3	11,38	16,36
Flown Distance [km]	7,7	8,03	-
Take-off Direction(s)	S	NE	-
Time [hh:mm]	02:29	02:29	02:29
Straight Distance [km]	8,14	8,27	7,66

Table 11.7.: Statistics Out-and-Return

a criterion as a flight with 1000 m altitude difference is often a better experience than one with just 400 m. Given the situation that the mountain peaks have already been covered in snow, it is sometimes necessary to find only those take-offs that are below a certain elevation. Finally, the take-off direction is crucial whenever the weather forecast predicts more than three kilometres per hour of wind. Without any restrictions set, 764 take-offs would be reachable for Aaron Durogati within two hours, starting at the landing area in Dorfgastein (see figure 11.7). But only four of them are official take-offs, which is less than one percent. In order to reach 138 out of these 764 take-offs, a flight in between would have to be made, which can be seen as dotted lines in the graph 11.7. This graph openly shows that there is a huge number of options within reach that most pilots would never think of. To further narrow down the results, a minimum altitude difference of 1000 m was set in the next step. As a take-off facing south is most likely to have good thermal activity, this information is combined with previous calculations and the number of options can be reduced to 16 (as shown in figure 11.8). Most of those options are situated around the peaks Haßeck, Fulseck, Tagkopf, and Mooseckhöhe while one option is the mountain saddle leading up to Laderdinger Gamskarspitz.

In some situations, time for a hike and fly venture is limited, hence, it is important to get routing results which are doable within a short amount of time (see figure 11.9). In this example we only wanted to receive official take-offs as results that are manageable in two hours out and return. As the official take-offs at Fulseck are close to each other, the needed time for the hike and fly for each take-off is between one hour and 41 minutes and one

11. Test Cases

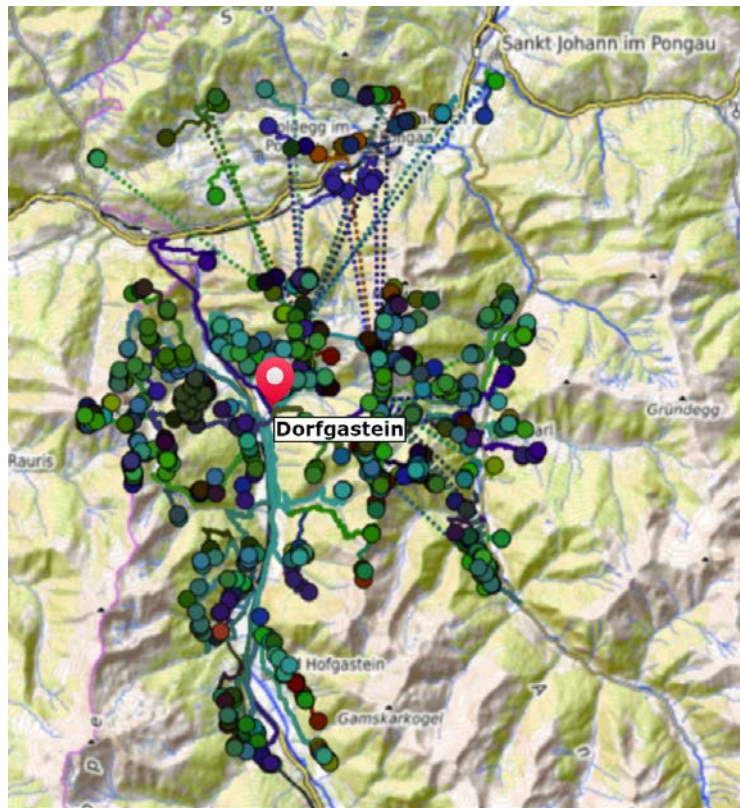


Figure 11.7.: Reachable take-offs around the landing field in Dorfgastein within two hours.

hour and 43 minutes.

11. Test Cases



Figure 11.8.: Take-off options around the landing field in Dorfgastein within two hours, an altitude difference of more than 1000m and a north west facing take-off.



Figure 11.9.: Possible hike and fly options around the landing field in Dorfgastein within two hours. Only official take-offs are chosen.

12. Conclusio And Outlook

This master's thesis aimed to investigate the different routing possibilities within paragliding hike and fly ventures. Varying needs within the paragliding community were identified. First, the need for automatic take-off detection was addressed, by calculating possible take-offs out of a DEM and a land use raster. Second, the possible flight paths were added to an OSM network and the data manipulated so that the network would be usable for routing purposes that include hiking as well as paragliding. Third, a cost function was developed which allows the user to get individualised results for route optimization. The possibility to set restrictions was also included to address different pilot skills and weather situations. This resulted in a prototype of a webpage which gives the users the option to perform individualised routing searches (from A to B, Out and Return maneuvers, or search for take-offs in the area).

All of this has shown that it is overall possible to calculate take-offs out of the given data. It has also shown, however, that in order to receive accurate results, the underlying data must be up to date. For example, grassy slopes in 2016 may be areas of young forest by 2022, which would make them unsuitable for taking-off today. This thesis further shows that it is hard to set the criteria for landing fields, since especially landing in small places is one of the most difficult parts of flying a paraglider. Amateurs may set the criteria so that only big and flat fields are taken into account, whereas Red Bull Xalps athletes would only need a small parking area, an empty street, or a little garden to meet fitting landing conditions.

This thesis has also shown that it is possible to create a network which not only includes paths on the ground but also paths through the air for calculating optimized routes. As you can literally fly in any direction those aerial paths are incredibly numerous in such a network. This huge number, unfortunately, is a limiting factor for the extend of the investigated area as the computational power grows exponentially with the number of edges in

a network.

The invented cost function additionally proves, that with the right parameters, the calculation results are in accordance with the time needed in real life. As mentioned before, individualised cost parameters can lead to results fitting the user. For flying a paraglider safely, different surrounding conditions must be fulfilled. Therefore, the option of setting restrictions is essential as flying may not be an option at all on some days and glidepaths would need to be avoided in the route calculation during that time. Finally, the three invented routing tasks cover a huge range of needs for professional competitors as well as for amateur pilots. The developed prototype of a webpage offers an easy way of planning the next adventure for everyone.

As a next step, live data should be used for accurate weather information when developing future routing tools for paragliding pilots. Additionally, the actual gliding range varies as well due to different wind situations and could be modelled more accurately rather than setting a fixed value when calculating the reachable landings around a take-off. Calculating take-offs based on the ideas of this master thesis, is a very time and computing power consuming process. For commercial use, hence, a faster way would need to be developed. This could be achieved, for example, by extracting take-offs out of existing tracklogs. In hindsight, with this method it is only possible to find places where already someone took off but no new take-offs could be found.

Concluding it can be said, that routing within paragliding hike and fly is possible and route optimization may lead to better race results in the future.

Bibliography

- [1] RedBullMediaHouse. *Route Red Bull X-Alps 2021*. 2021. URL: <https://www.redbullxalps.com/the-race/route.html> (cit. on p. 3).
- [2] Norbert Renner and contributors. *BRouter-Web*. 2018. URL: <https://brouther.de/brouther-web/> (cit. on p. 6).
- [3] AVS DAV ÖAV. *Alpenvereinaktiv*. 2020. URL: <https://www.alpenvereinaktiv.com/de/tourenplaner/> (cit. on p. 6).
- [4] Daniel Arndt. *Hike and Fly planer for paragliders*. 2021. URL: <https://www.parange.ch> (cit. on p. 6).
- [5] Craig Payne. *Paragliding Map*. 2021. URL: <https://www.paraglidingmap.com/app/> (cit. on p. 6).
- [6] DIN Deutsches Institut für Normung e. V. *DIN EN 926-2 Ausrüstung für das Gleitschirmfliegen - Gleitschirme - Teil 2: Anforderungen und Prüfverfahren zur Klassifizierung der sicherheitsrelevanten Flugeigenschaften; Deutsche Fassung EN 926-2:2013*. 2013. URL: <https://www.din.de/de/mitwirken/normenausschuesse/nasport/veroeffentlichungen/wdc-beuth:din21:233581971> (cit. on p. 9).
- [7] DHV. *Prüfungsfragen Paragleitfluglehrer*. 2019 (cit. on p. 9).
- [8] Flugschule Aufwind. *AUSBILDUNG ZUM PARAGLEITER PILOTEN LERNUNTERLAGE*. 2020 (cit. on pp. 10, 12).
- [9] Flybubble. *Speed to fly basics*. 2017. URL: <https://flybubble.com/blog/speed-to-fly-basics> (cit. on p. 9).
- [10] Österreichischer Nationalrat. "Bundesgesetz vom 2. Dezember 1957 über die Luftfahrt (Luftfahrtgesetz – LFG)." In: *Bundesgesetz vom 2. Dezember 1957 über die Luftfahrt (Luftfahrtgesetz – LFG)*. 2021 (cit. on p. 11).

- [11] Innovation und Technologie und Bundesministers für Landesverteidigung und Sport Bundesministers für Verkehr. *Luftverkehrsregeln*. 2014. URL: <https://www.ris.bka.gv.at/GeltendeFassung.wxe?Abfrage=Bundesnormen&Gesetzesnummer=20008992> (cit. on p. 11).
- [12] Myril B. Reed Lee M. Maxwell. *The Theory of Graphs: A Basis for Network Theory*. 1971 (cit. on pp. 13, 14).
- [13] Leonhard Euler. "Solutio problematis ad geometriam situs pertinentis." In: *Euler Archive - All Works* (1741) (cit. on p. 13).
- [14] M. Wieser B. Hofmann-Wellenhof K. Legat. *Navigation*. 2003 (cit. on pp. 15, 17, 22).
- [15] E. W. Dijkstra. "A note on two problems in connexion with graphs." In: *Numerische Mathematik* (1959) (cit. on p. 17).
- [16] Ronald L. Rivest Thomas H. Cormen Charles E. Leiserson and Clifford Stein. *Introduction To Algorithms*. 2003 (cit. on p. 20).
- [17] Open Data Austria. *Digitales 10m - Geländemodell aus Airborne Laserscan Daten in der Projektion EPSG:31287 (Lambert)*. 2018. URL: http://gis.ktn.gv.at/OGD/Geographie_Planung/ogd-10m-at.zip (cit. on p. 26).
- [18] Geoland.at. *Geoland.at*. 2018. URL: <http://www.geoland.at> (cit. on p. 26).
- [19] Umweltbundesamt GmbH. *High Resolution Land Cover Map*. 2017. URL: data.umweltbundesamt.at (cit. on pp. 27, 28).
- [20] OpenStreetMap Foundation. *Open Street Map*. 2021. URL: <https://www.openstreetmap.org> (cit. on p. 29).
- [21] Open Knowledge Foundation. *Open Data Commons*. 2021. URL: www.opendatacommons.org (cit. on p. 29).
- [22] Geofabrik GmbH. *Geofabrik Download Austria*. 2020. URL: <https://download.geofabrik.de/europe/austria.html> (cit. on p. 29).
- [23] DHV. *DHV Geländedatenbank*. 2021. URL: <https://www.dhv.de/piloteninfos/gelaende-luftraum-natur/fluggelaendeflugbetrieb/gelaendedaten/gelaendedaten-download/> (cit. on p. 29).
- [24] DHV. *Nutzungsbedingungen*. 2020. URL: https://www.dhv.de/fileadmin/user_upload/dbfiles/gelaendedaten/nutzung_gelaendedb.pdf (cit. on p. 29).

- [25] Austro Control GmbH. *Luftraumstruktur*. 2020. URL: https://www.austrocontrol.at/piloten/vor_dem_flug/aim_produkte/luftraumstruktur (cit. on p. 30).
- [26] Nationalparks Austria. *Nationalpark Hohe Tauern Aussengrenze (11/2018) auf Basis der Zonierung 11/2018 EPSG:4326 - WGS 84 - Geographic*. 2018. URL: <https://www.data.gv.at/katalog/dataset/nationalpark-hohe-tauern-aussengrenze-11-2018> (cit. on p. 32).
- [27] Land Salzburg. *Die Stichtagsdaten der Verwaltungsgrenzdatenbank (VGD) "Salzburg" beinhalten die offiziellen Grenzen der Verwaltungseinheiten des Landes Salzburg in den Maßstäben 1:50 000 und 1:250 000 (jeweils generalisiert) sowie die grundstücksgenaue Verwaltungsgrenzen*. 2020. URL: <https://www.data.gv.at/katalog/dataset/3d9514ef-7f1e-47f9-a9bd-e6bc2ce4af58> (cit. on p. 33).
- [28] ESRI. *ArcGIS Pro*. 2021. URL: <https://www.esri.com/en-us/arcgis/products/arcgis-pro/overview> (cit. on p. 34).
- [29] OSGeo. *QGIS*. 2021. URL: <https://qgis.org/en/site/> (cit. on p. 34).
- [30] Microsoft. *Visual Studio Code*. 2021. URL: <https://code.visualstudio.com/> (cit. on p. 35).
- [31] Python Software Foundation. *python*. 2021. URL: <https://www.python.org/> (cit. on p. 36).
- [32] PostgreSQL Global Development Group. *PostgreSQL*. 2021. URL: <https://www.postgresql.org/> (cit. on p. 36).
- [33] Dipl.-Ing. (FH) Stefan Luber. "Definition Was ist eine relationale Datenbank?" In: *Big Data Insider* (2017) (cit. on p. 36).
- [34] PostGIS Project Steering Committee. *PostGIS*. 2021. URL: <https://postgis.net/> (cit. on p. 37).
- [35] Inc. Free Software Foundation. *GNU General Public License version 2*. 1991. URL: <https://opensource.org/licenses/gpl-2.0.php> (cit. on p. 37).
- [36] pgRouting Community. *pgRouting*. 2021. URL: <https://pgrouting.org/> (cit. on p. 37).
- [37] Dr. Jeffrey Jaffe Tim Berners-Lee. *World Wide Web Consortium (W3C)*. 2021. URL: <https://www.w3.org/Consortium/> (cit. on p. 37).

Bibliography

- [38] Till Adams Marc Jansen. *OpenLayers*. 2010 (cit. on p. 38).
- [39] IBM OpenJS Foundation StrongLoop et al. *Express*. 2017. URL: <https://expressjs.com/> (cit. on p. 39).
- [40] Joysent Inc. OpenJS Foundation. *Node.js*. 2021. URL: <https://nodejs.org> (cit. on p. 39).